



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE
Corso di Laurea Triennale in Informatica

Un workflow LLM per la creazione interattiva di basi di dati: modulo converter

CANDIDATO:
Fabio Meloni

RELATORE:
Prof. Vincenzo Bonnici

*Alla mia famiglia,
a coloro che hanno riposto fiducia nelle mie capacità
e a chi mi ha accompagnato con costanza nel corso di questi tre anni.*

Indice

1	Introduzione	1
1.1	Motivazioni ed obiettivo	3
1.2	Struttura	4
2	Nozioni di base	5
2.1	Frontend	5
2.1.1	Design Pattern MVP	5
2.1.2	jQuery	7
2.1.3	HighlighterJS	8
2.1.4	Markdown-it	9
2.1.5	SheetJS	10
2.1.6	Bootstrap	10
2.2	Backend	11
2.2.1	Comunicazione Client-Server	11
2.2.2	Comunicazione Chunked	13
2.2.3	Design Pattern Gateway	17
2.2.4	MongoDB	19
2.2.5	SimpleXLSX	23
2.2.6	Bearer Token	24
2.2.7	Query Safety	26
2.2.8	Ordinamento Topologico	30
2.3	Backend LLM	33
2.3.1	Ollama	33
2.3.2	Design Pattern Agentic	34
2.3.3	Qwen2.5-Coder-7b-Instruction	37
2.3.4	Pydantic Models	39
2.3.5	Pandas	41
2.3.6	Prompt Engineeering	42

3	Stato dell'Arte	45
3.1	Strumenti Excel-to-SQL e motori di trasformazione di fogli di calcolo	45
3.2	LIMS open source e gestione strutturata dei campioni	46
3.3	ELN e piattaforme per la gestione degli esperimenti	47
3.4	Confronto delle soluzioni con il software in sviluppo	48
3.5	GMOD: un ecosistema modulare per la gestione di dati biologici	51
3.5.1	Confronto con il software in sviluppo	51
3.6	Valutazione complessiva	52
4	Metodologia	53
4.1	Docker	55
4.1.1	Configurazione Apache	57
4.1.2	Configurazione MariaDB	59
4.1.3	Configurazione MongoDB	60
4.1.4	Configurazione Ollama	61
4.1.5	Configurazione dell'ambiente Python	61
4.2	Backend LLM	61
4.2.1	Server Flask e registrazione delle route	64
4.2.2	Modelli Pydantic e contratti di risposta	64
4.2.3	ChatEngine: orchestrazione del ciclo di chat	66
4.2.4	Gestione dei file e caching degli artefatti	71
4.2.5	Tooling e ToolManager	74
4.2.6	Estrazione e validazione del JSON	76
4.2.7	Integrazione con il client Ollama	77
4.3	Gestione Utenti	78
4.3.1	Login	80
4.3.2	Configurazione Utente Admin	84
4.3.3	Creazione Utenti	87
4.3.4	Gestione Permessi Utente	91
4.3.5	Modifica Utenti	94
4.3.6	Gestione Gruppi Utenti	97
4.4	Gestione Categorie	100
4.4.1	Creazione Categorie	104
4.4.2	Associazione Categorie-Transazione	106
4.4.3	Modifica Categorie	108
4.5	Gestione Transazioni	111
4.5.1	Creazione Transazioni	115
4.5.2	Gestione Permessi	124
4.5.3	Modifica Informazioni Transazione	126
4.5.4	Creazione Vetrine Transazioni	128

4.6 Gestione Logging	130
5 Risultati	135
6 Conclusioni	137
Bibliografia	139

Elenco delle figure

2.1	Schema rappresentante i tre principali pattern architetturali. .	7
2.2	Schema generale di un'architettura client-server con frontend e backend applicativo.	11
2.3	Esempio di comunicazione tramite API RESTful.	12
2.4	Interazione tra backend applicativo e backend LLM tramite API RESTful.	13
2.5	Architettura multilivello con frontend, backend applicativo e backend LLM.	13
2.6	Evoluzione del supporto alla comunicazione chunked nelle versioni di HTTP.	15
2.7	Schema del funzionamento del chunked transfer encoding.	16
2.8	Flusso di comunicazione chunked tra server e client.	17
2.9	Flusso interno del Controller tra logica applicativa e servizi. .	18
2.10	Diagramma del pattern Gateway-Controller.	19
2.11	Esempio di documento BSON basato sulla struttura della collection bioCMS.	21
2.12	Replica set in MongoDB.	21
2.13	Esempio di funzionamento dello Sharding.	22
2.14	Richiesta HTTP con autenticazione tramite Bearer Token. . .	24
2.15	Schema del processo di validazione del Bearer Token da parte del server di risorse.	26
2.16	Schema architetturale dell'agente LLM e dei sottosistemi di orchestrazione.	36
2.17	Esempio di reinforcement learning from human feedback (RLHF). .	39
2.18	Struttura dei prompt Qwen2.5 basata su token speciali e turni conversazionali.	43
3.1	Posizionamento del software proposto rispetto a strumenti Excel-to-SQL e LIMS/ELN.	49
3.2	Schema del flusso di lavoro del software in analisi.	50
4.1	Schema del flusso di lavoro del software in analisi.	54

4.2	Diagramma di distribuzione con Ollama installato nel container.	56
4.3	Diagramma di distribuzione con Ollama installato sull'host.	57
4.4	Schema ER del database bioCMS_db.	60
4.5	Diagramma delle classi backed LLM.	63
4.6	Diagramma di sequenza del ciclo di chat orchestrato dal ChatEngine.	70
4.7	Diagramma della gestione dei file e dei tool associati.	73
4.8	Diagramma dei casi d'uso relativi alla gestione utenti.	79
4.9	Diagramma delle classi gestione utenti.	80
4.10	View login utente.	81
4.11	Diagramma di sequenza del processo di login.	84
4.12	View configurazione utente admin.	85
4.13	Diagramma di sequenza della configurazione utente admin.	87
4.14	View creazione utente.	88
4.15	Diagramma di sequenza della creazione utenti.	91
4.16	View modifica permessi utente.	92
4.17	Diagramma di sequenza della gestione permessi utente.	94
4.18	View modifica utente.	95
4.19	Diagramma di sequenza della modifica utenti.	97
4.20	View gestione gruppi.	98
4.21	Diagramma di sequenza della gestione gruppi.	100
4.22	Diagramma dei casi d'uso relativi alla gestione categorie.	101
4.23	Diagramma delle classi backend gestione categorie.	102
4.24	Diagramma delle classi frontend gestione categorie.	103
4.25	View creazione categorie.	104
4.26	Diagramma di sequenza processo di creazione di una categoria.	105
4.27	View selezione categorie.	107
4.28	Diagramma di sequenza associazione categorie-transazione.	108
4.29	View modifica categorie.	109
4.30	Diagramma di sequenza processo di modifica di una categoria.	110
4.31	View lista transazioni.	111
4.32	Diagramma dei casi d'uso relativi alla gestione transazioni.	112
4.33	Diagramma delle classi frontend gestione transazioni.	113
4.34	Diagramma delle classi backend gestione transazioni.	114
4.35	View creazione transazioni.	117
4.36	Reasoning prodotto dal modello linguistico.	119
4.37	Diagramma di sequenza processo di creazione di una transazione.	122
4.38	View gestione permessi transazioni.	125
4.39	View deprecazione transazioni.	127
4.40	View creazione vetrine.	129
4.41	Diagramma delle classi logging.	133

4.42	Diagramma di sequenza logging.	134
5.1	Tempi di elaborazione per dataset di dimensioni crescenti. . .	136

Elenco degli algoritmi

1	Implementazione Funzione Safe Query.	27
2	Pseudo-Codice Ordinamento Topologico.	30
3	Implementazione Ordinamento Topologico.	31
4	Implementazione Verifica Sicurezza Password.	89

Elenco delle tabelle

2.1	Confronto tra agenti deliberativi, reattivi e agenti LLM secondo il paradigma AI Agent.	35
3.1	Confronto tra il software sviluppato e le principali categorie di strumenti affini.	48
3.2	Confronto tra GMOD e il software sviluppato.	52
4.1	Descrizione delle variabili d'ambiente utilizzate.	56
4.2	Permessi attivabili sulle transazioni.	126

Capitolo 1

Introduzione

Nel settore della microbiologia, una delle attività fondamentali consiste nella raccolta, gestione e conservazione dei dati sperimentali. La crescente complessità delle analisi e l'aumento del numero di esperimenti condotti quotidianamente nei laboratori hanno reso evidente quanto la qualità dei dati e la loro organizzazione influenzino direttamente l'affidabilità dei risultati scientifici. In questo contesto, la trasformazione dei dati da formati non strutturati e semi-strutturati, come quelli tipicamente prodotti dagli strumenti di laboratorio o raccolti manualmente dai ricercatori, in dati strutturati archiviati all'interno di un sistema di gestione di basi di dati, rappresenta un passaggio cruciale per garantire coerenza, tracciabilità e interoperabilità delle informazioni. Disporre dei dati in formato strutturato, inoltre, semplifica in modo significativo l'esecuzione di algoritmi di calcolo, analisi statistiche e procedure automatizzate, rendendo più efficiente l'elaborazione dei dataset e favorendo l'integrazione con strumenti computazionali avanzati.

I dati semi-strutturati, spesso organizzati in tabelle, fogli di calcolo o file eterogenei, offrono una certa flessibilità nella fase di raccolta, ma presentano limiti significativi quando si tratta di integrarli in flussi di lavoro complessi o di analizzarli su larga scala. La loro natura ibrida, a metà tra dati completamente non strutturati e dati rigidamente organizzati, richiede strumenti e metodologie in grado di interpretarli, normalizzarli e convertirli in schemi coerenti e interrogabili. La crescente diffusione di approcci computazionali avanzati, unita alla necessità di garantire qualità e riproducibilità, ha reso sempre più importante l'adozione di sistemi che permettano questa transizione in modo affidabile e automatizzato.

Negli ultimi anni, lo sviluppo della bioinformatica ha profondamente trasformato lo studio delle discipline microbiologiche e dei sistemi microbici, introducendo strumenti avanzati per l'analisi e l'archiviazione dei dati che in precedenza non erano disponibili. Tali progressi hanno consentito ai ricerca-

tori di esplorare con maggiore profondità la complessità dei sistemi microbiologici, favorendo l'emergere di nuove prospettive interpretative e metodologiche. Parallelamente, l'evoluzione delle tecnologie analitiche e computazionali ha determinato un incremento significativo del volume e della varietà dei dati generati, rendendo imprescindibile l'adozione di software dedicati alla rappresentazione, gestione e conservazione strutturata delle informazioni.

Sebbene i dati vengano spesso archiviati in fogli di calcolo per facilitarne la condivisione tra diversi ricercatori, l'aumento esponenziale della quantità di informazioni da analizzare e conservare ha evidenziato i limiti di tale approccio e la necessità di sistemi informatici più robusti e scalabili. L'utilizzo di file tabellari, oltre a risultare inefficiente per dataset di grandi dimensioni, presenta numerose criticità: l'impossibilità di garantire l'integrità referenziale dei dati attraverso vincoli relazionali, con il conseguente rischio di generare o mantenere riferimenti a dati inesistenti; l'assenza di un audit trail che consenta di tracciare le modifiche e attribuirle ai rispettivi operatori; la scarsa scalabilità, che rende complessa la gestione di grandi volumi di dati e di gruppi di lavoro numerosi; l'insufficienza dei meccanismi di sicurezza per il controllo degli accessi; le prestazioni limitate nelle operazioni di filtraggio e analisi su dataset estesi; e, infine, la difficoltà nel generare report strutturati e riproducibili da condividere con la comunità scientifica.

1.1 Motivazioni ed obiettivo

In un contesto in cui le tecnologie per la raccolta dei dati assumono un ruolo sempre più centrale e i modelli di intelligenza artificiale vengono progressivamente integrati nei flussi operativi delle applicazioni, l'obiettivo del presente lavoro è la realizzazione di un software che consenta ai ricercatori di convertire dati semi-strutturati in una base di dati strutturata. Tale trasformazione non solo permette di garantire maggiore coerenza e qualità delle informazioni, ma rende anche più agevole l'esecuzione di algoritmi di calcolo, analisi statistiche e procedure automatizzate, che richiedono dati organizzati secondo schemi rigorosi per poter operare in modo efficiente e affidabile. In particolare, il sistema è pensato per supportare coloro che archiviano le proprie informazioni in fogli di calcolo, offrendo loro la possibilità di migrare verso un'infrastruttura più solida, coerente e adatta alla gestione di dataset in continua crescita.

Il software garantirà la persistenza delle informazioni, la gestione degli utenti appartenenti all'organizzazione e dei relativi permessi, nonché l'autenticazione alla piattaforma prima del suo utilizzo. Sarà inoltre implementato un audit trail per assicurare la tracciabilità delle modifiche e la loro attribuzione ai rispettivi operatori. Il sistema prevede anche un componente dedicato alla gestione dei file e un modulo per la creazione di vetrine pubblicabili e condivisibili, basate sui dataset forniti. Tali vetrine potranno essere generate mediante l'impiego di un modello LLM, che consentirà la produzione di grafici a partire da semplici prompt descrittivi forniti dall'utente.

Il software è stato progettato per offrire un'interfaccia user-friendly, ottimizzare i processi di condivisione dei dati tra i ricercatori e supportare le operazioni di modifica e filtraggio dei dataset. Il risultato atteso è uno strumento di facile utilizzo che permetta ai ricercatori di passare dall'impiego di file semi-strutturati a quello di una base di dati tramite un'interfaccia grafica web, disponendo al contempo di un insieme di tool per la gestione dei file e la condivisione dei dati attraverso vetrine generate con il supporto di un modello LLM.

1.2 Struttura

La tesi è strutturata nei seguenti capitoli:

- **Capitolo 1.** Introduzione: presentazione del contesto attuale, delle motivazioni, degli obiettivi e della struttura complessiva del lavoro.
- **Capitolo 2.** Nozioni di base: descrizione delle conoscenze fondamentali necessarie per comprendere la soluzione proposta e le tecniche di programmazione adottate.
- **Capitolo 3.** Stato dell'arte: analisi delle soluzioni attualmente disponibili che affrontano lo stesso problema o problematiche affini.
- **Capitolo 4.** Metodologia: descrizione dettagliata della soluzione sviluppata e degli algoritmi utilizzati.
- **Capitolo 5.** Risultati: presentazione e discussione dei risultati ottenuti.
- **Capitolo 6.** Conclusioni: sintesi dei risultati, individuazione delle criticità ancora aperte e possibili sviluppi futuri della soluzione proposta.

Capitolo 2

Nozioni di base

In questo capitolo vengono presentate le nozioni considerate essenziali per la piena comprensione delle attività descritte nel capitolo Metodologia. Al fine di agevolare la consultazione e garantire una maggiore chiarezza espositiva, i contenuti sono stati organizzati secondo la struttura dei tre moduli che compongono il software: frontend, backend e backend LLM.

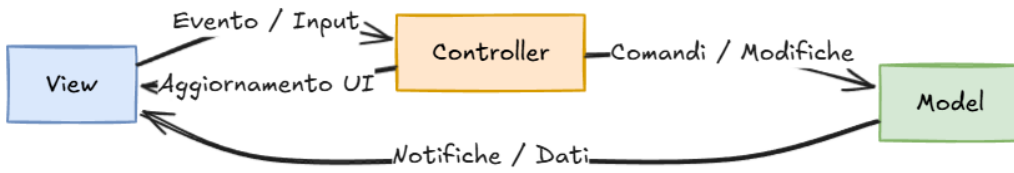
2.1 Frontend

In questa sezione vengono trattate tutte le nozioni di base riferite al frontend.

2.1.1 Design Pattern MVP

Il Model–View–Presenter (MVP) è un pattern architetturale che nasce con l’obiettivo di migliorare la separazione delle responsabilità all’interno delle applicazioni software, in particolare quelle dotate di interfacce grafiche complesse. Derivato dal più noto Model–View–Controller (MVC), il pattern MVP si afferma negli anni Novanta grazie ai lavori sviluppati presso Taligent, dove si cercava un modello più rigoroso per isolare la logica di presentazione dai dettagli dell’interfaccia utente [Garcia, 2023b]. Una rappresentazione sintetica della struttura del pattern è mostrata in Figura 2.1b. Nel contesto di MVP, la struttura dell’applicazione viene suddivisa in tre componenti concettualmente distinte. Il Model rappresenta il nucleo informativo e la logica di business: gestisce i dati, le trasformazioni e l’accesso alle risorse esterne, mantenendo un’indipendenza totale dalla rappresentazione grafica. La View, al contrario, si limita a mostrare le informazioni all’utente e a raccoglierne gli input, senza interpretarne il significato. Essa espone un insieme di metodi che costituiscono un vero e proprio contratto, attraverso cui il Presenter può

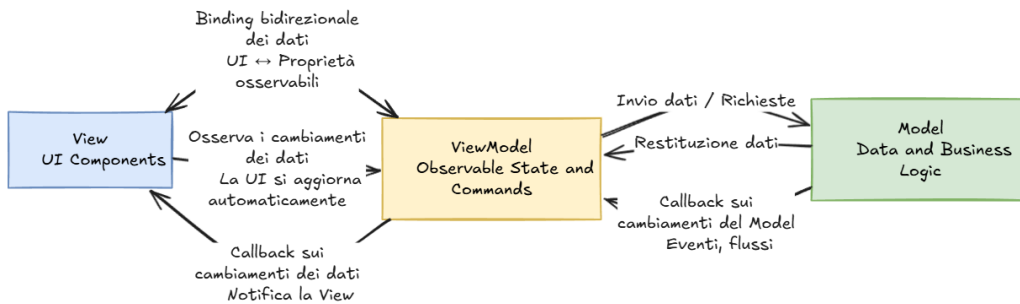
aggiornare lo stato visivo dell'interfaccia [Garcia, 2023a]. Infine, il Presenter funge da mediatore: riceve gli eventi dalla View, li elabora secondo la logica applicativa, interagisce con il Model e restituisce alla View i risultati da visualizzare, come illustrato nello schema di Figura 2.1b. Uno degli aspetti più rilevanti del pattern è la natura passiva della View. A differenza di quanto accade in MVC, la View non contiene logica di presentazione e non decide autonomamente come reagire agli input dell'utente. Tutto il processo decisionale è delegato al Presenter, che diventa così il punto centrale della gestione del flusso informativo. Questa scelta progettuale consente di ottenere un livello di disaccoppiamento molto elevato: la View può essere sostituita, modificata o testata in modo indipendente, senza influire sul comportamento del Model o del Presenter. Dal punto di vista metodologico, MVP si distingue per la sua forte orientazione alla testabilità. Poiché la logica applicativa risiede nel Presenter e la View può essere facilmente simulata tramite mock, è possibile verificare il comportamento dell'applicazione in modo sistematico e automatizzato. Ciò rende il pattern particolarmente adatto a contesti in cui la qualità del software e la manutenibilità nel lungo periodo rappresentano requisiti fondamentali, come nello sviluppo mobile o nelle applicazioni enterprise [Zhang and Luo, 2010]. Il funzionamento complessivo del pattern può essere descritto come un ciclo continuo di interazioni: l'utente compie un'azione sulla View, la View notifica l'evento al Presenter, il Presenter elabora l'informazione e interroga il Model, il Model restituisce i dati aggiornati e il Presenter provvede a rifletterli nella View. Questo flusso lineare e ben definito, rappresentato graficamente in Figura 2.1b, contribuisce a rendere l'architettura prevedibile, modulare e facilmente estendibile. Nel panorama delle architetture software moderne, MVP continua a rappresentare un riferimento importante, soprattutto in quei contesti in cui è necessario mantenere un controllo rigoroso sulla logica di presentazione e garantire un'elevata testabilità. La sua influenza è evidente anche in pattern più recenti, come MVVM (Figura 2.1c), che ne riprendono alcuni principi adattandoli a tecnologie e paradigmi contemporanei.



(a) Schema concettuale del pattern MVC.



(b) Schema concettuale del pattern MVP.



(c) Schema concettuale del pattern MVVM.

Figura 2.1: Schema rappresentante i tre principali pattern architetturali.

2.1.2 jQuery

La libreria jQuery rappresenta uno dei contributi più significativi allo sviluppo del web moderno, grazie alla sua capacità di semplificare in modo radicale l'interazione con il Document Object Model (DOM) e di fornire un'interfaccia coerente tra browser differenti. Introdotta da John Resig nel 2006, jQuery nasce in un contesto caratterizzato da forti incompatibilità tra i motori di rendering e da un crescente bisogno di strumenti capaci di supportare la transizione verso applicazioni web più dinamiche e interattive [Resig, 2011, jQuery Foundation, 2023]. La sua sintassi compatta, basata su selettori e sulla possibilità di concatenare operazioni, ha introdotto un paradigma di programmazione che ha influenzato profondamente l'evoluzione successiva dei framework JavaScript. L'importanza di jQuery non può essere compresa senza considerare il ruolo che essa ha svolto nel superare le difficoltà tecniche del Web 2.0. Prima della sua diffusione, la manipolazione del DOM richiedeva codice prolisso e spesso specifico per ciascun browser. jQuery ha fornito un livello di astrazione

zione uniforme, permettendo agli sviluppatori di concentrarsi sulla logica applicativa piuttosto che sulla gestione delle differenze tra implementazioni [Chaudhary and Kumar, 2017, jQuery Foundation, 2023]. Il suo motore di selezione, Sizzle, ha reso possibile un'interazione più espressiva con la struttura della pagina, anticipando in parte le funzionalità che sarebbero poi state standardizzate nelle API dei browser. Dal punto di vista dell'impatto sul settore, jQuery ha raggiunto una diffusione senza precedenti: secondo analisi recenti, è utilizzata dal 77% dei dieci milioni di siti più popolari, confermandosi come la libreria JavaScript più adottata a livello globale [Wikipedia contributors, 2022]. Tale predominanza è dovuta non solo alla sua semplicità, ma anche alla vasta quantità di codice legacy che continua a dipendere da essa, rendendola un elemento imprescindibile per la manutenzione di numerosi sistemi esistenti. La modularità della libreria ha inoltre favorito la nascita di un ecosistema di plugin che ha ampliato notevolmente le sue capacità, consentendo la creazione di componenti riutilizzabili e di interfacce ricche. La sua influenza è stata tale da portare aziende come Microsoft e Nokia a integrarla nativamente nei propri ambienti di sviluppo e piattaforme software, riconoscendone il valore come strumento standard per la costruzione di applicazioni web [Wikipedia contributors, 2022].

2.1.3 HighlighterJS

La libreria Highlight.js si configura come uno strumento centrale nell'ecosistema della pubblicazione tecnica online grazie alla combinazione di rilevamento automatico della sintassi, ampiezza del supporto linguistico e architettura modulare, elementi che ne hanno favorito una diffusione capillare. Il meccanismo di autodetection consente di identificare il linguaggio del frammento di codice senza intervento dell'autore, riducendo il rischio di errori e garantendo uniformità nell'evidenziazione della sintassi, come documentato nelle specifiche ufficiali [Highlight.js Team, 2024]. Parallelamente, la disponibilità di oltre 190 linguaggi nella versione più recente amplia il raggio d'azione della libreria, rendendola adatta tanto alla documentazione software quanto alla comunicazione scientifica e tecnica [Highlight.js Project, 2024]. La struttura modulare, che permette di includere solo i componenti necessari, contribuisce inoltre a ottimizzare le prestazioni e a facilitarne l'integrazione in sistemi web articolati [Highlight.js Contributors, 2024]. In questo quadro si inserisce anche highlighterJS, che svolge la funzione di motore operativo responsabile dell'analisi e della trasformazione dei blocchi di codice in output evidenziato. Esso interpreta le regole sintattiche associate ai linguaggi supportati, applica gli stili definiti dai temi selezionati e coordina il processo di parsing necessario a generare una rappresentazione coerente e leggibile del codice. Highlighte-

rJS può essere descritto come un componente di astrazione che media tra la struttura formale del linguaggio sorgente e la sua resa visiva, operando come un interprete specializzato che traduce pattern sintattici in marcatori semantici destinati alla visualizzazione. Tale ruolo è essenziale per garantire che l'evidenziazione non sia un semplice abbellimento grafico, ma una vera e propria forma di supporto cognitivo alla comprensione del codice. L'adozione della libreria risulta agevole sia nel browser sia in ambiente Node.js: nel primo caso è sufficiente includere i file CSS e JavaScript e invocare la funzione `hljs.highlightAll()`, mentre nel secondo è possibile importare il modulo tramite `require` o `import`, mantenendo così coerenza d'uso tra contesti client-side e server-side. Questa flessibilità ha favorito l'integrazione di Highlight.js in numerosi progetti open source.

2.1.4 Markdown-it

La libreria markdown-it rappresenta uno dei parser Markdown più completi e performanti disponibili nell'ecosistema JavaScript contemporaneo. Progettata con l'obiettivo dichiarato di offrire un'implementazione veloce, estensibile e conforme agli standard [markdown-it Contributors, 2024], essa si distingue per la capacità di coniugare rigore sintattico, modularità e un'elevata compatibilità con le specifiche formali del linguaggio Markdown. Il progetto si fonda infatti sul rispetto delle CommonMark Specifications, un insieme di regole definite per garantire coerenza e interoperabilità tra differenti parser [CommonMark Working Group, 2024]. Tale aderenza costituisce un elemento essenziale per l'impiego della libreria in contesti complessi, nei quali la prevedibilità del rendering testuale assume un ruolo centrale. Dal punto di vista architetturale, markdown-it adotta un modello di parsing a più livelli, basato su una pipeline modulare che consente di intervenire su ciascuna fase del processo di trasformazione del testo. Questa struttura permette di estendere il comportamento del parser attraverso plugin, i quali possono introdurre nuove sintassi, modificare regole esistenti o integrare funzionalità avanzate come la gestione delle formule matematiche, delle tabelle estese o dei diagrammi. Tale estensibilità è stata concepita per favorire l'integrazione della libreria in sistemi complessi, incluse piattaforme di pubblicazione scientifica e ambienti di sviluppo collaborativo [markdown-it Team, 2024]. Un ulteriore elemento distintivo è rappresentato dall'attenzione alle prestazioni. Benchmark indipendenti riportati dagli sviluppatori mostrano come markdown-it raggiunga velocità di parsing significativamente superiori rispetto a molte alternative, pur mantenendo un'elevata fedeltà semantica [markdown-it Contributors, 2024]. Questa caratteristica risulta particolarmente rilevante nei contesti in cui il rendering deve avvenire in tempo reale,

come nelle applicazioni web interattive o negli editor WYSIWYG basati su Markdown.

2.1.5 SheetJS

La libreria SheetJS rappresenta uno dei toolkit JavaScript più completi e consolidati per la gestione di dati in formato spreadsheet. La sua architettura è progettata per offrire un'interfaccia unificata a un'ampia gamma di formati, dai moderni standard XML utilizzati da Microsoft Excel Dal punto di vista funzionale, SheetJS offre un insieme articolato di strumenti per il parsing, la manipolazione e la generazione di workbook. Le API principali consentono di leggere e scrivere file tramite funzioni dedicate. Queste funzioni sono progettate per operare in ambienti eterogenei, dai browser ai server Node.js, fino a contesti più specifici come estensioni per Adobe o applicazioni basate su Deno [SheetJS Team, 2026]. Questa portabilità rende la libreria particolarmente adatta a sistemi distribuiti e applicazioni web complesse che richiedono un'elaborazione affidabile dei dati tabellari. Un ulteriore elemento distintivo è la capacità della libreria di operare su strutture dati ad alto livello, come array di oggetti o tabelle HTML, convertendole automaticamente in fogli di calcolo. Tale approccio semplifica l'integrazione con framework moderni e consente di automatizzare processi di importazione ed esportazione dei dati [SheetJS Contributors, 2024].

2.1.6 Bootstrap

Il framework Bootstrap rappresenta uno dei sistemi di progettazione frontend più influenti e diffusi nel panorama dello sviluppo web contemporaneo. Nato come progetto interno di Twitter, esso fu sviluppato da Mark Otto e Jacob Thornton con l'obiettivo di uniformare l'interfaccia grafica dei numerosi strumenti interni dell'azienda, riducendo incoerenze e costi di manutenzione [Wikipedia contributors, 2025]. La prima versione pubblica, rilasciata il 19 agosto 2011, segnò l'inizio di una rapida diffusione che portò il progetto a diventare, nel giro di pochi anni, uno dei repository GitHub più apprezzati e forkati a livello globale [Wikipedia contributors, 2025]. La documentazione ufficiale conferma come il framework sia stato concepito per fornire un insieme coerente di componenti, stili e strumenti capaci di accelerare la creazione di interfacce web moderne e responsive [Bootstrap Team, 2025]. Dal punto di vista tecnico, Bootstrap si fonda su un sistema modulare che integra HTML, CSS e JavaScript, offrendo una vasta gamma di componenti predefiniti — tra cui pulsanti, moduli, sistemi di navigazione e layout a griglia — progettati per garantire coerenza visiva e facilità di utilizzo. L'introduzione del grid

system a dodici colonne, formalizzata con Bootstrap 2, ha rappresentato un punto di svolta nella progettazione responsive, consentendo agli sviluppatori di adattare le interfacce a dispositivi con differenti dimensioni dello schermo [Wikipedia contributors, 2025]. Le versioni successive hanno ulteriormente raffinato l'architettura del framework: Bootstrap 3 ha introdotto un approccio mobile-first, mentre Bootstrap 4 ha segnato la migrazione verso Sass e l'adozione estesa del modello flexbox. Con Bootstrap 5, il progetto ha abbandonato la dipendenza da jQuery, adottando JavaScript nativo e tecnologie più moderne come le CSS custom properties [Bootstrap Team, 2025].

2.2 Backend

In questa sezione vengono trattate tutte le nozioni di base riferite al backend.

2.2.1 Comunicazione Client-Server

Il modello di comunicazione client-server rappresenta un paradigma fondamentale dei sistemi distribuiti e descrive un'interazione strutturata in cui un'entità client invia richieste a un'entità server, che elabora tali richieste e restituisce una risposta. Nelle architetture web moderne, tale paradigma si articola in configurazioni multilivello che consentono di separare responsabilità, ottimizzare le prestazioni e integrare componenti eterogenei, inclusi sistemi avanzati basati su Large Language Models (LLM). Una soluzione composta da un frontend, un backend applicativo e un backend LLM riflette questa evoluzione, introducendo una stratificazione funzionale che permette di orchestrare flussi informativi complessi.

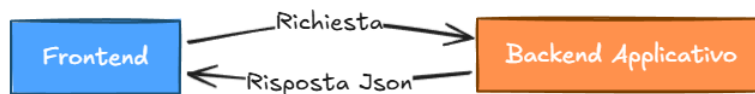


Figura 2.2: Schema generale di un'architettura client-server con frontend e backend applicativo.

La comunicazione tra frontend e backend applicativo avviene tipicamente tramite API REST, un insieme di interfacce che seguono i principi dell'architettura Representational State Transfer. Le API REST definiscono un modello di interazione basato su risorse identificate tramite URI e manipolate attraverso metodi standard del protocollo HTTP, quali GET, POST, PUT e DELETE. L'adozione di REST consente di ottenere un'interfaccia

uniforme, stateless e facilmente scalabile, caratteristiche che ne hanno favorito la diffusione nelle architetture web contemporanee. In questo contesto, il frontend agisce come client primario e invia richieste al backend applicativo incapsulando dati e parametri necessari all'elaborazione, mentre il backend applicativo risponde con rappresentazioni strutturate, spesso in formato JSON, mantenendo una chiara separazione tra logica di presentazione e logica di elaborazione.

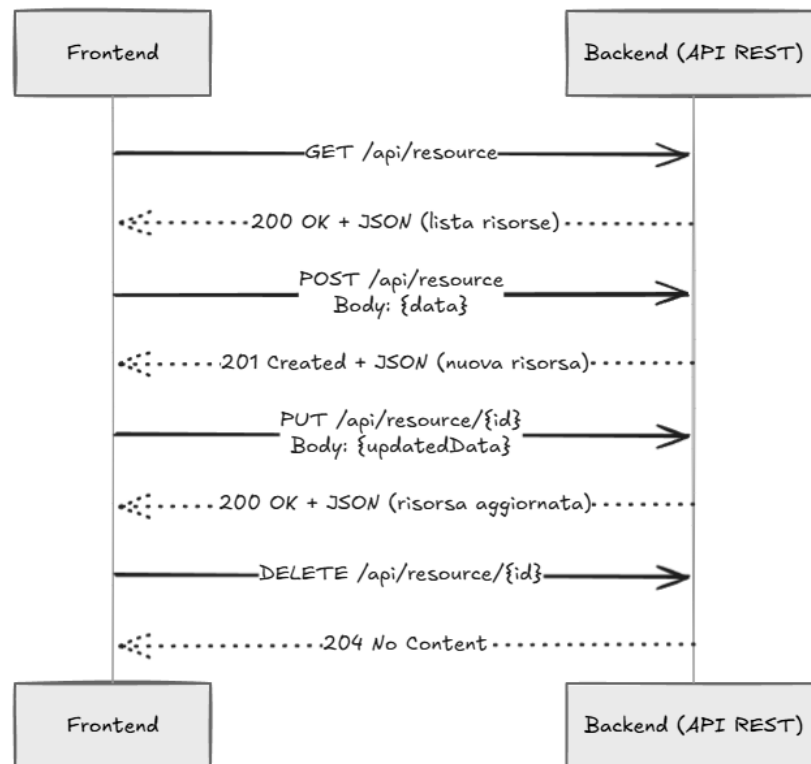


Figura 2.3: Esempio di comunicazione tramite API RESTful.

Il backend applicativo svolge una duplice funzione: da un lato opera come server rispetto al frontend, gestendo la logica applicativa, l'autenticazione, la validazione dei dati e la composizione delle risposte; dall'altro lato si comporta come client nei confronti del backend LLM. Tale backend LLM è generalmente implementato come servizio remoto ad alta intensità computazionale, ottimizzato per operazioni di inferenza basate su modelli linguistici di grandi dimensioni. La comunicazione tra backend applicativo e backend LLM avviene anch'essa tramite API RESTful, che permettono di inviare prompt, parametri di generazione e contesto semantico, ricevendo in risposta output testuali generati dal modello.



Figura 2.4: Interazione tra backend applicativo e backend LLM tramite API RESTful.

Il backend LLM elabora le richieste ricevute attraverso modelli addestrati su larga scala e restituisce risposte che vengono integrate dal backend applicativo nel flusso informativo complessivo. Studi recenti mostrano come l'integrazione di LLM all'interno di architetture client-server richieda una progettazione attenta dei flussi di comunicazione, al fine di garantire scalabilità, affidabilità e coerenza semantica.

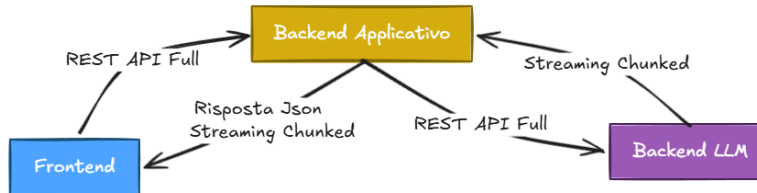


Figura 2.5: Architettura multilivello con frontend, backend applicativo e backend LLM.

Nel complesso, la comunicazione tra frontend, backend applicativo e backend LLM rappresenta un'estensione moderna del paradigma client-server, in cui ogni livello contribuisce in modo complementare alla funzionalità globale del sistema. Il frontend gestisce l'interazione con l'utente, il backend applicativo coordina la logica e funge da intermediario intelligente, mentre il backend LLM fornisce capacità avanzate di elaborazione. L'uso di API RESTful garantisce un'interfaccia uniforme e scalabile, permettendo di integrare efficacemente componenti eterogenei e di sfruttare appieno le potenzialità dei modelli di grandi dimensioni.

2.2.2 Comunicazione Chunked

La comunicazione chunked rappresenta una delle innovazioni più significative introdotte nell'evoluzione del protocollo HTTP, poiché consente la trasmissione di dati in streaming senza la necessità di conoscere anticipatamente la dimensione complessiva del contenuto. Prima dell'introduzione di HTTP/1.1, il protocollo richiedeva che il server specificasse la lunghezza totale del corpo della risposta tramite l'header Content-Length, limitando la

possibilità di inviare contenuti generati dinamicamente o prodotti progressivamente. Tale vincolo risultava particolarmente restrittivo per applicazioni che necessitavano di bassa latenza o che producevano dati in tempo reale [Fielding et al., 1997, Fielding, 2000].

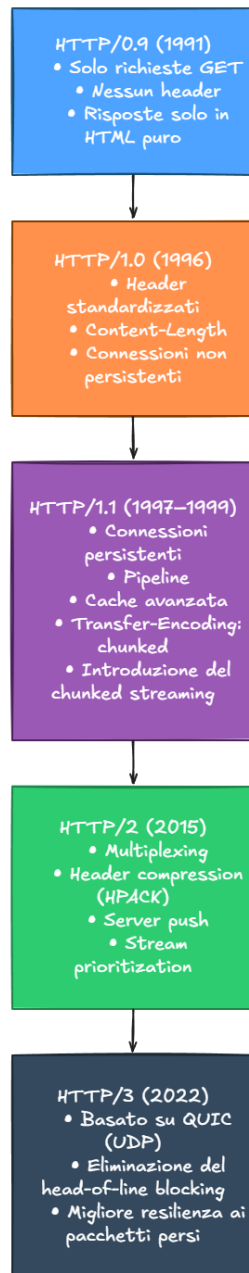


Figura 2.6: Evoluzione del supporto alla comunicazione chunked nelle versioni di HTTP.

Il supporto ufficiale al chunked transfer encoding è stato introdotto con HTTP/1.1, inizialmente nella RFC 2068 del 1997 e successivamente perfezionato nella RFC 2616 del 1999 [Fielding et al., 1997, Fielding et al., 1999]. Con questa estensione, il protocollo ha reso possibile inviare il corpo della

risposta suddiviso in blocchi, detti chunk, ciascuno preceduto dalla propria dimensione espressa in formato esadecimale. Il server può così iniziare la trasmissione immediatamente, senza attendere la generazione completa del contenuto. La comunicazione termina con un chunk di dimensione zero, che segnala la conclusione dello stream e permette al client di interpretare correttamente la fine della risposta.

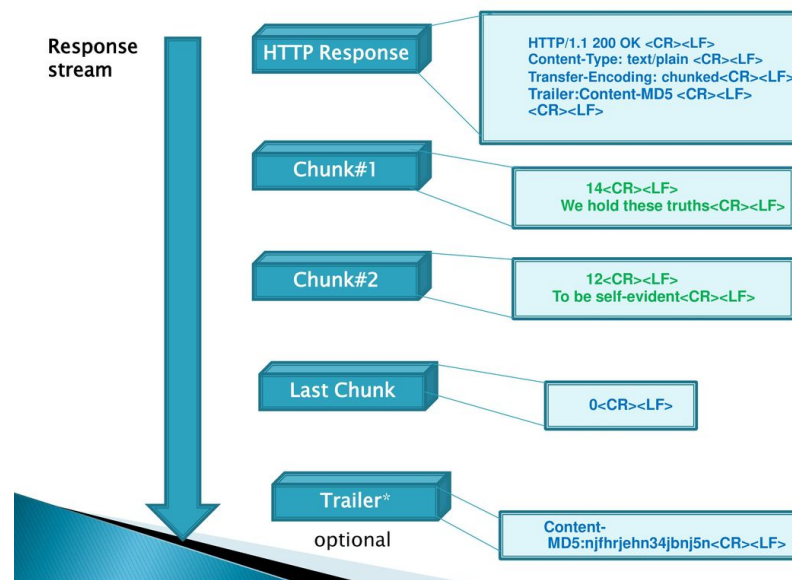


Figura 2.7: Schema del funzionamento del chunked transfer encoding.

Il meccanismo si basa sull'header Transfer-Encoding: chunked, che istruisce il client a interpretare la risposta come una sequenza di blocchi. A differenza dell'uso del Content-Length, tale approccio non richiede la conoscenza preventiva della dimensione totale del payload e consente una gestione più efficiente delle risorse. Il client può iniziare l'elaborazione dei dati non appena riceve il primo chunk, migliorando la reattività percepita dall'utente e riducendo la latenza complessiva del sistema.

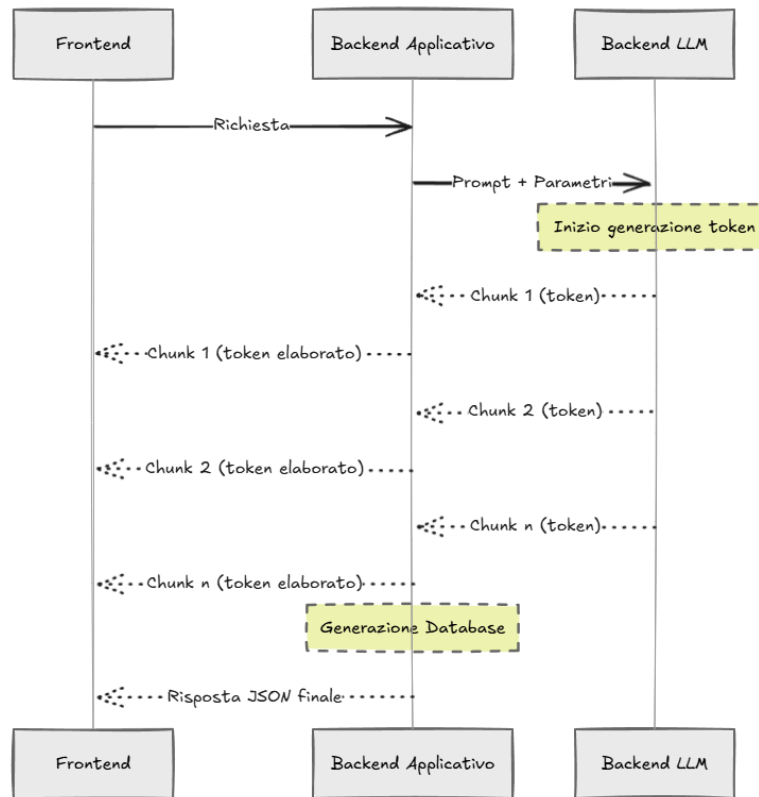


Figura 2.8: Flusso di comunicazione chunked tra server e client.

Nelle architetture moderne, la comunicazione chunked è ampiamente utilizzata nei sistemi che integrano backend applicativi e backend basati su modelli linguistici di grandi dimensioni (LLM). In tali scenari, il backend applicativo invia una richiesta al backend LLM tramite REST API e riceve in risposta una sequenza di chunk contenenti token generati progressivamente dal modello. Questi chunk possono essere elaborati, filtrati o arricchiti dal backend applicativo prima di essere inoltrati al frontend, che a sua volta li riceve in streaming. Tale modalità consente di mantenere un flusso continuo di dati verso l'utente, riducendo la latenza percepita e migliorando l'esperienza complessiva.

2.2.3 Design Pattern Gateway

Il design pattern Gateway–Controller rappresenta una soluzione architettuale orientata alla separazione delle responsabilità nei sistemi software distribuiti. Esso distingue chiaramente tra un Gateway, responsabile della gestione delle richieste in ingresso, e un Controller, incaricato dell'orchestrazione della

logica applicativa. Tale approccio è coerente con i principi di modularità e incapsulamento discussi da Ferrer Garcia nell'ambito delle architetture MVP [Garcia, 2023b, Garcia, 2023a].

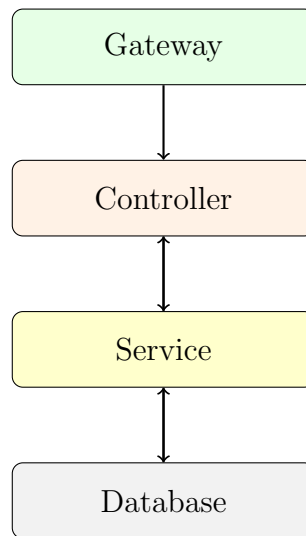


Figura 2.9: Flusso interno del Controller tra logica applicativa e servizi.

Il Gateway funge da punto di accesso centralizzato, occupandosi di validazione preliminare, normalizzazione dei protocolli, autenticazione e instradamento verso i componenti interni. Questa struttura richiama i moderni API Gateway utilizzati nelle architetture a microservizi.

Il Controller, ricevendo richieste già filtrate, coordina i servizi applicativi, applica le regole di business e produce risposte coerenti. Tale distinzione richiama i modelli architetturali presentati da Zhang e Luo, che evidenziano come la strutturazione delle responsabilità riduca la complessità nei sistemi interattivi [Zhang and Luo, 2010].

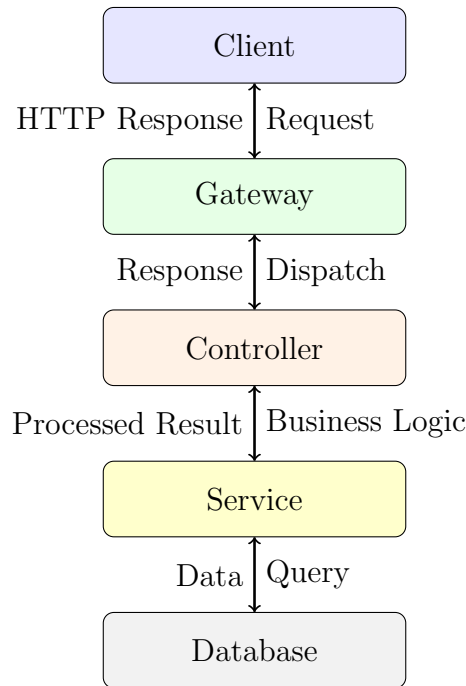


Figura 2.10: Diagramma del pattern Gateway-Controller.

L'evoluzione del pattern è strettamente legata allo sviluppo dei protocolli HTTP: dalle prime specifiche HTTP/1.1 [Fielding et al., 1997, Fielding et al., 1999], alla formalizzazione REST di Fielding [Fielding, 2000], fino alle ottimizzazioni introdotte da HTTP/2 [Bishop, 2015] e HTTP/3 [Bishop, 2022]. Questi progressi hanno reso il Gateway un componente sempre più sofisticato, capace di gestire compressione, multiplexing, caching e trasformazioni dei payload, consolidando il ruolo del Controller come nucleo della logica applicativa.

2.2.4 MongoDB

MongoDB è un sistema di gestione di basi di dati NoSQL orientato ai documenti, progettato per fornire elevata scalabilità orizzontale, flessibilità nella modellazione dei dati e prestazioni ottimizzate in contesti distribuiti. La sua architettura si discosta dai modelli relazionali tradizionali, adottando un paradigma basato su documenti BSON (Binary JSON), che consente una rappresentazione più naturale e dinamica delle entità applicative. La documentazione ufficiale di MongoDB evidenzia come tale modello favorisca l'evoluzione incrementale dello schema e l'ottimizzazione delle query basate su aggregazioni complesse [MongoDB Inc., 2024].

Il modello dei dati di MongoDB si fonda su tre concetti fondamentali: documenti, collezioni e database. I documenti, rappresentati in formato BSON, costituiscono l'unità atomica di memorizzazione e possono contenere strutture annidate e array, rendendo possibile una modellazione dei dati più vicina agli oggetti applicativi rispetto alle tabelle rigidamente schematizzate dei sistemi relazionali. Le collezioni rappresentano insiemi di documenti logicamente correlati. A differenza delle tabelle dei database relazionali, esse non impongono uno schema rigido: documenti appartenenti alla stessa collezione possono presentare campi differenti, tipologie di dati eterogenee e strutture annidate variabili. Questa caratteristica, nota come *schema flexibility*, consente un'evoluzione incrementale del modello dei dati senza necessità di migrazioni strutturali invasive [MongoDB Inc., 2024]. Le collezioni possono inoltre essere indicizzate mediante indici tradizionali, geospaziali, testuali o composti, ottimizzando così le prestazioni delle query anche in presenza di dataset di grandi dimensioni. Il livello superiore dell'organizzazione dei dati è costituito dal database, che funge da contenitore logico per più collezioni. Ogni database mantiene il proprio spazio dei nomi, le proprie configurazioni e i propri metadati, garantendo isolamento e indipendenza tra applicazioni o moduli funzionali diversi. In un contesto distribuito, i database possono essere replicati o suddivisi tramite sharding, mantenendo coerenza e disponibilità elevate grazie ai meccanismi di replica set e ai config server. Tale struttura gerarchica — database, collezioni, documenti — riflette un modello concettuale progettato per massimizzare la scalabilità orizzontale e la resilienza, in linea con le architetture distribuite moderne [Fielding, 2000].

Documento BSON (esempio reale)

```
{
  "_id": { "$oid": "69f08c69e46de31988019252" },
  "attachment_path": "3839f28c764d2247",
  "attachments": [
    {
      "attachment_type": "csv",
      "id": "69f08c6be61ec8ce290fe8d2",
      "is_user_upload": true,
      "original_name": "customers-1000.csv",
      "relative_path": "./uploads/customers-1000.csv"
    }
  ],
  "closed": true,
  "creation_timestamp": "2026-04-28 12:31:05",
  "db_name": "ma3PYefhIm3PgmKC",
  "is_base_transaction": true,
  "messages": [ ... ],
  "status": "Data Structure Generated",
  "transaction_name": "Load Customers"
}
```

Figura 2.11: Esempio di documento BSON basato sulla struttura della collection bioCMS.

MongoDB implementa nativamente un'architettura distribuita basata sui replica set, gruppi di nodi che mantengono copie sincronizzate dei dati. Tale meccanismo garantisce tolleranza ai guasti e disponibilità elevata attraverso un protocollo di elezione automatica del nodo primario. Questi principi sono coerenti con le architetture network-based descritte da Fielding nella sua analisi dei sistemi distribuiti [Fielding, 2000].

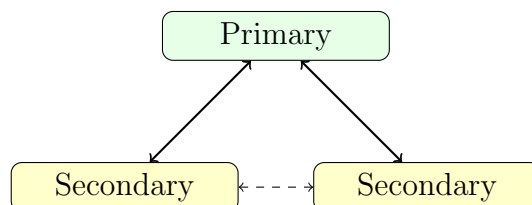


Figura 2.12: Replica set in MongoDB.

Per gestire dataset di grandi dimensioni, MongoDB utilizza lo sharding, una tecnica che suddivide i dati su più nodi secondo una chiave di partizionamento. Lo sharding consente di distribuire il carico sia in termini di memorizzazione sia in termini di elaborazione delle query, permettendo al sistema di scalare orizzontalmente in modo lineare. A differenza della scalabilità verticale, che richiede l'impiego di hardware più potente, la scalabilità orizzontale ottenuta tramite sharding permette di aggiungere nuovi nodi al cluster in maniera incrementale, mantenendo costi e complessità sotto controllo.

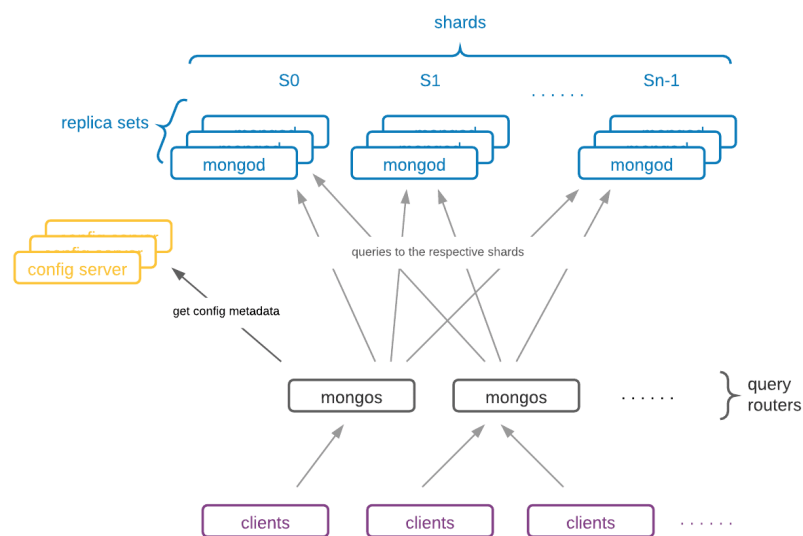


Figura 2.13: Esempio di funzionamento dello Sharding.

Il funzionamento dello sharding si basa su tre componenti fondamentali:

- **Shard:** nodi che contengono fisicamente i dati. Ogni shard è tipicamente implementato come un replica set, garantendo ridondanza e tolleranza ai guasti.
- **Config Server:** memorizza la mappa della distribuzione dei dati, ovvero l'associazione tra intervalli di chiavi e shard [MongoDB Inc., 2024].
- **Router mongos:** riceve le query dai client, consulta i config server e instrada le richieste verso gli shard appropriati.

La distribuzione dei dati avviene tramite una shard key, un campo o insieme di campi che determinano come i documenti vengono partizionati. MongoDB suddivide lo spazio delle chiavi in intervalli chiamati chunk. Ogni

chunk viene assegnato a uno shard, e quando uno shard diventa sovraccarico, il sistema esegue automaticamente operazioni di splitting e balancing [MongoDB Inc., 2024]. Questo approccio è pienamente coerente con i principi delle architetture distribuite moderne, che privilegiano parallelismo, riduzione della latenza e resilienza. In particolare, le evoluzioni dei protocolli HTTP, come HTTP/2 [Bishop, 2015] e HTTP/3 [Bishop, 2022], introducono meccanismi di multiplexing, riduzione dell'overhead e miglioramento della gestione delle connessioni, favorendo modelli di comunicazione ad alta efficienza. In un cluster MongoDB, tali caratteristiche si traducono in una maggiore rapidità nell'elaborazione delle query, una minore congestione del traffico interno e una migliore distribuzione del carico tra i nodi. Lo sharding rappresenta quindi un elemento essenziale per garantire la scalabilità e l'affidabilità dei sistemi basati su MongoDB, soprattutto in contesti caratterizzati da elevati volumi di dati, accessi concorrenti e distribuzione geografica.

2.2.5 SimpleXLSX

La libreria SimpleXLSX rappresenta uno dei più diffusi strumenti open source per la lettura di file in formato XLSX all'interno dell'ecosistema PHP.

Il progetto, sviluppato e mantenuto da Sergey Shuchkin, si configura come un parser conforme alle specifiche dei workbook Microsoft Excel 2007 e successivi, basato su un'architettura che utilizza esclusivamente estensioni interne del linguaggio, quali SimpleXML e i meccanismi nativi di decompressione ZIP. Tale caratteristica consente l'impiego della libreria senza dipendenze esterne, rendendola particolarmente adatta a contesti di produzione in cui la portabilità del codice costituisce un requisito fondamentale [Shuchkin, 2024b]. La libreria implementa un insieme coerente di metodi per la gestione del parsing, tra cui `parse`, `parseFile` e `parseData`, i quali permettono rispettivamente la lettura da percorso locale, da file e da dati binari in memoria. Il processo di parsing è progettato per restituire un oggetto SimpleXLSX oppure un messaggio di errore dettagliato tramite il metodo `parseError`, che consente di identificare condizioni quali file non trovati, archivi non validi o componenti XML mancanti. La presenza di codici di errore specifici, documentati nella reference tecnica, contribuisce a rendere la libreria uno strumento robusto e trasparente dal punto di vista diagnostico [Shuchkin, 2024a]. Una volta completata la fase di parsing, la libreria mette a disposizione un insieme articolato di metodi per l'accesso ai dati. Tra questi, `rows` consente di ottenere l'intero contenuto di un foglio di lavoro come matrice bidimensionale, mentre `readRows` fornisce un iteratore orientato alla gestione di file di grandi dimensioni, riducendo l'impatto sulla memoria. Ulteriori funzionalità includono la possibilità di estrarre informazioni strut-

turali, come il numero di fogli, i nomi dei fogli, le dimensioni delle tabelle e i metadati associati. La libreria offre inoltre metodi per la conversione diretta in HTML, sia in forma semplice sia con preservazione degli stili, ampliando le possibilità di integrazione con applicazioni web [Shuchkin, 2024b]. Dal punto di vista dell’ecosistema PHP, SimpleXLSX si distingue per la sua diffusione e per la disponibilità tramite Composer, che ne facilita l’integrazione nei progetti moderni [Packagist Contributors, 2024].

In conclusione, SimpleXLSX si configura come una soluzione matura, stabile e tecnicamente solida per la lettura dei file XLSX in PHP. La combinazione di un’API chiara, un’implementazione priva di dipendenze esterne e una documentazione completa ne fanno uno strumento di riferimento per sviluppatori e ricercatori che necessitano di un parser affidabile, a basso impatto sulla memoria e conforme agli standard dei formati Excel moderni.

2.2.6 Bearer Token

Il protocollo HTTP fornisce un supporto nativo all’utilizzo dei Bearer Token attraverso l’header `Authorization`, che costituisce il meccanismo standardizzato per la trasmissione di credenziali nel framework OAuth 2.0. La RFC 6750 [Jones and Hardt, 2012] stabilisce che il client debba includere il token secondo la sintassi:

```
Authorization: Bearer <token>
```

```
GET /transaction HTTP/1.1
Host: 127.0.0.1:8082
User-Agent: Mozilla/5.0
Accept: application/json
Connection: keep-alive
Authorization: Bearer \
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.\
eyJzdWIiOiJldGVudGUlLCJzY29wZSI6WyJ0cmFuc2FjdGlvbi5yZWZkIiwiaWF0Ij0i
dHJhbnNhY3Rpb24ud3JpdGUlXSwiaWF0IjoxNjk5OTk5OTk5fQ.\
dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

Figura 2.14: Richiesta HTTP con autenticazione tramite Bearer Token.

L’utilizzo del Bearer Token risponde a due esigenze centrali nel modello di autorizzazione, come illustrato nel flusso riportato in Figura 2.15. Da un lato, esso consente di identificare l’utente che effettua la richiesta, fungendo

da rappresentazione dell'identità autenticata presso il server d'autenticazione. Dall'altro, permette di verificare (autorizzazione) che l'utente disponga dei permessi necessari per eseguire specifiche operazioni su determinati insiemi di dati, poiché il token incorpora o fa riferimento a un insieme di privilegi associati al soggetto autenticato. Una volta ricevuto il token, il server di risorse procede alla sua validazione seguendo le fasi descritte in Figura 2.15, determinando se l'operazione richiesta rientri nelle autorizzazioni concesse. Tale processo può avvenire mediante introspezione del token, mediante verifica crittografica nel caso di token auto-contenuti oppure tramite consultazione del server di autorizzazione. In caso di token mancante, scaduto o invalido, il server può richiedere nuove credenziali tramite l'header WWW-Authenticate, fornendo indicazioni utili alla risoluzione dell'errore. Sebbene HTTP consenta anche la trasmissione del token tramite parametri nel corpo della richiesta o nella query string, tali modalità sono considerate meno sicure e non raccomandate, poiché aumentano la probabilità di esposizione accidentale del token nei log applicativi o nei sistemi intermedi. L'uso dell'header Authorization, congiunto a un canale cifrato come TLS, costituisce pertanto la soluzione più conforme alle linee guida di sicurezza definite dallo standard, come rappresentato nel flusso di Figura 2.15.

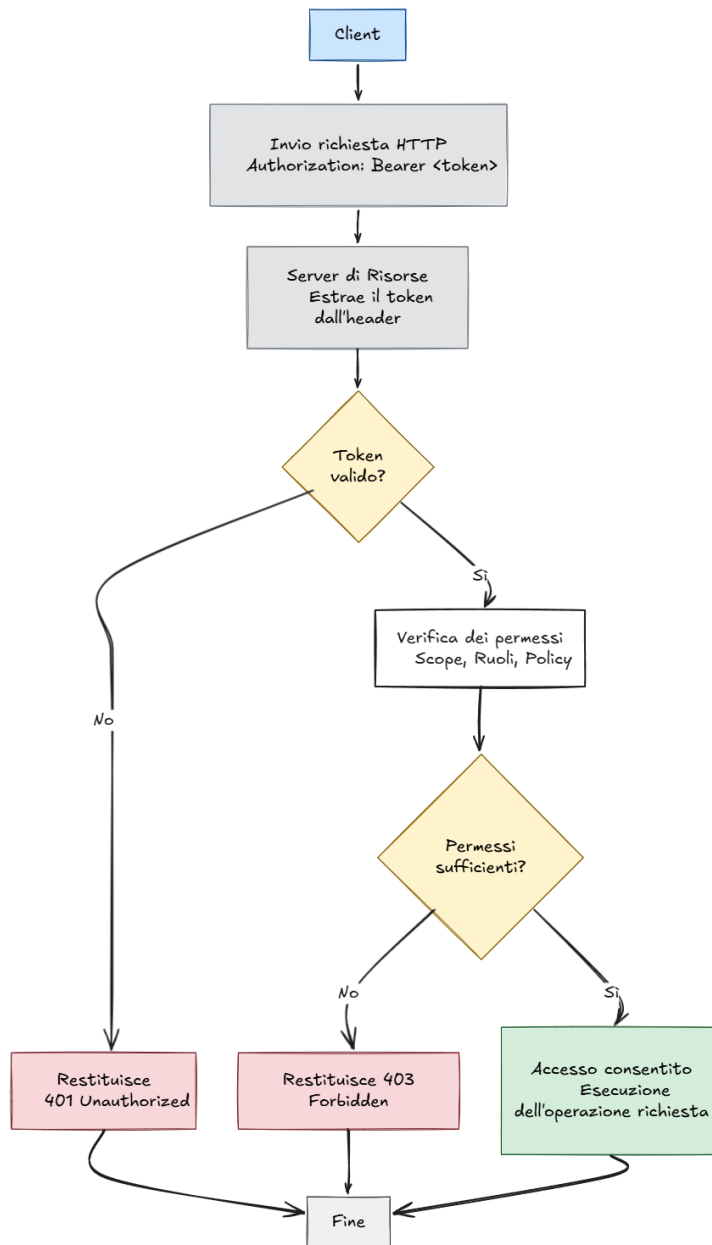


Figura 2.15: Schema del processo di validazione del Bearer Token da parte del server di risorse.

2.2.7 Query Safety

La protezione delle interrogazioni SQL rappresenta un elemento essenziale nella progettazione di sistemi applicativi sicuri. L'esecuzione diretta e non


```
33     }
34 }
35
36 // Boolean operators only inside WHERE
37 if (preg_match("/\b(and|or|xor)\b/", $q)) {
38     if (strpos($q, " where ") === false) {
39         return false;
40     }
41     $where = extractWhereClause($q);
42     if (!preg_match("/\b(and|or|xor)\b/", $where)) {
43         return false;
44     }
45 }
46
47 // Forbidden keywords
48 $forbidden = [
49     "insert", "update", "delete", "drop", "alter", "truncate",
50     "replace", "create", "outfile", "dumpfile", "union",
51     "information_schema"
52 ];
53 foreach ($forbidden as $kw) {
54     if (preg_match("/\b$kw\b/", $q)) {
55         return false;
56     }
57 }
58
59 // Enforce simple SELECT structure
60 if (!preg_match(
61     "/^select [a-z0-9_.*, ]+ from [a-z0-9_]+( where .+)?( limit \d+)?$/",
62     $q
63 )) {
64     return false;
65 }
66
67 return true;
68 }
```

La funzione impone che la query inizi con l'istruzione SELECT, limitando così l'interazione con il database a operazioni di sola lettura. Tale vincolo elimina intere classi di attacchi basati su comandi distruttivi o modificativi, come DROP, DELETE o ALTER. La validazione prosegue impedendo l'uso di commenti SQL, spesso utilizzati per troncare parti della query originale, e bloccando la presenza di sottoquery, che possono essere sfruttate per aggirare filtri superficiali o per estrarre informazioni sensibili [Halfond et al., 2006].

Particolare attenzione è dedicata alle funzioni considerate pericolose, come `sleep()`, `benchmark()` o `load_file()`, frequentemente impiegate in attacchi basati sul tempo o sull'accesso al file system. La funzione verifica inoltre che gli operatori booleani AND, OR e XOR compaiano esclusivamente all'interno della clausola WHERE, prevenendo manipolazioni logiche che potrebbero alterare l'esito della query in modo malevolo.

L'ultimo controllo impone una struttura sintattica semplice e prevedibi-

le, composta da un'istruzione `SELECT`, un elenco di colonne, una clausola `FROM` e, opzionalmente, una clausola `WHERE` o un limite numerico tramite `LIMIT`. Tale modello ridotto consente di minimizzare la complessità e di prevenire l'inserimento di costrutti avanzati che potrebbero essere sfruttati per scopi malevoli.

L'esecuzione libera e non controllata di query SQL lato backend, in assenza di un sistema di validazione strutturale come quello descritto, espone l'applicazione a un insieme eterogeneo di vulnerabilità critiche. La superficie d'attacco si amplia infatti in modo significativo quando l'input fornito dall'utente viene incorporato direttamente nella query, consentendo a un attaccante di manipolare la logica dell'interrogazione o di introdurre costrutti malevoli. Le principali tipologie di attacchi riconducibili alla SQL Injection includono le forme in-band, inferential e out-of-band, ciascuna caratterizzata da modalità operative e obiettivi differenti.

Gli attacchi in-band rappresentano la categoria più immediata e comprendono tecniche come la Union-based Injection, nella quale l'attaccante sfrutta l'operatore `UNION` per combinare la query legittima con una seconda interrogazione arbitraria, e la Error-based Injection, che induce il database a restituire messaggi di errore contenenti informazioni sensibili sulla struttura interna del sistema. Gli attacchi inferential, o blind, non producono risultati diretti ma si basano sull'osservazione del comportamento del server: tra questi rientrano le tecniche Boolean-based, che alterano la veridicità delle condizioni logiche, e le Time-based, che sfruttano funzioni ritardanti come `SLEEP()` o `BENCHMARK()` per inferire informazioni dal tempo di risposta. Infine, gli attacchi out-of-band utilizzano canali secondari, come richieste DNS o HTTP generate dal database, per esfiltrare dati senza che essi compaiano nella risposta principale.

In questo contesto, l'assenza di un sistema di controllo come `isSafeQuery()` consentirebbe a un attaccante di ottenere accesso non autorizzato a dati riservati, modificare informazioni sensibili, compromettere la disponibilità del servizio o addirittura eseguire comandi arbitrari sul database. L'adozione di un modello di Query Safety costituisce pertanto un livello di difesa essenziale, poiché limita drasticamente la complessità sintattica ammessa e impedisce l'inserimento di costrutti tipicamente associati a comportamenti malevoli. Tale approccio deve essere considerato complementare all'uso di prepared statements, alla sanitizzazione degli input e alle politiche di least privilege, ovvero garantire all'utente solamente i privilegi strettamente necessari, contribuendo in modo determinante alla preservazione dell'integrità, della riservatezza e della sicurezza complessiva del sistema.

2.2.8 Ordinamento Topologico

L'ordinamento topologico rappresenta uno strumento fondamentale nell'ambito dei grafi diretti aciclici, poiché consente di stabilire una sequenza lineare dei nodi tale che ogni arco sia orientato da un nodo precedente verso uno successivo [Cormen et al., 2022]. Questa proprietà risulta particolarmente rilevante quando si affronta il problema della creazione ordinata delle tabelle SQL in presenza di vincoli di chiave esterna. Ogni tabella può essere interpretata come un nodo di un grafo diretto, mentre ogni vincolo FOREIGN KEY costituisce un arco che collega la tabella dipendente alla tabella referenziata. L'obiettivo è determinare un ordine di creazione che garantisca che nessuna tabella venga generata prima delle proprie dipendenze, evitando così errori di integrità referenziale [Sedgewick and Wayne, 2011].

La modellazione del problema come grafo consente di applicare un algoritmo di visita ricorsiva che, attraverso una marcatura dei nodi già esplorati, produce una sequenza lineare coerente con le dipendenze. Tale approccio si basa sulla profondità di visita e sulla post-elaborazione dei nodi [Kahn, 1962], i quali vengono aggiunti all'ordine finale soltanto dopo aver completato l'esplorazione delle loro dipendenze. L'assenza di cicli è una condizione necessaria e sufficiente per l'esistenza di un ordinamento topologico [Cormen et al., 2022], mentre la presenza di eventuali dipendenze mancanti viene gestita ignorando i riferimenti non presenti nel grafo.

L'algoritmo può essere espresso nel seguente pseudocodice:

Algoritmo 2 Pseudo-Codice Ordinamento Topologico.

```
function TOPOLOGICALSORT(Tables)
  deps ← ExtractDependencies(Tables)
  visited ← empty set
  order ← empty list
  for all table in deps do
    Visit(table)
  return order
function VISIT(table)
  if table ∈ visited then
    return
  Mark table as visited
  for all parent in deps[table] do
    if parent exists in deps then
      Visit(parent)
  Append table to order
```

L'implementazione concreta dell'algoritmo, riportata di seguito, riflette fedelmente la struttura concettuale appena descritta. Essa estrae le dipendenze analizzando i vincoli di chiave esterna tramite espressioni regolari, costruisce dinamicamente il grafo delle relazioni e applica una procedura ricorsiva per determinare l'ordine corretto di creazione delle tabelle. Il risultato finale è una sequenza priva di duplicati, ottenuta mantenendo l'ordine di visita e garantendo che ogni tabella compaia soltanto dopo le proprie dipendenze.

Algoritmo 3 Implementazione Ordinamento Topologico.

```

1 private function resolveTableOrder(): array
2 {
3     $deps = [];
4
5     foreach ($this->tables as $tableName => $info) {
6         $deps[$tableName] = [];
7
8         foreach ($info['constraints'] as $constraint)
9             if (preg_match('/FOREIGN KEY\s*(.*?)\s*REFERENCES\s+([A-Za-z0
-9_]+>)/i', $constraint, $m))
10                 $deps[$tableName][] = $m[1];
11     }
12
13     $visited = [];
14     $order = [];
15
16     $visit = function ($table) use (&$visit, &$visited, &$order, $deps) {
17         if (isset($visited[$table]))
18             return;
19         $visited[$table] = true;
20
21         foreach ($deps[$table] as $parent) {
22             if (!isset($deps[$parent]))
23                 continue;
24             $visit($parent);
25         }
26
27         $order[] = $table;
28     };
29
30     foreach (array_keys($deps) as $table)
31         $visit($table);
32
33     return array_values(array_unique($order));
34 }

```

L'adozione dell'ordinamento topologico in questo contesto consente di automatizzare un processo che, se svolto manualmente, risulterebbe complesso e soggetto a errori, soprattutto in presenza di schemi articolati [Sedgewick and Wayne, 2011] e permette di ridurre il carico computazionale delegato al modello LLM, il quale non deve verificare che la

sequenza di CREATE TABLE generata non presenti problemi di integrità referenziale. La rappresentazione grafica delle dipendenze, come mostrato nella figura seguente, evidenzia la natura gerarchica delle relazioni e la necessità di un approccio sistematico per la determinazione dell'ordine di creazione. Per comprendere appieno l'importanza di tale approccio, è utile analizzare un caso concreto in cui le tabelle vengano fornite in un ordine arbitrario e non sempre coerente con le loro dipendenze.

Si considerino tre tabelle: `OrderItems`, `Orders` e `Users`. Supponendo di riceverle nel seguente ordine:

`[OrderItems, Orders, Users]`

la loro creazione sequenziale produrrebbe inevitabilmente errori di integrità referenziale. La tabella `OrderItems` contiene infatti un vincolo di chiave esterna verso `Orders`, mentre `Orders` dipende a sua volta da `Users`. Tentare di creare `OrderItems` come prima tabella comporterebbe un errore, poiché la tabella `Orders` non esiste ancora al momento della definizione del vincolo.

Le definizioni SQL semplificate, riportate di seguito nell'ordine originale, rendono evidente la natura del problema:

```
CREATE TABLE OrderItems (  
    id INT PRIMARY KEY,  
    order_id INT,  
    FOREIGN KEY (order_id) REFERENCES Orders(id)  
);
```

```
CREATE TABLE Orders (  
    id INT PRIMARY KEY,  
    user_id INT,  
    FOREIGN KEY (user_id) REFERENCES Users(id)  
);
```

```
CREATE TABLE Users (  
    id INT PRIMARY KEY,  
    name VARCHAR(255)  
);
```

L'analisi delle dipendenze mostra che la sequenza iniziale non può essere utilizzata. Applicando l'algoritmo di ordinamento topologico, il grafo diretto delle relazioni viene esplorato ricorsivamente, garantendo che ogni tabella venga inserita nell'ordine finale soltanto dopo aver visitato tutte le entità da

cui essa dipende. La procedura ricostruisce così una sequenza coerente con i vincoli di chiave esterna.

Il risultato dell'applicazione dell'algoritmo è la sequenza corretta:

`[Users, Orders, OrderItems]`

che rispetta pienamente la gerarchia delle dipendenze e consente la creazione delle tabelle senza alcuna violazione di integrità referenziale. L'esempio dimostra come l'ordinamento topologico costituisca uno strumento indispensabile per la generazione automatica di schemi SQL complessi, soprattutto quando l'ordine di input non è controllato, come nel caso in esame in cui lo schema viene generato da un modello LLM che può commettere errori di ordinamento nella fase di generazione dello schema o non riflette la struttura logica del database.

2.3 Backend LLM

In questa sezione vengono trattate tutte le nozioni di base riferite al backend del servizio LLM.

2.3.1 Ollama

La libreria `ollama` per Python costituisce un'interfaccia ad alto livello per l'esecuzione locale di modelli linguistici di grandi dimensioni, consentendo l'invocazione di modelli generativi direttamente dal sistema dell'utente senza la necessità di servizi esterni. Tale approccio si inserisce nel più ampio paradigma dell'esecuzione on-device dei modelli di intelligenza artificiale, che negli ultimi anni ha acquisito crescente rilevanza grazie alla disponibilità di architetture ottimizzate e di tecniche di quantizzazione avanzate [Ollama Project, 2024a]. L'obiettivo principale della libreria è fornire un meccanismo semplice e coerente per inviare prompt ai modelli, ricevere risposte in streaming o in forma completa e gestire configurazioni quali parametri di temperatura, token massimi e selezione del modello.

L'utilizzo della libreria si fonda su un'astrazione che permette di trattare i modelli come risorse locali, riducendo la latenza e garantendo un maggiore controllo sui dati elaborati. Tale caratteristica risulta particolarmente rilevante in contesti nei quali la riservatezza delle informazioni rappresenta un requisito fondamentale, poiché l'intero processo di generazione avviene all'interno dell'ambiente dell'utente senza trasmissione verso server remoti. La libreria si integra inoltre con l'infrastruttura di Ollama, che gestisce il carica-

mento, la configurazione e l'esecuzione dei modelli, offrendo un'architettura modulare e facilmente estendibile [Ollama Project, 2024b].

Dal punto di vista operativo, l'interazione con la libreria avviene tramite chiamate dirette alle funzioni di generazione, che accettano un prompt testuale e restituiscono la risposta del modello. È possibile adottare sia una modalità sincrona, nella quale l'output viene restituito integralmente al termine dell'elaborazione, sia una modalità in streaming, che consente di ricevere progressivamente i token generati. Questa seconda modalità risulta particolarmente utile per applicazioni interattive o per sistemi che richiedono una risposta immediata, come interfacce conversazionali o strumenti di assistenza alla scrittura.

L'architettura della libreria è progettata per essere compatibile con una vasta gamma di modelli, inclusi quelli basati su architetture moderne come LLaMA, Mistral e Qwen, purché disponibili nel formato supportato da Ollama. Tale flessibilità consente di adottare modelli differenti a seconda delle esigenze applicative, bilanciando qualità della generazione, velocità di esecuzione e requisiti computazionali. La possibilità di eseguire modelli localmente permette inoltre di sperimentare configurazioni personalizzate e di integrare la libreria in pipeline di elaborazione più complesse, come sistemi di analisi testuale, agenti autonomi o strumenti di automazione.

Nel complesso, la libreria ollama rappresenta un elemento significativo nel panorama degli strumenti per l'esecuzione locale di modelli linguistici, offrendo un equilibrio tra semplicità d'uso, controllo operativo e capacità di integrazione. La sua adozione consente di sviluppare applicazioni avanzate basate su modelli generativi mantenendo al contempo un elevato livello di autonomia e sicurezza dei dati.

2.3.2 Design Pattern Agentic

L'adozione del pattern AI Agent rappresenta una delle evoluzioni più recenti nell'ambito dei sistemi basati su modelli linguistici di grandi dimensioni. Tale paradigma si fonda sull'idea di delegare a un agente autonomo la gestione dell'intero ciclo di ragionamento, dell'interazione con strumenti esterni e della validazione dei risultati [Salatti, 2024]. Le sue radici concettuali possono essere ricondotte ai modelli di agenti deliberativi e reattivi. Gli agenti deliberativi sono caratterizzati da una rappresentazione esplicita dello stato del mondo e da un processo decisionale basato su pianificazione: essi costruiscono modelli interni, formulano obiettivi e selezionano azioni attraverso procedure di ragionamento simbolico [Russell and Norvig, 2021]. Gli agenti reattivi, al contrario, non mantengono una rappresentazione interna complessa, ma rispondono direttamente agli stimoli dell'ambiente mediante regole condizio-

nali, privilegiando rapidità ed efficienza rispetto alla profondità del ragionamento. Il pattern AI Agent integra elementi di entrambe le tradizioni, combinando capacità di pianificazione locale con risposte immediate guidate dal modello linguistico. In questo quadro teorico, il pattern AI Agent si integra con le più recenti architetture di orchestrazione per LLM, come i sistemi a loop iterativo ReAct e Toolformer [Yao et al., 2022, Schick et al., 2023]. Il framework ReAct (Reasoning + Acting) introduce un paradigma nel quale il modello alterna esplicitamente fasi di ragionamento e fasi di azione: il modello produce prima una traccia di pensiero strutturata, poi decide quale strumento utilizzare e infine integra il risultato dell'azione nel ciclo successivo. Questo schema consente al modello di costruire un processo decisionale incrementale, simile a una forma di pianificazione locale, migliorando la capacità di risolvere compiti complessi attraverso interazioni iterative con l'ambiente. Toolformer, invece, propone un approccio differente: il modello viene addestrato a generare autonomamente chiamate a strumenti esterni durante la fase di training, imparando quando e come utilizzare funzioni specializzate per migliorare la qualità delle risposte. In questo modo, l'uso degli strumenti non è più un meccanismo imposto dall'esterno, ma diventa parte integrante delle competenze del modello, che sviluppa una forma di meta-ragionamento sull'impiego ottimale delle risorse disponibili. La tabella che segue offre una sintesi strutturata delle principali categorie di agenti e delle loro proprietà distintive.

Tipo di Agente	Caratteristiche Principali	Punti di Forza	Limiti
Agente deliberativo	Basato su modelli interni, pianificazione esplicita e ragionamento simbolico.	Elevata precisione e coerenza logica.	Richiede risorse computazionali significative.
Agente reattivo	Risponde direttamente agli stimoli senza rappresentazione interna complessa.	Massima rapidità ed efficienza.	Limitato nelle capacità di pianificazione profonda.
Agente LLM (AI Agent)	Integra ragionamento generativo, uso di strumenti e cicli iterativi percezione-decisione-azione.	Flessibile, adattivo e capace di ragionamento contestuale.	Qualità dipendente dal modello e dalla gestione degli strumenti.

Tabella 2.1: Confronto tra agenti deliberativi, reattivi e agenti LLM secondo il paradigma AI Agent.

Nel server Python realizzato, l'architettura segue fedelmente i principi del pattern AI Agent. La classe ChatEngine svolge il ruolo di agente centrale, orchestrando l'intero processo decisionale: riceve l'input dell'utente, invia messaggi al modello LLM, interpreta le risposte, estrae eventuali chiamate a strumenti e ne coordina l'esecuzione. Tale struttura ricalca il modello di agenti dotati di ciclo percezione-decisione-azione, come formalizzato nei sistemi multi-agente [Wooldridge, 2021]. L'agente non si limita a generare testo,

ma mantiene uno stato conversazionale, applica vincoli di validazione tramite modelli Pydantic e gestisce iterativamente il flusso fino alla produzione di un output coerente.

Un elemento distintivo dell'implementazione è la gestione dei tool calls, che consente all'agente di interagire con strumenti esterni quali lettura di file, campionamento di CSV e inferenza di schemi. Tale meccanismo riflette il paradigma dell'azione strumentale, secondo cui un agente intelligente deve essere in grado di estendere le proprie capacità attraverso strumenti specializzati [Goyal et al., 2023]. La presenza di un sistema di caching persistente, integrato nel FileManager, riduce il carico computazionale e consente all'agente di evitare operazioni ridondanti, migliorando l'efficienza complessiva del ciclo di ragionamento.

La struttura del server può essere rappresentata come un grafo di componenti agentici, nel quale il ChatEngine funge da nodo centrale che coordina i moduli di parsing, validazione, gestione file e interazione con il modello LLM:

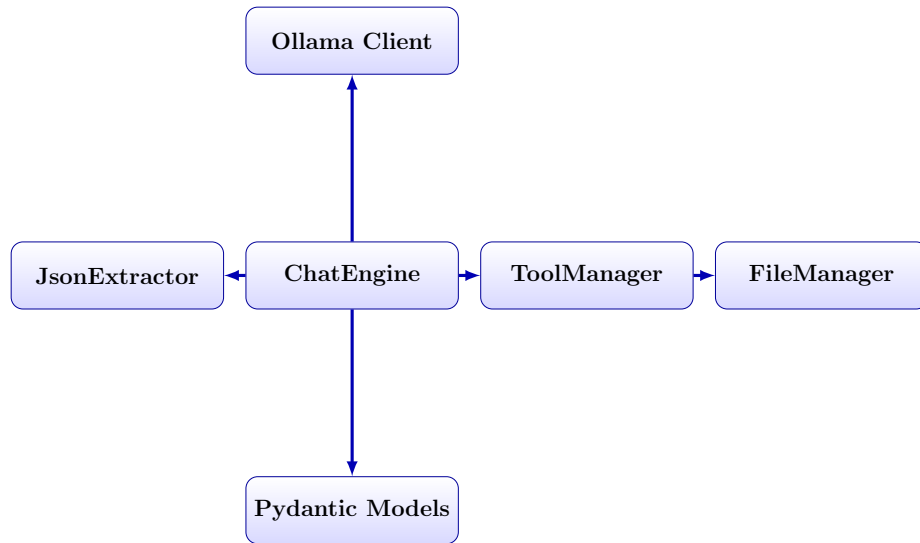


Figura 2.16: Schema architetturale dell'agente LLM e dei sottosistemi di orchestrazione.

Il comportamento iterativo dell'agente, caratterizzato da un massimo di dodici cicli di ragionamento e da una fase di finalizzazione forzata, riflette i principi dei sistemi agentici che prevedono un controllo esplicito della convergenza del processo decisionale. Tale approccio è coerente con i modelli di bounded rationality, secondo cui un agente deve operare entro limiti computazionali definiti [Simon, 1955].

Nel complesso, il server Python implementa un'architettura agentica completa, nella quale il modello LLM non è utilizzato come semplice generatore di testo, ma come componente cognitiva all'interno di un sistema più ampio. L'agente coordina strumenti, gestisce memoria temporanea, valida strutture dati e produce risposte affidabili, incarnando pienamente i principi del pattern AI Agent.

2.3.3 Qwen2.5-Coder-7b-Instruction

Il modello Qwen2.5-Coder-7b-Instruct appartiene alla famiglia Qwen2.5, una serie di modelli linguistici di grandi dimensioni sviluppati con l'obiettivo di fornire capacità avanzate di ragionamento, generazione e comprensione del linguaggio naturale. Qwen2.5 introduce miglioramenti significativi rispetto alle versioni precedenti, grazie a un processo di pretraining su larga scala, a un'architettura Transformer ottimizzata e a un insieme esteso di capacità multimodali e di programmazione [DeepWiki Contributors, 2024]. Il pretraining su larga scala consiste nell'addestramento del modello su un corpus estremamente ampio e diversificato, comprendente testi naturali, documentazione tecnica, codice sorgente e contenuti provenienti da molteplici domini. Tale fase consente al modello di apprendere strutture linguistiche complesse, relazioni semantiche profonde e pattern sintattici ricorrenti, secondo principi coerenti con le leggi di scaling dei modelli neurali [Kaplan et al., 2020]. L'esposizione a dati eterogenei permette inoltre di migliorare la robustezza del modello e la sua capacità di generalizzare a compiti non visti durante l'addestramento. L'architettura Transformer impiegata in Qwen2.5 è stata ottimizzata attraverso tecniche moderne quali l'uso di meccanismi di attenzione più efficienti, strategie di normalizzazione avanzate e miglioramenti nella gestione del contesto esteso. Queste ottimizzazioni consentono al modello di elaborare sequenze più lunghe, ridurre il costo computazionale dell'attenzione e mantenere una maggiore stabilità numerica durante l'inferenza. Tali progressi si collocano nel solco delle evoluzioni architetturali dei Transformer di ultima generazione, che mirano a incrementare la capacità di ragionamento mantenendo l'efficienza operativa [Qwen Team, 2024]. Le capacità multimodali e di programmazione derivano dall'integrazione, nel corpus di addestramento, di dati provenienti da formati differenti e da linguaggi di programmazione eterogenei. La componente multimodale consente al modello di trattare non solo testo naturale, ma anche strutture formali, tabelle, codice e rappresentazioni simboliche. La componente orientata alla programmazione, invece, permette al modello di riconoscere pattern sintattici specifici dei linguaggi di codice, comprendere la semantica delle funzioni e generare soluzioni coerenti con le convenzioni dei principali ecosistemi software. Questa combi-

nazione rende Qwen2.5-Coder-7b-Instruct particolarmente adatto a compiti di sviluppo, debugging e reasoning procedurale [Qwen Team, 2024].

La denominazione 7B indica che il modello contiene circa sette miliardi di parametri, una dimensione che rappresenta un compromesso efficace tra capacità espressiva e costi computazionali. Modelli di questa scala sono in grado di catturare strutture linguistiche complesse, mantenendo al contempo requisiti di memoria e latenza compatibili con sistemi server di fascia media. La legge di scaling mostra come l'aumento del numero di parametri sia correlato a una maggiore capacità di generalizzazione e a prestazioni superiori in compiti di ragionamento astratto [Kaplan et al., 2020].

La variante Coder è specificamente addestrata per attività legate alla programmazione, tra cui generazione di codice, completamento contestuale, refactoring, debugging e spiegazione di frammenti complessi. Tale specializzazione deriva da un pretraining mirato su grandi quantità di codice sorgente e documentazione tecnica, che consente al modello di apprendere pattern sintattici, strutture semantiche e convenzioni idiomatiche dei principali linguaggi di programmazione. La pagina ufficiale del modello conferma che Qwen2.5-Coder-7b-Instruct è ottimizzato per scenari di sviluppo software e reasoning procedurale [Qwen Team, 2024].

L'etichetta Instruct indica che il modello è stato sottoposto a una fase di fine-tuning supervisionato e/o reinforcement learning from human feedback (RLHF), con l'obiettivo di migliorare la capacità di seguire istruzioni complesse, mantenere coerenza contestuale e produrre risposte strutturate. Il fine-tuning supervisionato consiste nell'addestrare il modello su un insieme curato di coppie domanda-risposta, fornite da annotatori umani o da sistemi di generazione controllata, che mostrano esplicitamente come il modello debba comportarsi in scenari specifici. Tale procedura consente di allineare il comportamento del modello agli obiettivi desiderati, riducendo ambiguità, errori di interpretazione e deviazioni stilistiche. Il RLHF rappresenta un ulteriore livello di allineamento, nel quale il modello viene ottimizzato attraverso tecniche di apprendimento per rinforzo basate su valutazioni umane. In questo schema, annotatori esperti confrontano diverse risposte generate dal modello e forniscono preferenze qualitative; tali preferenze vengono poi utilizzate per addestrare un modello di ricompensa, che guida l'ottimizzazione del modello principale tramite algoritmi di reinforcement learning. Questo processo permette di modellare criteri complessi come utilità, sicurezza, coerenza e aderenza alle istruzioni, che risultano difficili da catturare tramite semplice supervisione. L'integrazione di RLHF migliora significativamente l'affidabilità, la controllabilità e la qualità delle risposte dei modelli linguistici allineati [Ouyang et al., 2022]. I modelli Instruct sono quindi progettati per interagire in modo più naturale con l'utente, interpretando correttamente

richieste esplicite, vincoli operativi e formati di output. L'adozione combinata di fine-tuning supervisionato e RLHF consente di ottenere modelli più robusti, coerenti e orientati al compito, rendendoli particolarmente adatti a scenari applicativi che richiedono precisione, affidabilità e capacità di seguire istruzioni complesse.

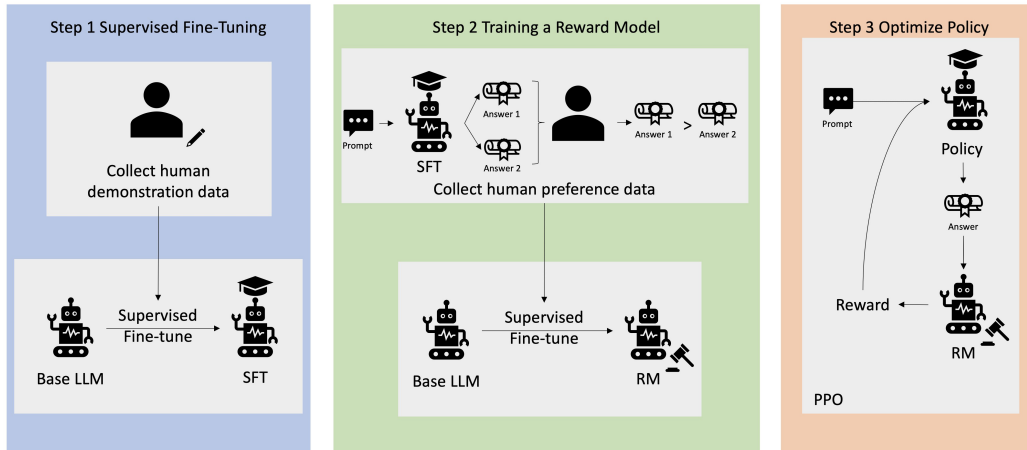


Figura 2.17: Esempio di reinforcement learning from human feedback (RLHF).

Qwen2.5-Coder-7b-Instruct si configura come un modello altamente versatile, capace di integrare competenze linguistiche generali con abilità avanzate di programmazione e con una forte propensione al rispetto delle istruzioni. La combinazione di dimensione intermedia, specializzazione sul codice e allineamento tramite tecniche instruction-following lo rende particolarmente adatto a sistemi agentici, pipeline di automazione e applicazioni che richiedono ragionamento procedurale, generazione affidabile di codice e interazione strutturata con strumenti esterni.

2.3.4 Pydantic Models

La libreria Pydantic rappresenta uno dei principali strumenti per la definizione e la validazione di modelli dati in Python, basandosi su tipizzazione statica e controllo semantico dei campi. Introdotta per semplificare la gestione di strutture complesse, Pydantic consente di definire classi che fungono da schema di validazione, garantendo coerenza tra input e output e riducendo la probabilità di errori di tipo o formato [Colvin and Team, 2025]. La sua architettura si fonda sull'uso delle data classes e dei type hints, integrando meccanismi di parsing automatico e conversione dei tipi.

Un esempio di utilizzo pratico è mostrato nel seguente frammento di codice, tratto dal software in esame che integra un modello Pydantic per la validazione della risposta generata dal modello LLM:

```
from pydantic import BaseModel

class SqlResponse(BaseModel):
    sql_schema: str
    column_mapping: str
```

In questo caso, la classe `SqlResponse` definisce uno schema di validazione per le risposte del modello, assicurando che i campi `sql_schema` e `column_mapping` siano stringhe e rispettino la struttura prevista. Quando l'applicazione riceve una risposta dal modello, essa viene validata tramite il metodo `model_validate_json()`, che garantisce la conformità ai vincoli definiti nel modello.

L'approccio di Pydantic si distingue per la capacità di combinare la semplicità sintattica con la robustezza semantica. Ogni istanza di un modello Pydantic è sottoposta a un processo di parsing e validazione che converte automaticamente i tipi compatibili e solleva eccezioni in caso di incongruenze. Questo paradigma è particolarmente utile in contesti di integrazione tra sistemi eterogenei, come nel caso di API RESTful o interfacce LLM, dove la coerenza dei dati è essenziale per garantire affidabilità e sicurezza [Ramírez, 2025].

Dal punto di vista architetturale, Pydantic si inserisce nel più ampio ecosistema di strumenti per la tipizzazione e la validazione in Python, insieme a librerie come `Marshmallow` e `Attrs`. Tuttavia, la sua integrazione nativa con `FastAPI` e la compatibilità con i modelli asincroni ne fanno una scelta privilegiata per applicazioni moderne basate su microservizi e intelligenza artificiale [Colvin and Team, 2025, Ramírez, 2025]. `FastAPI`, a sua volta, è un framework web moderno e ad alte prestazioni progettato per la costruzione di API in Python, fondato sull'impiego sistematico degli `type hints` e su un'architettura asincrona. Sviluppato da Sebastián Ramírez, esso si basa su `Starlette` per la gestione del routing e delle operazioni asincrone e su `Pydantic` per la validazione dei dati, integrando così un modello di sviluppo che privilegia la sicurezza dei tipi, la leggibilità del codice e l'efficienza computazionale. Le elevate prestazioni, comparabili a quelle di framework realizzati in linguaggi come Go o Node.js, derivano dall'adozione di un paradigma asincrono che consente una gestione scalabile delle richieste concorrenti. Inoltre, `FastAPI` genera automaticamente documentazione interattiva conforme allo standard `OpenAPI`, facilitando sia il processo di sviluppo sia l'integrazione

con servizi esterni [Ramírez, 2025]. In questo contesto, l'interazione tra FastAPI e Pydantic non rappresenta soltanto una scelta implementativa, ma costituisce un modello architetturale coerente con le esigenze delle applicazioni distribuite contemporanee, nelle quali la validazione rigorosa dei dati, la scalabilità e la rapidità di sviluppo assumono un ruolo centrale.

In sintesi, l'uso dei Pydantic Models consente di formalizzare la struttura dei dati, ridurre la complessità del codice e migliorare la manutenibilità dei sistemi software. La loro adozione è oggi considerata una best practice nello sviluppo di applicazioni Python ad alta affidabilità, specialmente in ambiti di validazione automatica e orchestrazione di modelli linguistici.

2.3.5 Pandas

La libreria Pandas costituisce uno degli strumenti fondamentali per la manipolazione di dati in Python, offrendo strutture dati ad alte prestazioni e un insieme esteso di operazioni per la gestione di dataset eterogenei. Introdotta per colmare il divario tra l'analisi dati tradizionale basata su fogli di calcolo e i flussi di lavoro riproducibili tipici della programmazione scientifica, Pandas fornisce un'infrastruttura robusta per il caricamento, la trasformazione e l'esportazione di dati tabulari [Pandas Development Team, 2025]. La sua architettura si fonda su due strutture principali: le Series, vettori monodimensionali indicizzati, e i DataFrame, tabelle bidimensionali analoghe alle strutture presenti nei database relazionali o nei file CSV [Passerini, 2025].

Un esempio tipico di utilizzo di Pandas consiste nella lettura di un file CSV, operazione centrale anche nel codice mostrato nella sezione 2.3.4, dove la libreria viene impiegata per caricare i dati tabellari da file CSV, passarli a un sistema LLM per l'elaborazione e procedere successivamente alla fase di validazione dei risultati generati. Nel seguente frammento, tratto da una pipeline di elaborazione dati, il file viene caricato in memoria e convertito in un DataFrame:

```
import pandas as pd

df = pd.read_csv("file.csv")
print(df.head())
```

Questa operazione consente di trasformare automaticamente il contenuto del file in una struttura tabellare, con inferenza dei tipi e gestione delle intestazioni. Una volta caricato il dataset, Pandas permette di eseguire operazioni di filtraggio, aggregazione e trasformazione con sintassi espressiva e ad alte prestazioni. Ad esempio, per selezionare solo le righe che soddisfano una determinata condizione:

```
filtered = df[df["colonna"] > 10]
```

Pandas rappresenta un ponte efficace tra dati grezzi e flussi di lavoro automatizzati, garantendo riproducibilità, auditabilità e controllo delle trasformazioni [Shapiro et al., 2024].

La flessibilità di Pandas si manifesta anche nella capacità di gestire formati multipli, tra cui CSV, TSV, Excel e database SQL, rendendolo uno strumento adatto a pipeline complesse di data engineering. La possibilità di combinare DataFrame, eseguire join, normalizzare colonne e validare i dati lo rende particolarmente utile in applicazioni che richiedono un pre-processing affidabile prima dell'interazione con modelli di machine learning o sistemi di generazione automatica del codice.

In sintesi, Pandas costituisce un elemento essenziale per la manipolazione dei dati in Python, fornendo un'infrastruttura solida per la preparazione, pulizia e trasformazione dei dataset. La sua integrazione con strumenti moderni di analisi e con modelli linguistici avanzati ne conferma il ruolo centrale nei flussi di lavoro contemporanei orientati alla riproducibilità e alla scalabilità.

2.3.6 Prompt Engineering

Il prompt engineering è una disciplina centrale nell'interazione con modelli linguistici di grandi dimensioni (LLM), poiché consente di controllarne il comportamento, ridurre l'ambiguità e ottenere output strutturati e riproducibili. La formulazione del prompt influenza direttamente accuratezza, coerenza e sicurezza delle risposte [White et al., 2023, Zhou et al., 2023].

Nei modelli Instruct, come Qwen2.5-Coder-7b-Instruct, tale effetto è amplificato dal processo di addestramento supervisionato e RLHF [Ouyang et al., 2022], come discusso nella sezione 2.3.3. Nel sistema in esame sono stati inoltre impiegati token specifici previsti dall'architettura Qwen2.5 [DeepWiki Contributors, 2024], al fine di ottimizzare la gestione delle istruzioni e la coerenza dell'output generato. Tra questi figurano i delimitatori `<|im_start|>` e `<|im_end|>` per separare i turni conversazionali, i token di sistema per definire il contesto operativo e ulteriori marcatori dedicati alla segmentazione delle istruzioni, che contribuiscono a una più precisa interpretazione del prompt da parte del modello.

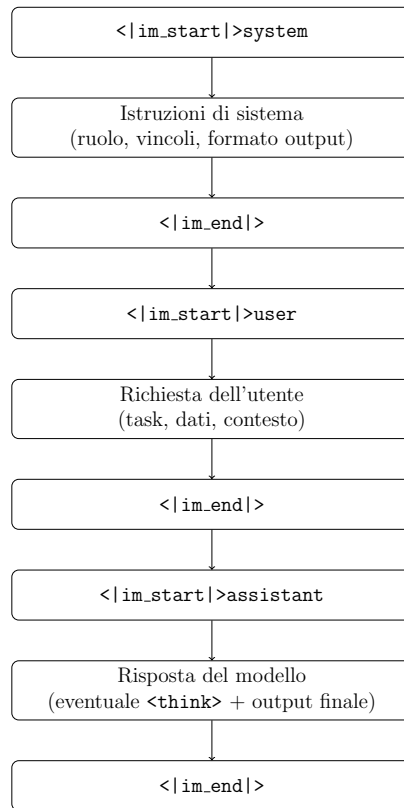


Figura 2.18: Struttura dei prompt Qwen2.5 basata su token speciali e turni conversazionali.

Un prompt efficace definisce il ruolo del modello, stabilisce vincoli formali e specifica il formato dell'output. Nel sistema descritto in questa tesi, i prompt assumono una funzione architettonica: guidano il modello attraverso le diverse fasi del workflow, impongono strutture deterministiche (come l'obbligo di produrre output in JSON), limitano comportamenti indesiderati e riducono l'ambiguità semantica. Il backend LLM contribuisce inoltre a orchestrare il processo, indirizzando il modello verso la generazione della risposta finale in modo controllato e coerente con i vincoli definiti.

Un esempio significativo è il prompt per l'analisi semantica dei CSV, che impone al modello di analizzare i file tramite strumenti dedicati, formulare domande non ridondanti e rispettare una struttura fissa:

“Your task in this phase is ONLY to analyze the data and ask semantic clarification questions when needed. You MUST NOT generate SQL schema or column mapping in this phase.”

Un secondo esempio riguarda la generazione deterministica di specifiche Vega-Lite e query SQL, dove il prompt stabilisce vincoli stringenti sull'output:

“Generate a Vega-Lite v6 chart specification WITHOUT the data field and a MySQL SQL query that retrieves exactly the fields used in the chart. Output ONLY a JSON object.”

Analogamente, il prompt per la classificazione delle transazioni impone un comportamento privo di spiegazioni e completamente deterministico:

“You MUST return at least one category. Output ONLY a JSON object. No explanations, no reasoning.”

Infine, il prompt per la generazione del nome della transazione mostra l'uso di vincoli sintattici e semantici:

“Use snake_case. The name must reflect the business purpose of the request.”

Questi esempi dimostrano come il prompt engineering consenta di trasformare un modello general-purpose in un componente affidabile e controllato, capace di operare in pipeline complesse con requisiti di determinismo, coerenza e verificabilità. Tale approccio è coerente con la visione emergente del prompt engineering come disciplina sistematica e fondamentale per l'utilizzo avanzato dei modelli LLM [Liu et al., 2023, Zhou et al., 2023].

Capitolo 3

Stato dell'Arte

La gestione dei dati sperimentali rappresenta oggi un elemento cruciale per i laboratori di microbiologia e, più in generale, per le scienze della vita. La crescente quantità di informazioni prodotte quotidianamente, spesso in forma semi-strutturata o eterogenea, rende necessario l'impiego di strumenti capaci di trasformare tali dati in strutture coerenti, interrogabili e adatte all'analisi computazionale. Nel panorama attuale esistono diverse soluzioni che affrontano questo problema da prospettive differenti. Alcune si concentrano sulla conversione diretta di fogli di calcolo in basi di dati, altre offrono sistemi complessi per la gestione dei campioni e dei workflow di laboratorio, mentre altre ancora privilegiano la documentazione delle attività sperimentali. A queste categorie si aggiungono ecosistemi modulari pensati per la gestione di dati biologici complessi, come GMOD. Il software sviluppato in questo lavoro si colloca in una posizione intermedia tra tali approcci, combinando facilità d'uso, capacità di trasformazione dei dati e integrazione con modelli LLM per la generazione di vetrine e la conversione dei dati da semi-strutturati a strutturati.

3.1 Strumenti Excel-to-SQL e motori di trasformazione di fogli di calcolo

Gli strumenti appartenenti alla categoria Excel-to-SQL sono progettati per convertire automaticamente fogli di calcolo in script SQL o in tabelle relazionali. Soluzioni come Sheet2SQL, Excel-Conversion-Tool e XcelSQL operano generalmente secondo un principio semplice: interpretano il contenuto tabellare e lo traducono in istruzioni `CREATE TABLE` e `INSERT INTO` [Fungo, 2023, Lin, 2024, Naqvi, 2024]. Questo processo è particolarmente utile quando si desidera trasferire rapidamente dati da un formato tabellare

a un database relazionale, ad esempio in operazioni di migrazione o in pipeline ETL, dove l'obiettivo principale è la continuità del flusso informativo più che la modellazione approfondita del dominio.

Tuttavia, tali strumenti si limitano a una trasformazione di tipo sintattico. La struttura dei dati viene tradotta in colonne e righe, ma il significato delle informazioni non viene interpretato. Non vengono riconosciute entità di dominio, come campioni, esperimenti o protocolli, né vengono proposte relazioni tra tali entità. Inoltre, quasi nessuno strumento appartenente a questa categoria è in grado di produrre uno schema realmente conforme alla terza forma normale. Quest'ultima rappresenta uno dei livelli più avanzati del processo di normalizzazione dei database relazionali e richiede che ogni attributo di una tabella dipenda esclusivamente dalla chiave primaria e non da altri attributi non chiave [Elmasri and Navathe, 2016]. In altre parole, la terza forma normale elimina le dipendenze transitive e impedisce che informazioni concettualmente distinte vengano collocate nella stessa tabella. Rispettare tale vincolo significa evitare ridondanze, prevenire inconsistenze e garantire che ogni dato sia collocato nel punto corretto dello schema logico. Poiché gli strumenti Excel-to-SQL operano senza alcuna comprensione semantica del dominio e si limitano a convertire la struttura superficiale del foglio di calcolo, essi non dispongono delle informazioni necessarie per identificare correttamente le dipendenze funzionali e, di conseguenza, non possono generare automaticamente un database normalizzato secondo i criteri della terza forma normale. Inoltre, questi strumenti non includono funzionalità di gestione degli utenti, sistemi di autorizzazione o audit trail. Il software in esame, invece, riesce mediante l'impiego di un modello LLM a comprendere la semantica di ciascun dato così da poter generare un database in terza forma normale. In un contesto laboratoristico, dove è essenziale sapere chi ha modificato cosa e quando, tale mancanza rappresenta un limite importante. L'interfaccia, spesso basata su riga di comando o script, richiede competenze tecniche che non tutti i ricercatori possiedono. Per questi motivi, pur essendo strumenti utili per compiti specifici, risultano insufficienti per la gestione strutturata e sicura dei dati sperimentali.

3.2 LIMS open source e gestione strutturata dei campioni

I sistemi LIMS (Laboratory Information Management System) rappresentano la soluzione più completa per la gestione dei dati di laboratorio. Strumenti come SENAITE e Aurora sono progettati per coprire l'intero ciclo di

vita del campione, dalla sua registrazione iniziale fino alla generazione del report finale [SENAITE, 2024, Zareie-Vaux and Abernethy, 2024]. La loro forza risiede nella capacità di modellare in modo esplicito le entità tipiche del laboratorio — campioni, analisi, protocolli, risultati — e di garantire un elevato livello di tracciabilità grazie a meccanismi di audit trail conformi agli standard internazionali, come la norma ISO/IEC 17025. In pratica, ogni operazione compiuta sul sistema viene registrata, consentendo di ricostruire la storia completa di un campione e delle analisi ad esso associate. Questi sistemi includono anche funzionalità avanzate di gestione degli utenti e dei permessi. È possibile definire ruoli distinti (ad esempio tecnico di laboratorio, responsabile di qualità, amministratore) e associare a ciascuno di essi specifici diritti di accesso. Ciò consente di controllare chi può visualizzare, modificare o approvare determinate informazioni, aumentando la sicurezza e la conformità alle normative. Tuttavia, la ricchezza funzionale dei LIMS comporta una complessità non trascurabile. La loro adozione richiede spesso una fase di configurazione articolata, l'integrazione con strumenti di laboratorio e la formazione del personale. Per gruppi di ricerca che utilizzano principalmente fogli di calcolo e che necessitano soprattutto di una migrazione verso una base di dati strutturata, un LIMS completo può risultare sovradimensionato rispetto alle reali esigenze operative.

3.3 ELN e piattaforme per la gestione degli esperimenti

Le piattaforme ELN (Electronic Lab Notebook), come SciNote, sono pensate per sostituire il taccuino di laboratorio tradizionale con uno strumento digitale che favorisca la documentazione, la collaborazione e la riproducibilità delle attività sperimentali [SciNote, 2024]. In un ELN, il ricercatore può organizzare i propri progetti in modo gerarchico, definire esperimenti, descrivere protocolli, allegare file (inclusi fogli di calcolo) e annotare osservazioni. Questo approccio consente di centralizzare le informazioni e di renderle facilmente condivisibili all'interno del gruppo di lavoro.

Nonostante la loro utilità, gli ELN non sono progettati per affrontare la trasformazione sistematica di dati semi-strutturati in schemi relazionali normalizzati. La loro funzione principale è la gestione della documentazione e dei flussi di lavoro, non la normalizzazione dei dati o la loro integrazione in un database strutturato. Di conseguenza, pur rappresentando un passo importante verso la digitalizzazione del laboratorio, essi non risolvono il problema

della migrazione dei dataset tabellari verso una base di dati relazionale, né offrono strumenti avanzati per l'analisi automatizzata dei dati.

3.4 Confronto delle soluzioni con il software in sviluppo

La Tabella 3.1 riassume le principali differenze tra il software in sviluppo e le categorie di strumenti analizzate.

Categoria di Strumento	Caratteristiche Principali	Punti di Forza	Limiti
Convertitori Excel-to-SQL	Strumenti focalizzati sulla trasformazione sintattica di fogli di calcolo in script SQL o tabelle. Esempi: Sheet2SQL, Excel-Conversion-Tool, Xcel-SQL [Fungo, 2023, Lin, 2024, Naqvi, 2024].	Semplici da usare, veloci, utili per operazioni ETL e conversioni immediate.	Assenza di gestione utenti, nessun audit trail, nessuna modellazione semantica del dominio, interfacce tecniche, impossibilità di produrre schemi normalizzati.
LIMS Open Source	Sistemi completi per la gestione dei campioni, dei workflow e dei risultati. Esempi: SENAITE, Aurora LIMS [SENAITE, 2024, Zareie-Vaux and Abernethy, 2024]	Audit trail avanzato, gestione utenti, conformità a standard, integrazione con strumenti di laboratorio.	Elevata complessità, sovradimensionati per gruppi che usano principalmente fogli Excel, curva di apprendimento significativa.
ELN (Electronic Lab Notebook)	Piattaforme orientate alla documentazione degli esperimenti, protocolli e attività di laboratorio. Esempio: SciNote [SciNote, 2024].	Collaborazione, organizzazione dei progetti, gestione di file eterogenei.	Non progettati per la conversione sistematica di dati semi-strutturati in schemi relazionali; limitata automazione dei flussi di migrazione.
Software Proposto	Sistema dedicato alla migrazione da fogli Excel semi-strutturati a una base di dati strutturata, con gestione utenti, audit trail, file management e generazione di vetrine tramite LLM.	Progettato per la microbiologia, interfaccia web user-friendly, integrazione LLM nativa, audit trail, gestione permessi, flusso di migrazione ottimizzato.	Meno generale di un LIMS completo; richiede configurazione iniziale dello schema.

Tabella 3.1: Confronto tra il software sviluppato e le principali categorie di strumenti affini.

Il confronto evidenzia come il software proposto si distingua dai convertitori Excel-to-SQL per la capacità di interpretare la struttura dei dati e di modellare entità e relazioni rilevanti per la microbiologia. Rispetto ai LIMS, esso offre un approccio più leggero e immediato, evitando la complessità gestionale tipica dei sistemi enterprise e concentrandosi sulla migrazione dei dati da fogli di calcolo. Rispetto agli ELN, integra funzionalità di trasformazione dei dati e non si limita alla sola documentazione delle attività sperimentali. In questo modo, il sistema si configura come una soluzione mirata per gruppi di ricerca che desiderano superare i limiti dei fogli di calcolo senza adottare infrastrutture troppo complesse.

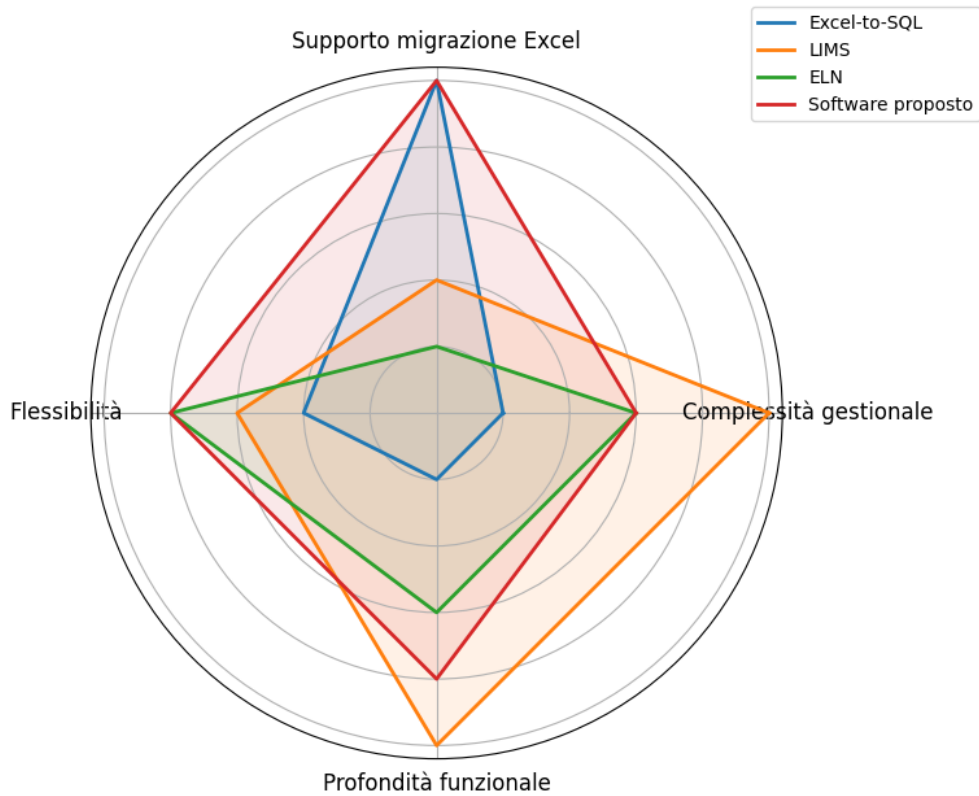


Figura 3.1: Posizionamento del software proposto rispetto a strumenti Excel-to-SQL e LIMS/ELN.

La Figura 3.1 confronta diverse soluzioni software rispetto a quattro dimensioni: complessità gestionale, supporto alla migrazione da Excel, profondità funzionale e flessibilità. Il software proposto si colloca in una posizione intermedia ed equilibrata: offre un buon livello di supporto alla migrazione dei fogli di calcolo, mantiene una complessità gestionale contenuta e garantisce una flessibilità superiore rispetto ai sistemi più strutturati. Pur non raggiungendo la profondità funzionale tipica dei LIMS, rappresenta un compromesso efficace che consente di trasformare dati semi-strutturati in basi di dati relazionali in modo più semplice e diretto.

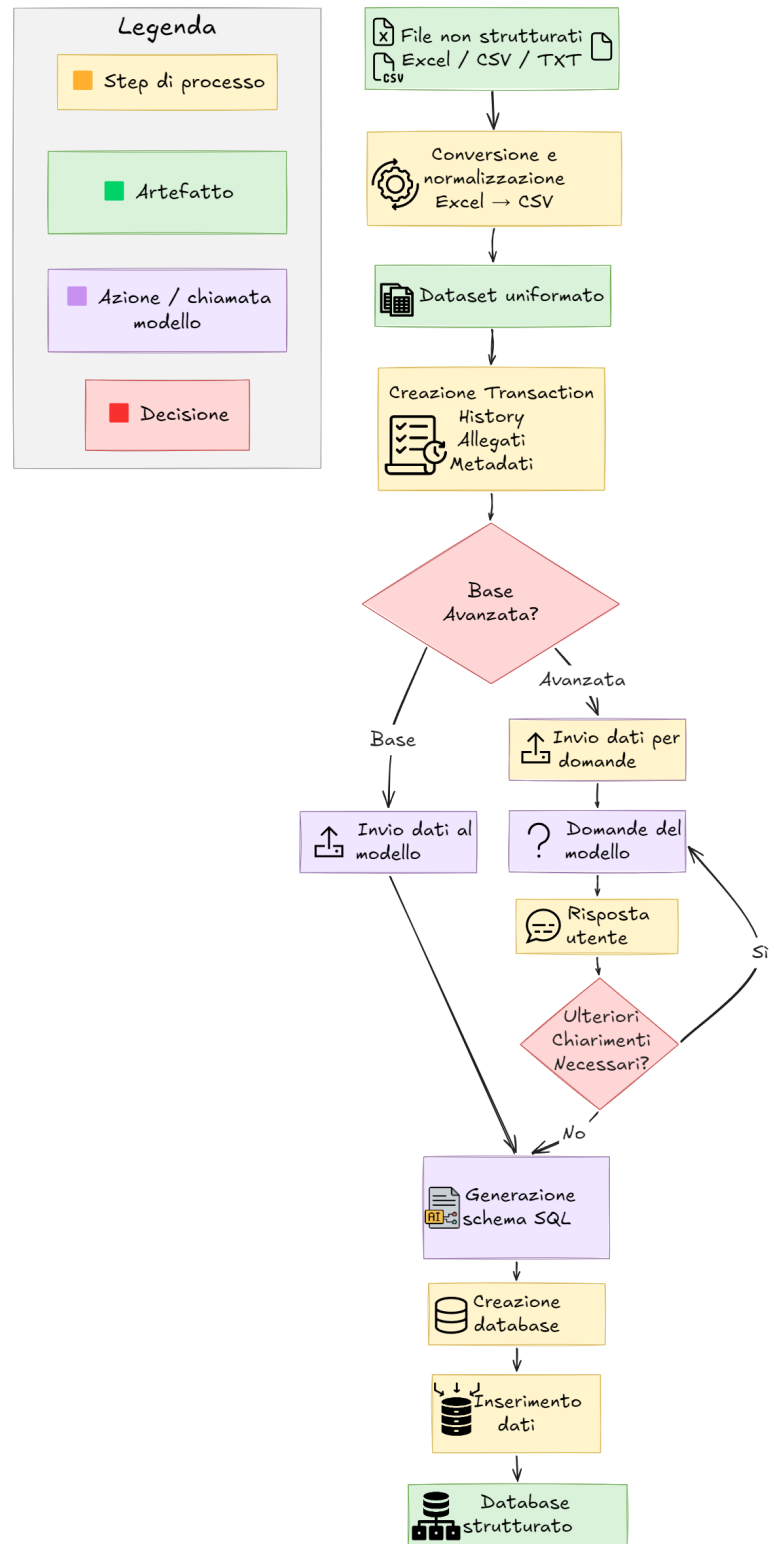


Figura 3.2: Schema del flusso di lavoro del software in analisi.

La Figura 3.2 mostra il flusso end-to-end: importazione dei fogli di calcolo, normalizzazione e mappatura verso lo schema relazionale, persistenza nel database. Questo schema aiuta a comprendere come il sistema accompagni il ricercatore in tutte le fasi della migrazione, offrendo strumenti specifici per ciascun passaggio.

3.5 GMOD: un ecosistema modulare per la gestione di dati biologici

Oltre alle categorie di strumenti precedentemente analizzate, merita particolare attenzione il progetto GMOD (Generic Model Organism Database Project), un ecosistema modulare e open source sviluppato per supportare la gestione, l'annotazione e la visualizzazione di dati biologici eterogenei [GMOD, 2024]. GMOD non è un singolo software, ma una collezione di componenti interoperabili progettati per costruire database biologici complessi, in particolare per organismi modello. Tra i moduli più rilevanti si possono citare Chado, uno schema di database altamente normalizzato; Tripal, un framework basato su Drupal per la creazione di portali web; JBrowse, un genome browser moderno e interattivo; e Apollo, una piattaforma collaborativa per l'annotazione manuale di genomi.

GMOD si distingue per la profondità della modellazione del dominio biologico e per la capacità di gestire dataset genomici di grandi dimensioni. Tuttavia, la sua complessità strutturale lo rende poco adatto a contesti in cui i dati di partenza sono fogli di calcolo semi-strutturati. L'ecosistema presuppone infatti che le informazioni siano già organizzate secondo standard biologici consolidati, come ontologie o formati specifici della genomica, e richiede competenze tecniche avanzate (ad esempio nella gestione di PostgreSQL e nella definizione di schemi complessi). Inoltre, GMOD non integra nativamente modelli LLM per la generazione automatica di visualizzazioni o vetrine, né è pensato per workflow microbiologici generici basati su dati tabellari.

3.5.1 Confronto con il software in sviluppo

La Tabella 3.2 estende il confronto precedente includendo GMOD.

Categoria di Strumento	Caratteristiche Principali	Punti di Forza	Limiti
GMOD	Ecosistema modulare per la gestione di dati biologici complessi. Include Chado, Tripal, JBrowse, Apollo [GMOD, 2024].	Schema biologico avanzato, elevata modularità, grande scalabilità, strumenti maturi per genomica e annotazione collaborativa.	Non orientato ai fogli Excel, complessità elevata, nessuna integrazione LLM, non focalizzato su dati microbiologici tabellari.
Software Proposto	Sistema dedicato alla migrazione da fogli Excel semi-strutturati a una base di dati strutturata, con gestione utenti, audit trail, file management e generazione di vetrine tramite LLM.	Progettato per la microbiologia, interfaccia web user-friendly, integrazione LLM nativa, audit trail, gestione permessi, flusso di migrazione ottimizzato.	Meno generale di un LIMS completo; richiede configurazione iniziale dello schema.

Tabella 3.2: Confronto tra GMOD e il software sviluppato.

Rispetto a GMOD, il software proposto si concentra sulla migrazione da dati semi-strutturati, mentre GMOD presuppone dati già strutturati secondo standard biologici complessi. Il sistema sviluppato offre un’interfaccia più semplice e orientata ai ricercatori, senza richiedere competenze su ontologie o modelli genomici, e integra nativamente modelli LLM per la generazione di grafici e vetrine, funzionalità assente nell’ecosistema GMOD. Inoltre, è progettato per dati microbiologici sperimentali, non per la genomica su larga scala.

3.6 Valutazione complessiva

In sintesi, il software in sviluppo si propone come una soluzione specializzata per la migrazione di dati semi-strutturati in ambito microbiologico verso una base di dati strutturata. Esso risulta più accessibile rispetto ai LIMS ed ecosistemi complessi come GMOD, grazie a un’interfaccia web user-friendly e a una minore complessità di adozione, e più ricco rispetto ai semplici strumenti Excel-to-SQL, grazie all’integrazione di gestione utenti, audit trail, moduli per la gestione dei file e un componente LLM per la generazione di vetrine e visualizzazioni e conversione dei file semi-strutturati in file strutturati. Queste caratteristiche lo rendono particolarmente adatto a gruppi di ricerca che desiderano superare i limiti dei fogli di calcolo mantenendo, al contempo, un controllo fine sulla qualità, la tracciabilità e la comunicabilità dei propri dati sperimentali.

Capitolo 4

Metodologia

Il presente capitolo descrive in modo sistematico il processo di realizzazione del software analizzato, illustrando le scelte architetturali e tecnologiche adottate durante lo sviluppo. Per garantire una comprensione chiara e progressiva, il capitolo è articolato in sezioni tematiche, ciascuna dedicata a un componente specifico dell'infrastruttura applicativa. La prima sezione introduce l'architettura basata su Docker, impiegata sia in fase di sviluppo sia in fase di esecuzione. L'immagine seguente mostra il funzionamento generale del software in analisi.

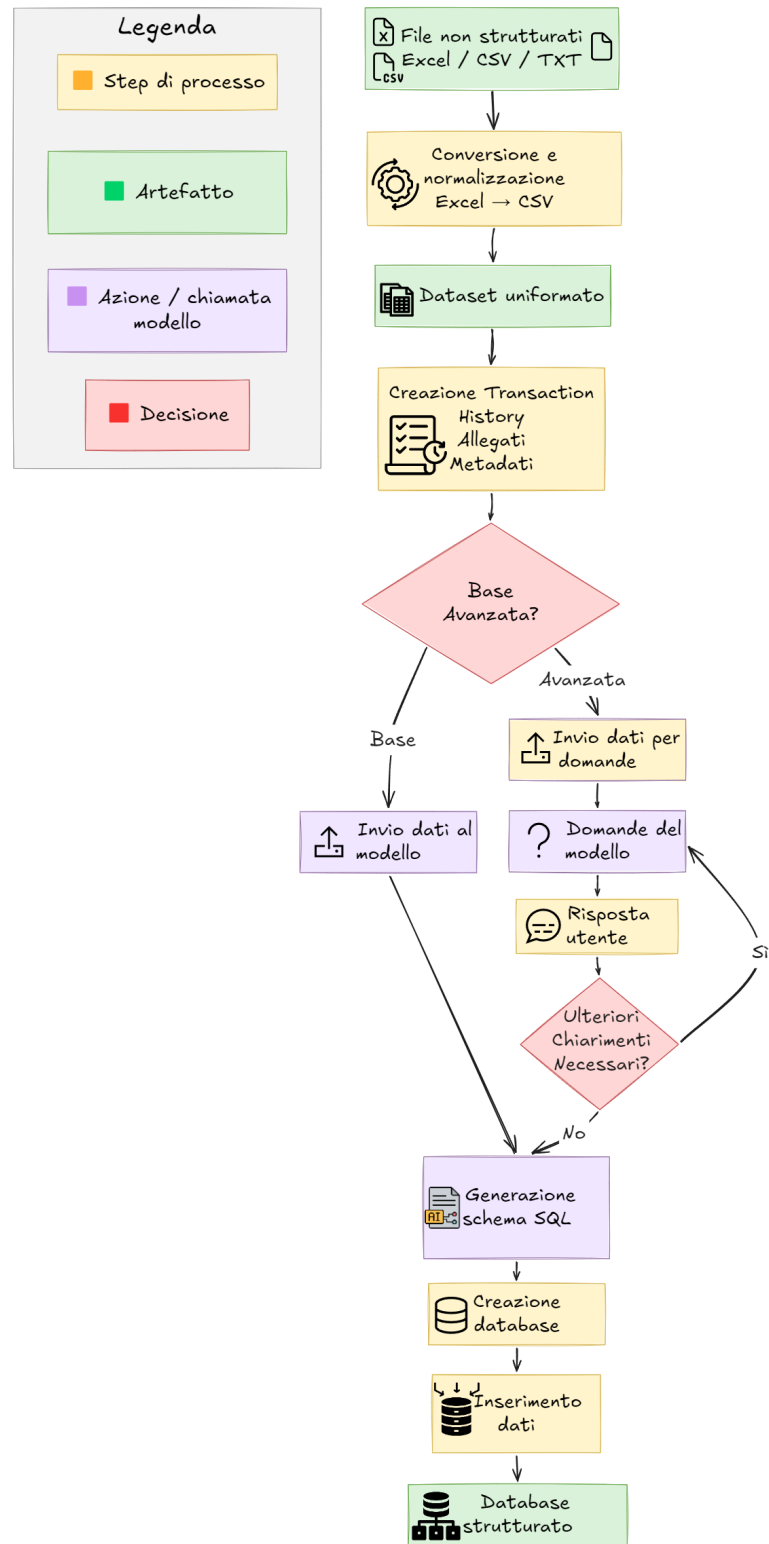


Figura 4.1: Schema del flusso di lavoro del software in analisi.

4.1 Docker

L'esigenza di garantire la portabilità del software su piattaforme eterogenee, incluse architetture diverse da quelle utilizzate durante lo sviluppo (principalmente Windows 11), ha reso opportuno l'impiego della tecnologia dei container. L'adozione di Docker consente infatti di superare molte delle limitazioni tipiche di un'installazione tradizionale, come la dipendenza dalla configurazione del sistema host, la difficoltà di replicare l'ambiente di esecuzione e la gestione complessa delle versioni dei servizi necessari [Merkel, 2014].

Una soluzione priva di container presenta vincoli significativi: l'installazione manuale dei servizi può generare conflitti tra versioni, la portabilità risulta ridotta, la replicabilità dell'ambiente è difficilmente garantibile e la manutenzione richiede interventi diretti sul sistema operativo. Inoltre, l'assenza di isolamento comporta il rischio di interferenze tra applicazioni diverse e rende complessa la gestione delle dipendenze.

Per definire l'ambiente di esecuzione è stato predisposto un Dockerfile, ovvero un file di configurazione dichiarativo che specifica i passaggi necessari alla costruzione dell'immagine del container. Tale file consente di descrivere in modo riproducibile l'installazione dei servizi, la configurazione del sistema e l'inclusione delle dipendenze, garantendo che ogni istanza generata sia identica alle precedenti [Boettiger, 2015].

Come immagine di base è stata scelta Ubuntu 26.04, selezionata sia per la sua attualità sia per la disponibilità di pacchetti preinstallati utili all'esecuzione dei servizi richiesti. All'interno del Dockerfile sono stati inseriti i comandi necessari per installare Apache con supporto PHP, MariaDB, MongoDB e, qualora richiesto dalla variabile d'ambiente OLLAMA_USE_LOCAL, anche il client Ollama.

Per semplificare la gestione delle variabili d'ambiente, dei volumi e del mapping delle porte, è stato utilizzato un file docker-compose, che consente di orchestrare l'intero ambiente in modo centralizzato. Le variabili d'ambiente impiegate e la loro funzione sono riportate nella Tabella 4.1.

La differenza tra le due modalità di integrazione del servizio Ollama, rispettivamente in esecuzione all'interno del container oppure sull'host, è illustrata nei diagrammi di distribuzione riportati di seguito.

CAPITOLO 4. METODOLOGIA

Variabile	Descrizione
MYSQL_ROOT_PASSWORD	Password dell'utente root di MySQL con privilegi amministrativi.
MYSQL_DATABASE	Nome del database MySQL utilizzato dall'applicazione.
MYSQL_USER	Utente MySQL dedicato all'applicazione.
MYSQL_PASSWORD	Password dell'utente applicativo MySQL.
MYSQL_HOST	Host o indirizzo IP del server MySQL.
MONGODB_HOST	URI di connessione al server MongoDB.
MONGODB_COLLECTION	Nome della collezione MongoDB utilizzata dall'applicazione.
MONGO_ADMIN_USER	Utente amministratore di MongoDB.
MONGO_ADMIN_PASSWORD	Password dell'utente amministratore MongoDB.
MONGO_APP_USER	Utente MongoDB dedicato all'applicazione.
MONGO_APP_PASSWORD	Password dell'utente applicativo MongoDB.
AUTHENTICATION_SOURCE	Database MongoDB usato come authentication source.
TOKEN_SECRET_KEY	Chiave segreta per la firma dei token JWT.
TOKEN_EXPIRATION	Durata di validità dei token JWT (in secondi).
OLLAMA_USE_LOCAL	Indica se usare un'istanza locale di Ollama.
OLLAMA_HOST	URL dell'host Ollama da utilizzare.
OLLAMA_MODEL	Modello Ollama da utilizzare.

Tabella 4.1: Descrizione delle variabili d'ambiente utilizzate.

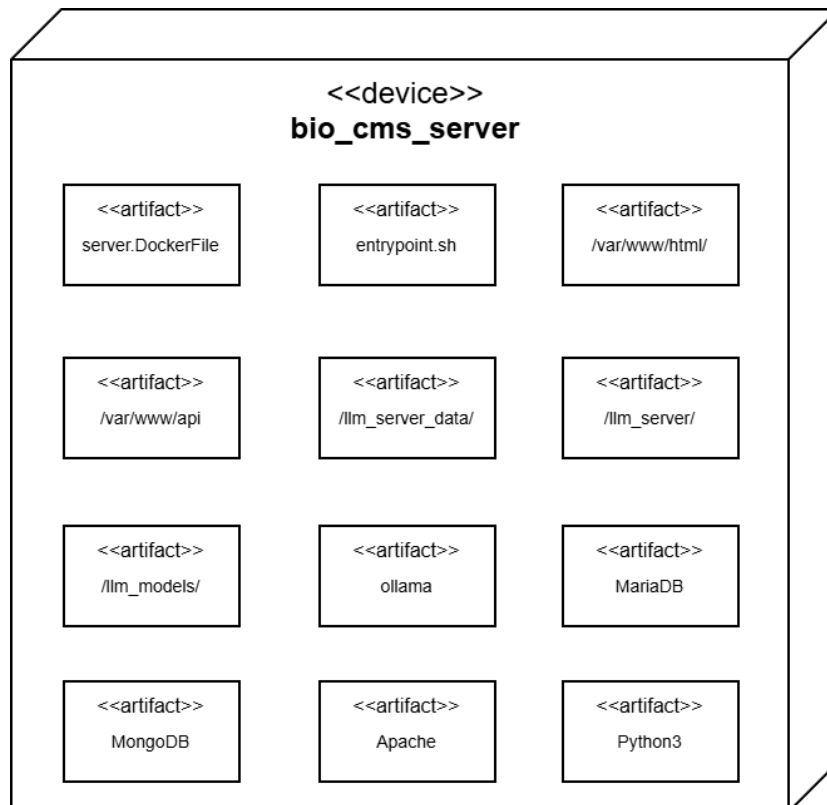


Figura 4.2: Diagramma di distribuzione con Ollama installato nel container.

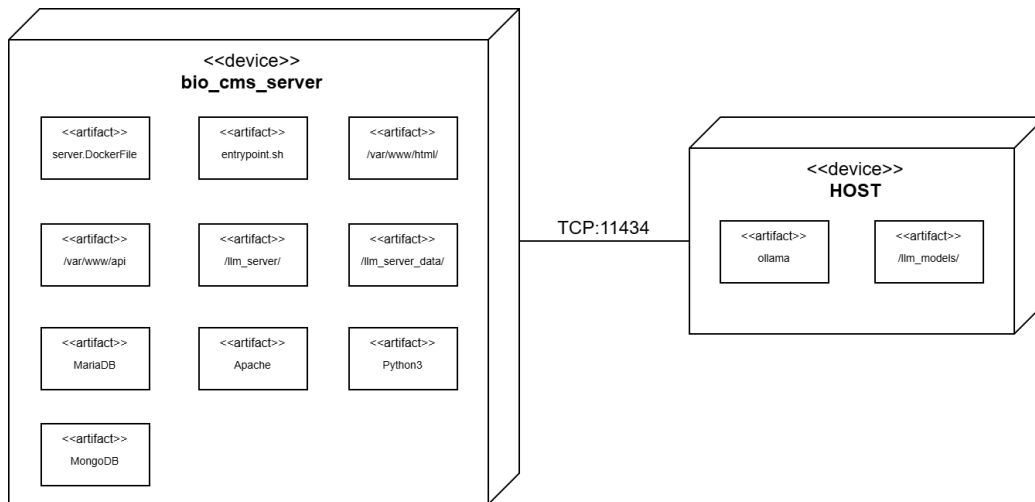


Figura 4.3: Diagramma di distribuzione con Ollama installato sull'host.

4.1.1 Configurazione Apache

Per supportare i diversi sotto-applicativi sviluppati (explorer, management, manual, backend) è stato necessario configurare più VirtualHost all'interno della stessa istanza di Apache. Un VirtualHost rappresenta un meccanismo che consente al server web di rispondere a richieste differenti come se si trattasse di siti o servizi distinti, pur condividendo la medesima installazione di Apache. Tale separazione logica permette di instradare le richieste in base alla porta, al nome di dominio o ad altri parametri, garantendo un'organizzazione modulare e una gestione indipendente delle diverse componenti applicative.

Nel caso specifico, i VirtualHost sono stati utilizzati per consentire la gestione di richieste su porte differenti, assegnando a ciascun sotto-applicativo un proprio contesto di esecuzione. Inoltre, per garantire la possibilità di caricare file fino a 1024 MB, sono stati modificati i parametri `upload_max_filesize` e `post_max_size`. La configurazione completa dei VirtualHost è riportata nel listato incluso nel documento.

```

1 ServerName localhost
2 <VirtualHost *:80>
3     ServerName localhost
4     DocumentRoot /var/www/explorer
5     <Directory /var/www/explorer>
6         Options Indexes FollowSymLinks
7         AllowOverride All
8         Require all granted
  
```

```
9     </Directory>
10     ErrorLog ${APACHE_LOG_DIR}/error_80.log
11     CustomLog ${APACHE_LOG_DIR}/access_80.log combined
12 </VirtualHost>
13
14 <VirtualHost *:8080>
15     ServerName localhost
16     DocumentRoot /var/www/management
17     <Directory /var/www/management>
18         Options Indexes FollowSymLinks
19         AllowOverride All
20         Require all granted
21     </Directory>
22     ErrorLog ${APACHE_LOG_DIR}/error_8080.log
23     CustomLog ${APACHE_LOG_DIR}/access_8080.log combined
24 </VirtualHost>
25
26 <VirtualHost *:8081>
27     ServerName localhost
28     DocumentRoot /var/www/manual
29     <Directory /var/www/manual>
30         Options Indexes FollowSymLinks
31         AllowOverride All
32         Require all granted
33     </Directory>
34     ErrorLog ${APACHE_LOG_DIR}/error_8081.log
35     CustomLog ${APACHE_LOG_DIR}/access_8081.log combined
36 </VirtualHost>
37
38 <VirtualHost *:8082>
39     ServerName localhost
40     DocumentRoot /var/www/api
41     <Directory /var/www/api>
42         Options Indexes FollowSymLinks
43         AllowOverride All
44         Require all granted
45     </Directory>
46     ErrorLog ${APACHE_LOG_DIR}/error_8082.log
47     CustomLog ${APACHE_LOG_DIR}/access_8082.log combined
48 </VirtualHost>
```

La configurazione riportata nel listato definisce quattro distinti Virtua-

lHost, ciascuno associato a una porta differente del server Apache. Ogni VirtualHost instrada le richieste verso un sotto-applicativo specifico, identificato dal rispettivo DocumentRoot. Per ciascuna directory viene inoltre abilitato l'utilizzo dei file .htaccess tramite la direttiva AllowOverride All, e viene garantito l'accesso ai contenuti mediante Require all granted. La configurazione prevede anche la separazione dei file di log, con un ErrorLog e un CustomLog dedicati per ogni VirtualHost, così da facilitare il monitoraggio e la diagnosi delle richieste relative ai diversi servizi applicativi.

4.1.2 Configurazione MariaDB

La configurazione iniziale di MariaDB è stata automatizzata mediante uno script shell, che provvede alla creazione degli utenti con le credenziali specificate nelle variabili d'ambiente e alla modifica dei permessi dell'utente amministratore, consentendone l'accesso anche dall'esterno del container. Lo script sincronizza inoltre la timezone del demone MariaDB con quella del container, garantendo coerenza temporale durante l'analisi dei log. Successivamente viene importata una configurazione iniziale del database bioCMS_db, utilizzato per la registrazione delle attività applicative.

La struttura logica del database bioCMS_db, utilizzato per la gestione dei log applicativi e delle informazioni operative, è rappresentata nello schema entità-relazioni riportato di seguito.

CAPITOLO 4. METODOLOGIA

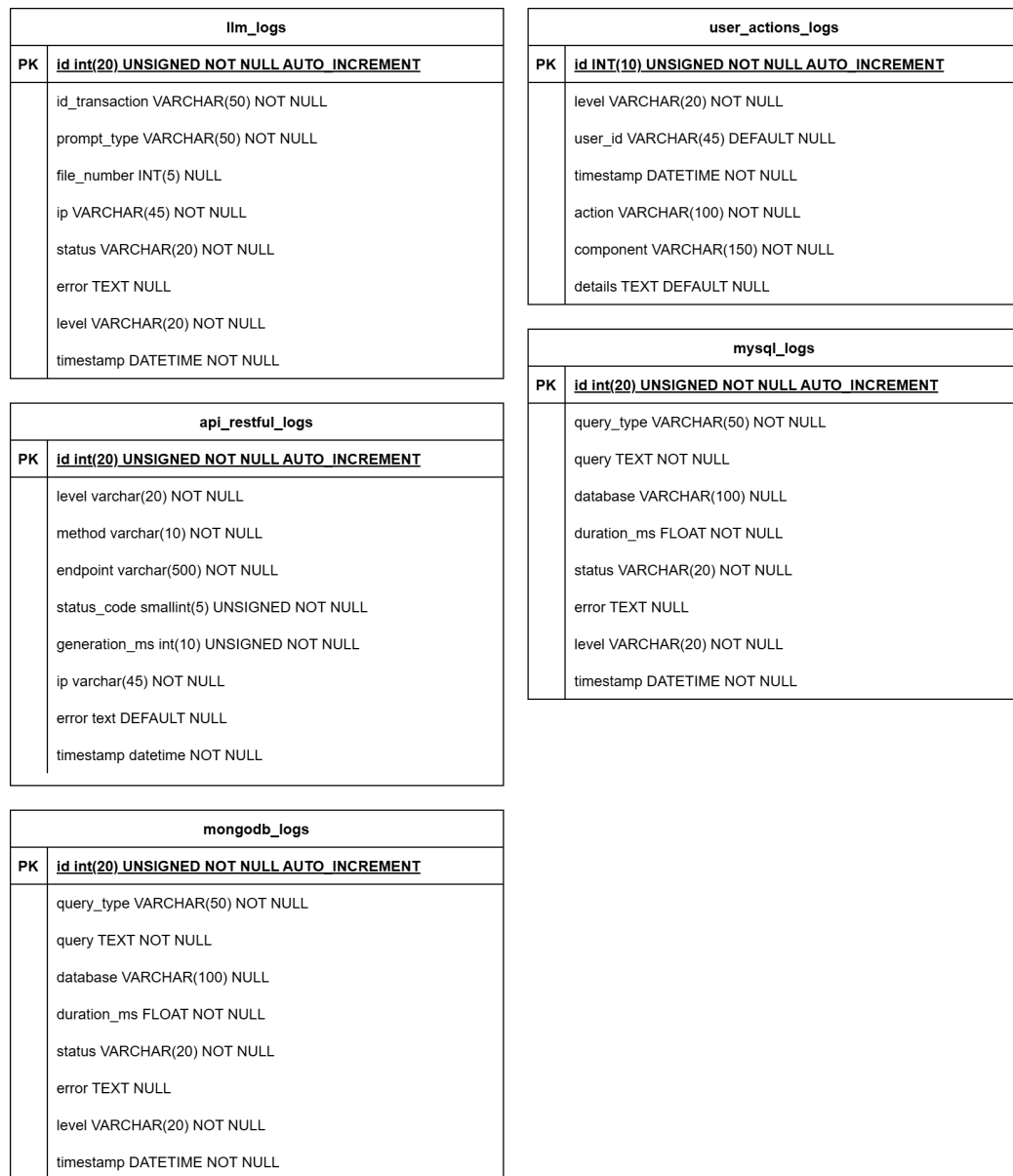


Figura 4.4: Schema ER del database bioCMS_db.

4.1.3 Configurazione MongoDB

Analogamente a quanto fatto per MariaDB, anche la configurazione di MongoDB è stata automatizzata tramite uno script shell. Tale script crea gli utenti specificati nelle variabili d'ambiente, inizializza il database bioCMS e genera le collezioni necessarie all'applicativo, tra cui transactions, categories, groups e users. Al termine della procedura viene creato un file marker che

indica l'avvenuta configurazione, evitando di ripetere l'operazione in caso di riavvio del container.

4.1.4 Configurazione Ollama

L'installazione di Ollama all'interno del container dipende dal valore della variabile d'ambiente `OLLAMA_USE_LOCAL`. Se impostata a `true`, viene installato il client Ollama e scaricato il modello specificato, in questo caso `Qwen2.5-Coder-7b-Instruct`. È inoltre necessario configurare la variabile `OLLAMA_HOST` affinché punti all'istanza corretta del servizio, che può risiedere nel container o sull'host o su altro server.

4.1.5 Configurazione dell'ambiente Python

L'ultima fase della procedura di inizializzazione riguarda la creazione dell'ambiente Python necessario all'esecuzione del backend LLM. A tal fine viene creato un ambiente virtuale (`venv`), ovvero un ambiente isolato che consente di installare dipendenze Python senza interferire con il sistema operativo o con altri progetti. Tale meccanismo permette di garantire riproducibilità, isolamento delle librerie e stabilità dell'ambiente di esecuzione [Foundation, 2024].

All'interno del `venv` vengono installate tutte le dipendenze elencate nel file `requirements.txt`, il quale definisce in modo esplicito le librerie necessarie al funzionamento del backend Python. In particolare, esso include `Flask` per la gestione del server web, `Pydantic` per la validazione strutturata dei dati, `Ollama` per l'interazione con il modello linguistico e `Pandas` per l'elaborazione dei dati tabellari. Il contenuto del file è riportato di seguito.

```
1 flask
2 pydantic
3 ollama
4 pandas
```

Successivamente viene avviato il server Python, che rimane in ascolto sulla porta 4891 per gestire le richieste provenienti dal backend PHP.

4.2 Backend LLM

Il backend si fonda su un'architettura modulare che separa in modo netto le responsabilità tra livello di esposizione HTTP, logica di orchestrazione della

conversazione, gestione dei file e validazione delle risposte. Il server principale utilizza il micro-framework Flask come punto di ingresso per tutte le richieste, mentre la logica di alto livello è incapsulata nella classe ChatEngine, che coordina le chiamate al modello LLM, l'esecuzione dei tool e la validazione del JSON risultante.

A livello concettuale, il sistema può essere visto come una pipeline in cui una richiesta HTTP viene trasformata in una sequenza di messaggi per il modello, arricchita con informazioni sui file caricati, sottoposta a iterazioni successive di generazione e, quando necessario, integrata con risultati provenienti da strumenti esterni. Il risultato finale è una risposta JSON che aderisce a uno dei modelli Pydantic definiti, garantendo così una struttura coerente e verificabile [Brown et al., 2020].

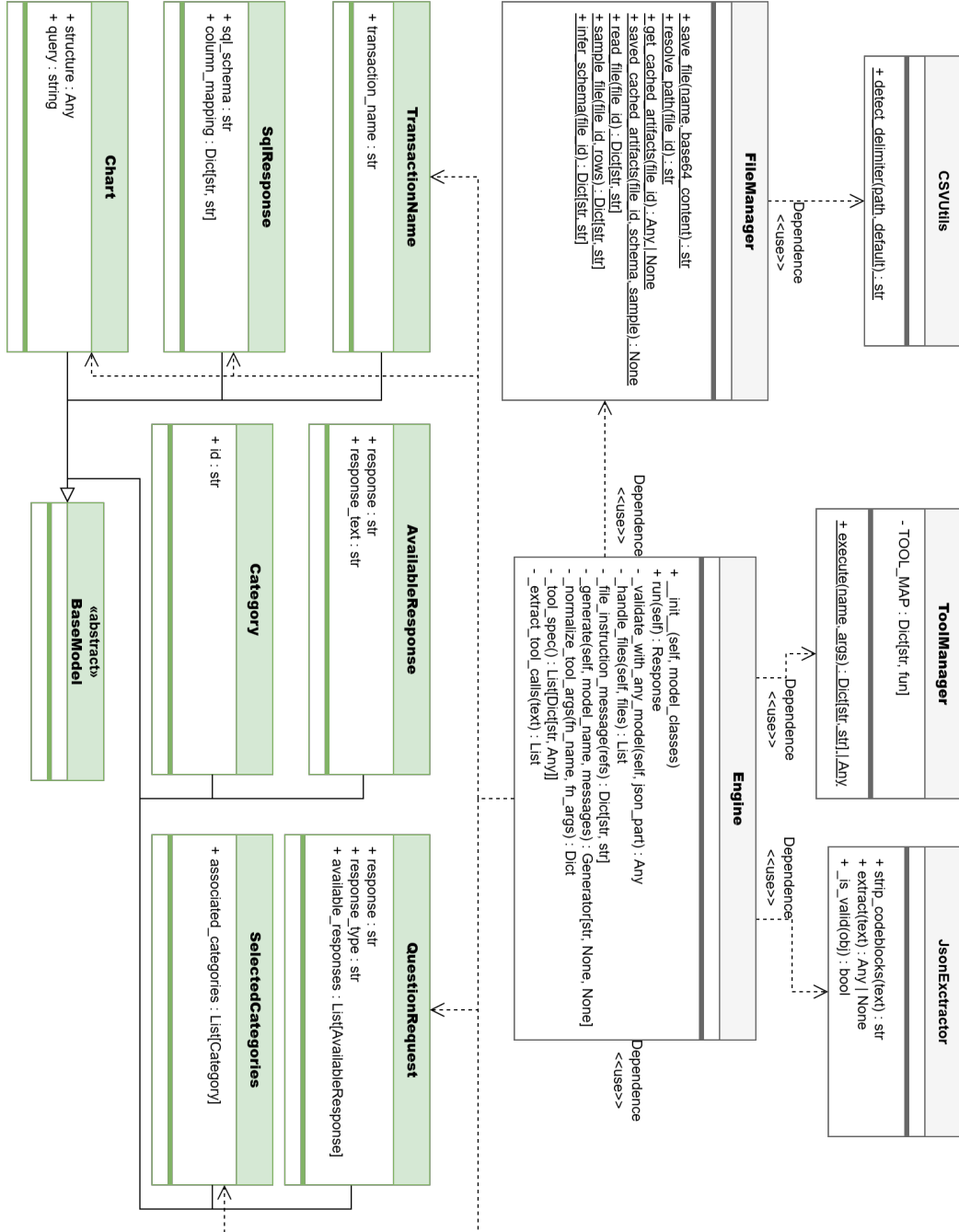


Figura 4.5: Diagramma delle classi backed LLM.

4.2.1 Server Flask e registrazione delle route

Il punto di ingresso dell'applicazione è definito nel modulo `server.py`, che inizializza l'istanza Flask e registra i blueprint corrispondenti alle diverse funzionalità esposte dal backend. Il codice seguente mostra la struttura essenziale del server.

```
1 from flask import Flask
2 from routes.routes_questions import bp_questions
3 from routes.routes_responses import bp_responses
4 from routes.routes_name_generation import bp_names
5 from routes.routes_category_selection import
    bp_categories
6 from routes.routes_chart_generation import bp_charts
7
8 app = Flask(__name__)
9
10 app.register_blueprint(bp_questions)
11 app.register_blueprint(bp_responses)
12 app.register_blueprint(bp_names)
13 app.register_blueprint(bp_categories)
14 app.register_blueprint(bp_charts)
15
16 @app.get("/")
17 def home():
18     return "LLM Server Running."
19
20 if __name__ == "__main__":
21     app.run(host="0.0.0.0", port=4891, threaded=True)
```

Codice 4.1: Server Flask principale

L'utilizzo dei blueprint consente di organizzare le route per dominio funzionale, mantenendo il server principale snello e facilmente estendibile. Ogni blueprint incapsula un insieme di endpoint che condividono una logica comune, come la generazione di domande, la produzione di risposte SQL, la generazione di nomi di transazioni, la selezione di categorie o la costruzione di strutture per grafici.

4.2.2 Modelli Pydantic e contratti di risposta

La validazione delle risposte del modello è affidata a una serie di modelli Pydantic definiti nel modulo `models.py`. Questi modelli rappresentano i con-

tratti strutturali che il backend si aspetta dal LLM. Di seguito è riportato un estratto significativo.

```
1 from pydantic import BaseModel
2 from typing import List, Dict, Any
3
4 class SqlResponse(BaseModel):
5     sql_schema: str
6     column_mapping: Dict[str, str]
7
8 class AvailableResponse(BaseModel):
9     response: str
10    response_text: str
11
12 class QuestionRequest(BaseModel):
13     request: str
14     request_type: str
15     available_responses: List[AvailableResponse]
16
17 class TransactionName(BaseModel):
18     transaction_name: str
19
20 class Category(BaseModel):
21     id: str
22
23 class SelectedCategories(BaseModel):
24     associated_categories: List[Category]
25
26 class Chart(BaseModel):
27     structure: Any
28     query: str
```

Codice 4.2: Modelli Pydantic per la validazione delle risposte

Ogni modello definisce un particolare tipo di risposta che il sistema può produrre. Ad esempio, `SqlResponse` descrive uno schema SQL e una mappatura tra colonne, mentre `QuestionRequest` rappresenta una richiesta di domanda con un insieme di risposte disponibili. La presenza di questi modelli consente al `ChatEngine` di tentare la validazione del JSON generato dal modello rispetto a più schemi possibili, selezionando quello che risulta coerente con il contenuto prodotto.

4.2.3 ChatEngine: orchestrazione del ciclo di chat

La classe ChatEngine costituisce il cuore logico del backend. Essa riceve in ingresso l'elenco delle classi di modello Pydantic utilizzabili per la validazione e implementa l'intero ciclo di interazione con il modello LLM, inclusa la gestione dello streaming, l'esecuzione dei tool e l'estrazione del JSON finale. Il metodo run funge da punto di ingresso per le route Flask:

```
1 class ChatEngine:
2     """Main orchestrator for the LLM chat loop."""
3
4     def __init__(self, model_classes: list):
5         self.model_classes = model_classes
6         self.max_iterations = 12
7         self.force_finalize_at = 8
8
9     def run(self) -> Response:
10        """Entry point for Flask routes."""
11        data = request.get_json(force=True)
12        model_name = data.get("model", "qwen2.5-coder:7b
-instruct")
13        messages = data.get("messages", [])
14        files = data.get("files", [])
15
16        print("[ENGINE] run() called")
17        print(f"[ENGINE] model: {model_name}")
18        print(f"[ENGINE] initial messages: {len(messages
)}}")
19        print(f"[ENGINE] files uploaded: {len(files)}")
20
21        file_refs = self._handle_files(files)
22        if file_refs:
23            print("[ENGINE] Adding file instructions to
messages")
24            messages.append(self.
_file_instruction_message(file_refs))
25
26        return Response(
27            self._generate(model_name, messages),
28            mimetype="text/plain"
29        )
```

Codice 4.3: Metodo run della ChatEngine

Il metodo `run` estrae dal corpo della richiesta il nome del modello, la lista dei messaggi e l'eventuale elenco di file caricati. I file vengono immediatamente delegati a `_handle_files`, che si occupa di salvarli e di recuperare eventuali artefatti di cache. Se sono presenti riferimenti a file, viene aggiunto ai messaggi un ulteriore messaggio di sistema che istruisce il modello su come utilizzare gli strumenti di analisi dei file.

Il ciclo di generazione vero e proprio è implementato nel metodo `_generate`, che esegue iterazioni successive di chiamate al modello, gestendo lo streaming dell'output, l'estrazione di eventuali chiamate a tool e la forzatura della finalizzazione dopo un certo numero di iterazioni. Un estratto significativo è riportato di seguito.

```

1 def _generate(self, model_name, messages) -> Generator[
2     str, None, None]:
3     print("[ENGINE] Starting _generate() loop")
4
5     ollama_messages = [
6         {"role": m["role"], "content": m["content"]} for
7         m in messages
8     ]
9
10    executed_tools = set()
11
12    for i in range(self.max_iterations):
13        print(f"\n[ENGINE] == Iteration {i+1}/{self.
14            max_iterations} ==")
15        print(f"[ENGINE] Sending {len(ollama_messages)}
16            messages to Ollama")
17
18        stream = call_ollama(
19            model_name=model_name,
20            messages=ollama_messages,
21            tools=self._tool_spec(),
22            stream=True
23        )
24
25        full_msg = ""
26        in_think = False
27        pending = ""
28        flush_start = 0
29        CLOSE_TAG = "</think>"
30        SAFE_TAIL = len(CLOSE_TAG)

```

```
27
28     for chunk in stream:
29         msg = chunk.get("message", {})
30         content = msg.get("content", "")
31         if not content:
32             continue
33
34         full_msg += content
35         pending += content
36
37         while True:
38             if not in_think:
39                 start = pending.find("<think>")
40                 if start == -1:
41                     if len(pending) > 50:
42                         pending = pending[-50:]
43                     break
44
45                 in_think = True
46                 pending = pending[start + len("<think>"):]
47                 flush_start = 0
48
49             else:
50                 end = pending.find(CLOSE_TAG)
51                 if end == -1:
52                     safe_limit = max(flush_start,
len(pending) - SAFE_TAIL)
53                     if safe_limit > flush_start:
54                         new_chunk = pending[
flush_start:safe_limit]
55                         if new_chunk:
56                             yield new_chunk
57                         flush_start = safe_limit
58                     break
59
60                 if flush_start < end:
61                     new_chunk = pending[flush_start:
end]
62                     if new_chunk:
63                         yield new_chunk
```

```
64  
65         pending = pending[end + len(  
CLOSE_TAG):]  
66         in_think = False  
67         flush_start = 0
```

Codice 4.4: Estratto del ciclo di generazione in `_generate`

Questo frammento mostra come il backend gestisca lo streaming dell'output del modello, filtrando esplicitamente eventuali sezioni racchiuse tra tag `<think>` e `</think>`. Tale meccanismo consente di sopprimere il ragionamento interno del modello, esponendo al client solo la parte rilevante della risposta. La logica di gestione del buffer pending e degli indici di flush garantisce che il testo venga inviato progressivamente al client senza includere i tag di controllo.

CAPITOLO 4. METODOLOGIA

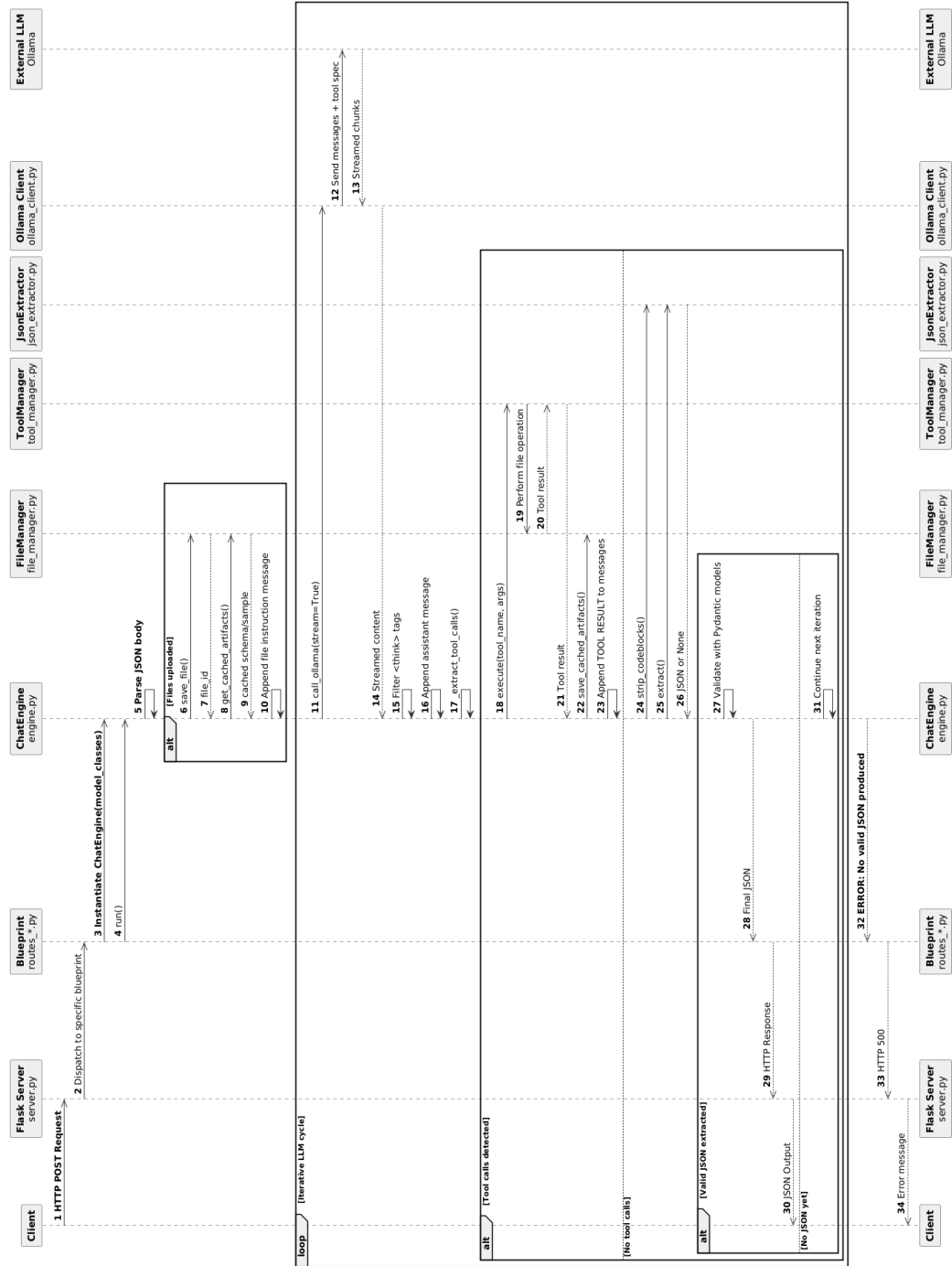


Figura 4.6: Diagramma di sequenza del ciclo di chat orchestrato dal ChatEngine.

4.2.4 Gestione dei file e caching degli artefatti

La gestione dei file caricati dal client è affidata alla classe `FileManager`, che si occupa di salvare i contenuti su disco, risolvere i percorsi, campionare righe da file CSV e inferire lo schema delle colonne. Un aspetto rilevante è l'uso di un identificatore deterministico basato sull'hash SHA256 del contenuto del file, che consente di riutilizzare la cache tra richieste diverse. Il metodo `save_file` ne è un esempio:

```

1 class FileManager:
2     """Handles file operations for tool_files.* and
3     caching."""
4
5     @staticmethod
6     def save_file(name: str, base64_content: str) -> str
7     :
8         """
9         Saves an uploaded file to /tmp and returns a
10        deterministic file_id
11        based on the SHA256 hash of the file content.
12        This ensures cache reuse across requests.
13        """
14        raw = base64.b64decode(base64_content)
15
16        file_hash = hashlib.sha256(raw).hexdigest()
17        file_id = file_hash
18
19        path = f"/tmp/{file_id}_{name}"
20
21        if not os.path.exists(path):
22            with open(path, "wb") as f:
23                f.write(raw)
24        return file_id

```

Codice 4.5: Salvataggio dei file e generazione di `file_id`

La cache persistente è gestita tramite un file JSON memorizzato in `/tmp/file_cache.json`. I metodi `get_cached_artifacts` e `save_cached_artifacts` permettono di memorizzare e recuperare, per ciascun `file_id`, sia lo schema inferito sia un campione di righe. Questo approccio riduce il carico computazionale, evitando di ripetere analisi costose su file già elaborati.

Per l'analisi dei file CSV, il `FileManager` si appoggia alla classe `CSVUtils`, che implementa una semplice ma efficace logica di rilevamento del delimita-

tore tramite `csv.Sniffer`. Il metodo `sample_file` utilizza quindi `pandas` per leggere il file e restituire le prime righe in forma di lista di dizionari, mentre `infer_schema` costruisce una descrizione dello schema delle colonne, includendo tipo di dato, nullabilità, unicità, questo permette di ridurre il carico computazionale sul modello ed ottenere uno speedup importante in termini di generazione, in quanto non è più necessario che il modello legga un numero elevato di righe per conoscere il tipo di dato, la nullabilità e unicità di una colonna.

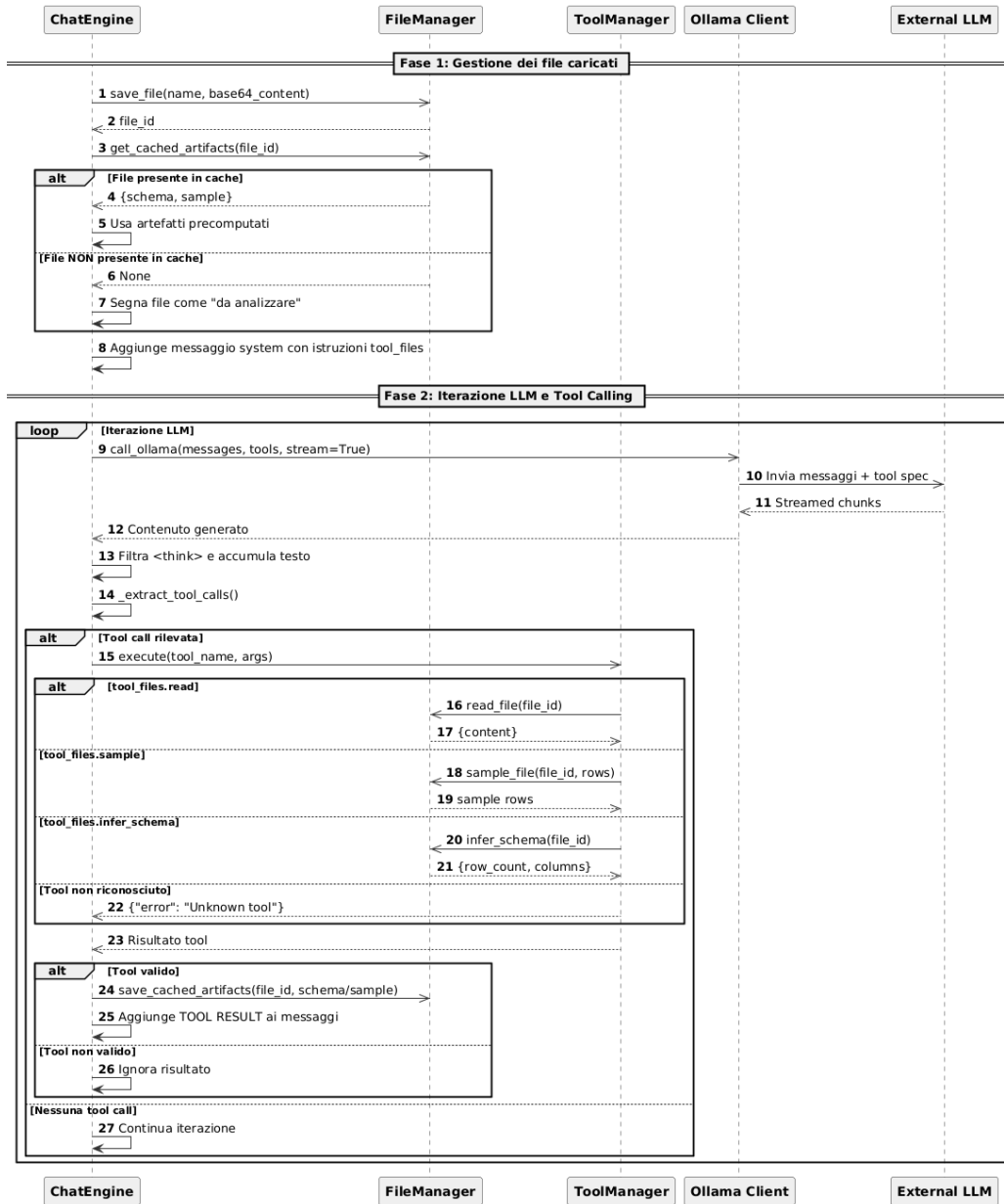


Figura 4.7: Diagramma della gestione dei file e dei tool associati.

4.2.5 Tooling e ToolManager

L'integrazione con strumenti esterni è realizzata tramite un meccanismo di tool calling che consente al modello LLM di richiedere esplicitamente l'esecuzione di funzioni come la lettura di un file, il campionamento di righe o l'inferenza dello schema. Il ChatEngine espone al modello una specifica di tool tramite il metodo `_tool_spec`, che definisce i parametri richiesti per ciascuna funzione.

```
1 @staticmethod
2 def _tool_spec():
3     """Correct Ollama tool specification."""
4     print("[ENGINE] Returning tool spec")
5     return [
6         {
7             "name": "tool_files.read",
8             "description": "Read full file content",
9             "parameters": {
10                 "type": "object",
11                 "properties": {"file_id": {"type": "
string"}}},
12             "required": ["file_id"]
13         },
14         {
15             "name": "tool_files.sample",
16             "description": "Sample rows from CSV",
17             "parameters": {
18                 "type": "object",
19                 "properties": {
20                     "file_id": {"type": "string"},
21                     "rows": {"type": "integer"}
22                 },
23                 "required": ["file_id"]
24             },
25         },
26         {
27             "name": "tool_files.infer_schema",
28             "description": "Infer CSV schema",
29             "parameters": {
30                 "type": "object",
31                 "properties": {"file_id": {"type": "
string"}}
```

```

33     string"}}},
34         "required": ["file_id"]
35     }
36 ]

```

Codice 4.6: Specifiche dei tool esposti al modello

Quando il modello produce nel suo output una struttura JSON che rappresenta una chiamata a tool, il ChatEngine la intercetta tramite il metodo `_extract_tool_calls`, che utilizza espressioni regolari per individuare oggetti JSON contenenti il nome del tool e gli argomenti. Le chiamate così estratte vengono poi inoltrate al ToolManager, che funge da dispatcher.

```

1 class ToolManager:
2     """Executes tool calls requested by the LLM."""
3
4     TOOL_MAP = {
5         "tool_files.read": FileManager.read_file,
6         "tool_files.sample": FileManager.sample_file,
7         "tool_files.infer_schema": FileManager.
infer_schema,
8     }
9
10    @staticmethod
11    def execute(name: str, args: Dict[str, Any]):
12        """
13        Executes a tool by name.
14        """
15        fn = ToolManager.TOOL_MAP.get(name)
16        if not fn:
17            return {"error": f"Unknown tool: {name}"}
18
19        try:
20            return fn(**args)
21        except Exception as e:
22            return {"error": str(e)}

```

Codice 4.7: Esecuzione dei tool tramite ToolManager

Questo meccanismo consente di mantenere disaccoppiata la logica di orchestrazione dalla logica specifica dei tool, facilitando l'estensione futura con nuovi strumenti. Inoltre, il ChatEngine mantiene un insieme di tool già ese-

guiti per evitare chiamate duplicate con gli stessi argomenti, migliorando l'efficienza complessiva del sistema.

4.2.6 Estrazione e validazione del JSON

Una volta che il modello ha prodotto una risposta testuale, il ChatEngine tenta di estrarre un blocco JSON valido che corrisponda a uno dei modelli Pydantic forniti. L'estrazione è delegata alla classe JsonExtractor, che rimuove innanzitutto eventuali code block Markdown e poi scansiona il testo alla ricerca di sequenze di parentesi graffe bilanciate.

```
1 class JsonExtractor:
2     """Extracts and validates JSON blocks from LLM
3     output."""
4
5     @staticmethod
6     def strip_codeblocks(text: str) -> str:
7         """Removes markdown code fences."""
8         text = re.sub(r"```[\s\S]*?```", "", text)
9         text = text.replace("```", "").replace("`", "")
10        return text.strip()
11
12    @staticmethod
13    def extract(text: str):
14        """Extracts the first valid JSON block matching
15        expected schemas."""
16        stack = []
17        start = None
18        candidates = []
19
20        for i, ch in enumerate(text):
21            if ch == '{':
22                if not stack:
23                    start = i
24                    stack.append('{')
25            elif ch == '}':
26                if stack:
27                    stack.pop()
28                if not stack and start is not None:
29                    candidate = text[start:i+1]
30                    candidates.append(candidate)
```

```
30     for c in reversed(candidates):
31         if '"' in c and "'" not in c:
32             continue
33         try:
34             json.loads(c)
35             return c
36         except Exception:
37             continue
38
39     return None
```

Codice 4.8: Estrazione del JSON tramite JsonExtractor

Il ChatEngine, una volta ottenuta una stringa JSON, la passa al metodo `_validate_with_any_model`, che tenta la validazione con ciascuna delle classi Pydantic fornite in ingresso. Il primo modello che riesce a validare il JSON viene considerato quello corretto e il risultato viene serializzato e restituito al client. In caso di fallimento per tutti i modelli, viene sollevata un'eccezione che contiene il dettaglio degli errori di validazione per ciascuna classe.

Questo approccio consente di supportare, con un unico endpoint, più tipologie di risposta strutturata, delegando al contenuto del JSON la scelta implicita del modello più adatto. Dal punto di vista architetturale, ciò rappresenta una forma di polimorfismo guidato dai dati, in cui la struttura del payload determina il tipo di oggetto applicabile.

4.2.7 Integrazione con il client Ollama

La comunicazione con il modello LLM è incapsulata nel modulo `ollama_client.py` che fornisce una funzione `call_ollama` per invocare l'API di chat di Ollama. La funzione costruisce un payload che include il nome del modello, la sequenza di messaggi, l'eventuale specifica dei tool e alcune opzioni di generazione.

```
1 import ollama
2
3 def call_ollama(model_name: str, messages: list, tools:
4     list = None, stream: bool = False):
5     payload = {
6         "model": model_name,
7         "messages": messages,
8         "stream": stream,
9         "keep_alive": "5m",
10        "options": {
11            "temperature": 0.1,
```

```
11         "top_p": 0.9,  
12         "repeat_penalty": 1.1,  
13         "num_ctx": 16384  
14     }  
15 }  
16  
17 if tools is not None:  
18     payload["tools"] = tools  
19  
20 return ollama.chat(**payload)
```

Codice 4.9: Wrapper per la chiamata al modello Ollama

L'uso di un wrapper dedicato consente di isolare i dettagli dell'API esterna dal resto del codice, facilitando eventuali modifiche future alla configurazione del modello o alla libreria utilizzata. Inoltre, la possibilità di abilitare lo streaming permette al ChatEngine di ricevere l'output del modello in modo incrementale, migliorando la reattività percepita dal client.

L'architettura descritta realizza un backend robusto e flessibile per l'interazione con LLM, combinando un'interfaccia HTTP basata su Flask, un motore di orchestrazione capace di gestire cicli di chat complessi, un sistema di tool calling per l'elaborazione di file e una rigorosa validazione delle risposte tramite modelli Pydantic. La separazione delle responsabilità tra ChatEngine, FileManager, ToolManager, JsonExtractor e i modelli di dominio consente di mantenere il codice modulare, estensibile e facilmente manutenibile.

L'uso di identificatori deterministici per i file e di una cache persistente riduce il costo computazionale delle operazioni ripetute, mentre la gestione esplicita delle sezioni di ragionamento interno del modello tramite tag `<think>` permette di controllare con precisione il contenuto esposto al client.

4.3 Gestione Utenti

La gestione degli utenti nel sistema bioCMS costituisce un modulo centrale dell'architettura applicativa. Essa integra componenti frontend e backend in un flusso coerente che comprende autenticazione a due fasi, configurazione iniziale dell'amministratore, creazione di nuovi utenti, gestione granulare dei permessi, modifica dei profili e organizzazione degli utenti in gruppi. L'interazione tra i moduli è mediata da API REST implementate nel gateway UserGateway.php, mentre il frontend coordina le viste tramite il presenter

AppPresenter. Nelle immagini che seguono è possibile vedere il diagramma delle classi e dei casi d'uso necessari per realizzare questa funzionalità.

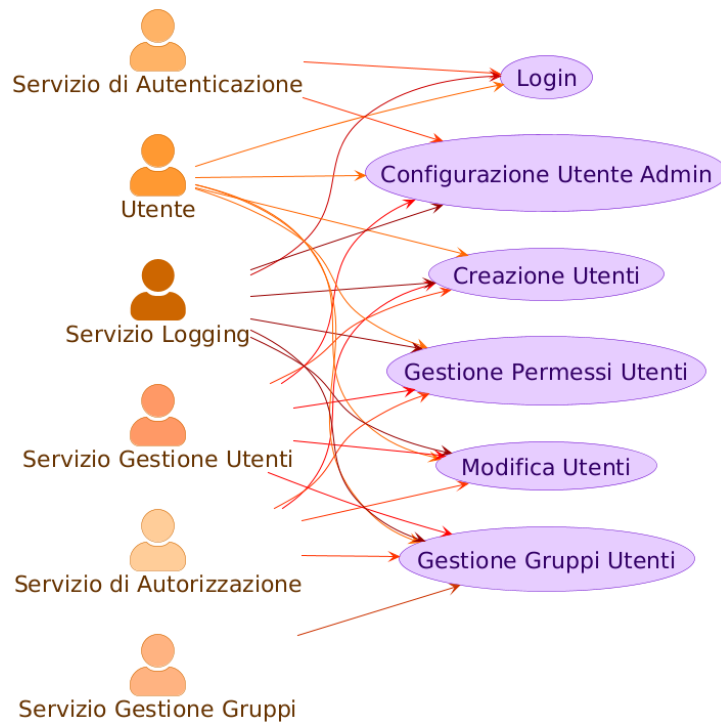


Figura 4.8: Diagramma dei casi d'uso relativi alla gestione utenti.

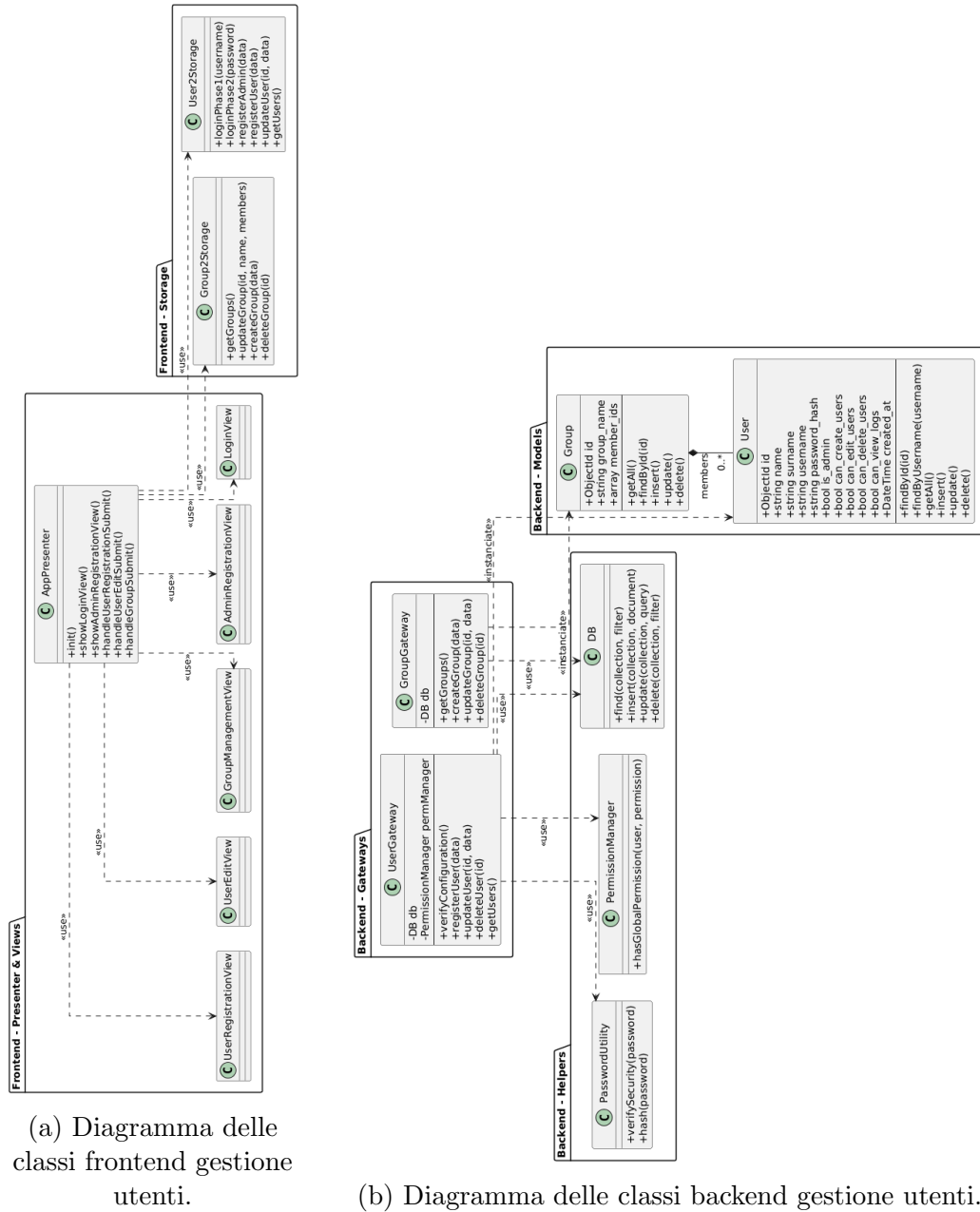


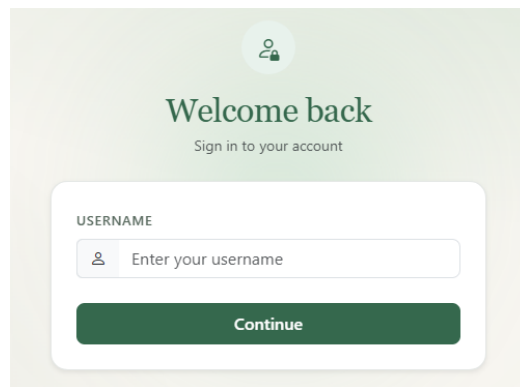
Figura 4.9: Diagramma delle classi gestione utenti.

4.3.1 Login

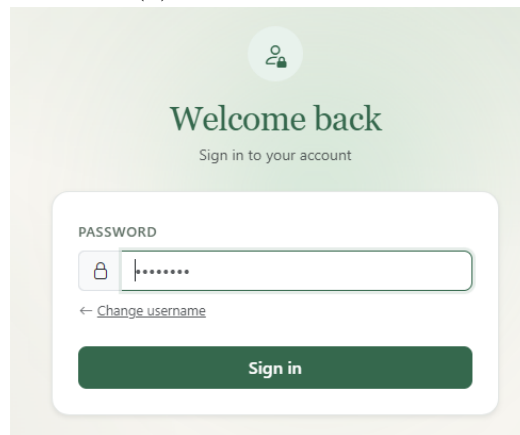
Il processo di autenticazione implementato in bioCMS è strutturato secondo un modello a due fasi, progettato per aumentare la sicurezza e permettere la gestione centralizzata di eventi critici come il reset forzato della password

o l'inattività prolungata dell'utente. Tale architettura consente al backend di controllare in modo granulare lo stato dell'account prima di consentire l'accesso completo al sistema.

La distinzione tra fase 1 e fase 2 non è solo concettuale, ma è codificata direttamente nella logica del gateway `UserGateway.php`, che interpreta la richiesta in base alla presenza o meno del campo `usrn` (username) nel token JWT intermedio. Il frontend, tramite la classe `User2Storage`, invia due richieste distinte, ciascuna con un ruolo specifico nel processo di autenticazione.



(a) View login fase 1.



(b) View login fase 2.

Figura 4.10: View login utente.

Fase 1: Verifica dell'esistenza dell'utente e dello stato dell'account

La prima fase ha l'obiettivo di verificare che l'utente esista e che il suo account sia in uno stato valido per procedere con l'autenticazione.

Il frontend attiva questa fase tramite la chiamata seguente.

```
1 async loginPhase1(username) {
2     const client = new App.GenericApiClient(this.
  endpoint + '/login');
3     return await client.MakeCall(
4         App.HTTPMethod.POST,
5         { username },
6         App.HTTPType.JSON,
7         App.HTTPContentType.FORM_URL_ENCODED
8     );
9 }
```

Codice 4.10: Chiamata alla fase 1 del login

Il backend riceve la richiesta e utilizza il metodo `User::findByUsername()` per verificare la presenza dell'utente nel database. Se l'utente non esiste o risulta eliminato, viene restituito un errore. In caso contrario, il sistema valuta lo stato della password.

La logica di controllo è particolarmente significativa: se la password è nulla, significa che l'amministratore ha forzato un reset; se l'ultimo accesso risale a più di 90 giorni, il sistema invalida automaticamente la password per motivi di sicurezza. In entrambi i casi, il backend risponde con codice 301, che il frontend interpreta come richiesta di reimpostazione della password.

```
1 if ($user->password_hash == null)
2     $responseCode = 301; // Password reset forced by the
  admin
3 else if ((new DateTime())->diff(new DateTime($user->
  last_login))>days > 90) {
4     $user->password_hash = null; // Password reset
  forced due inactivity
5     $user->update();
6     $responseCode = 301;
7 }
```

Codice 4.11: Reset password forzato nel backend

In questa fase il backend genera un token JWT intermedio contenente solo il campo `usrn`, che permette di identificare l'utente nella fase successiva senza ancora concedere accesso al sistema.

Fase 2: Verifica della password e generazione del token JWT completo Una volta superata la fase 1, il frontend mostra il modulo di inserimento password o, in caso di reset forzato, il modulo di reimpostazione. La fase 2 viene attivata tramite:

```
1 async loginPhase2(password) {  
2     const client = new App.GenericApiClient(this.  
3     endpoint + '/login/password');  
4     return await client.MakeCall(  
5         App.HTTPMethod.POST,  
6         { password },  
7         App.HTTPType.JSON,  
8         App.HTTPContentType.FORM_URL_ENCODED  
9     );  
}
```

Codice 4.12: Chiamata alla fase 2 del login

Il backend, riconoscendo la presenza del campo `usn` nel token intermedio, recupera l'utente corrispondente e procede alla verifica della password tramite il metodo `User::login()`.

```
1 if (!password_verify($password, $this->password_hash))  
2     return false;  
3  
4 $this->last_login = date("Y-m-d H:i:s");  
5 $this->db->update("users", [  
6     "filter" => ["_id" => new ObjectId($this->id)],  
7     "update" => ['$set' => ["last_login" => $this->  
8     last_login]]  
9 ]);
```

Codice 4.13: Verifica password e aggiornamento timestamp

Se la verifica ha successo, il backend genera un token JWT completo contenente il campo `uid`, che identifica univocamente l'utente autenticato. Questo token viene memorizzato dal frontend tramite `App.tokenManager` e abilita l'accesso alle funzionalità protette del sistema.

CAPITOLO 4. METODOLOGIA

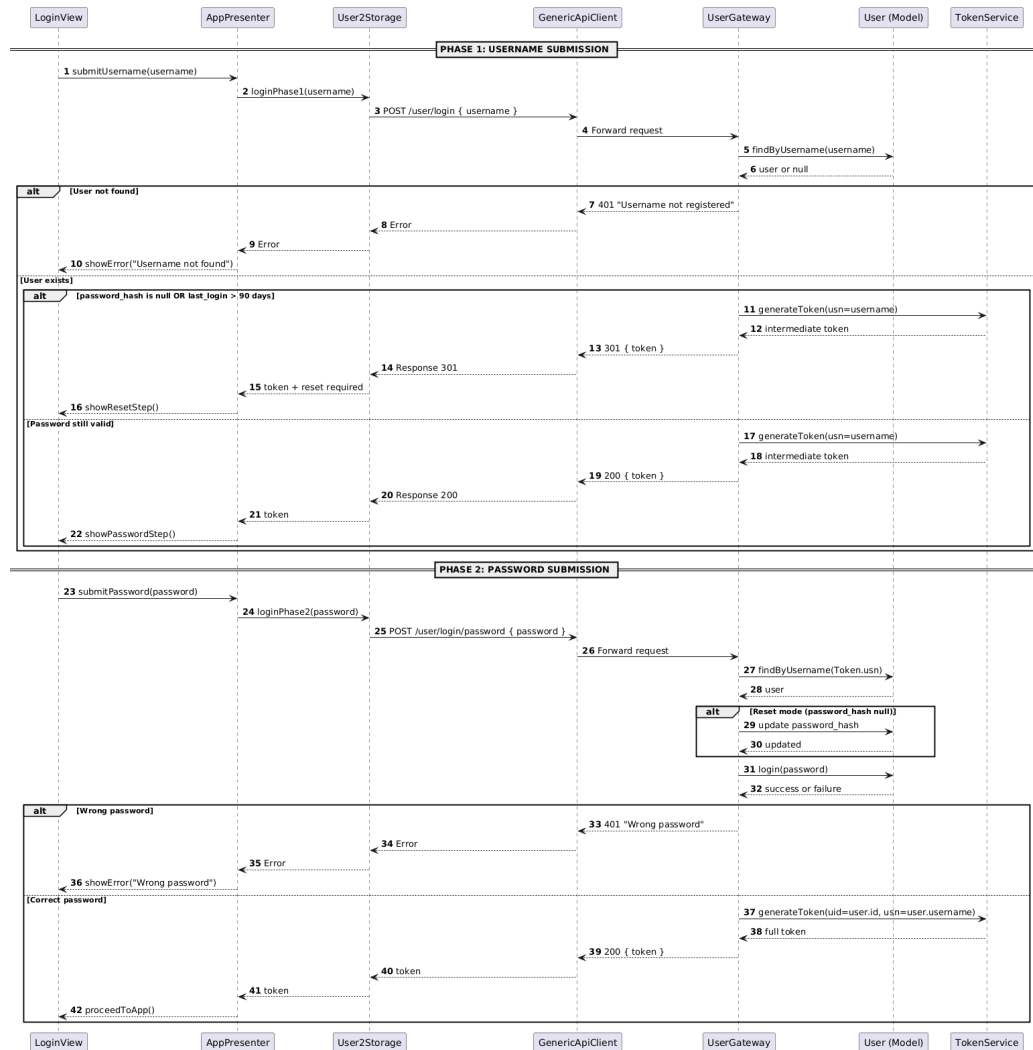


Figura 4.11: Diagramma di sequenza del processo di login.

4.3.2 Configurazione Utente Admin

La configurazione dell'utente amministratore costituisce il punto di avvio dell'intero sistema bioCMS. Poiché molte funzionalità critiche — come la gestione degli utenti, dei gruppi e dei permessi — richiedono un controllo centralizzato, l'architettura prevede che almeno un amministratore sia presente prima che il sistema possa essere utilizzato in modo ordinario. Questa scelta progettuale impedisce che la piattaforma venga eseguita in uno stato privo di supervisione, condizione che comprometterebbe la sicurezza e la governabilità dell'applicazione.

Figura 4.12: View configurazione utente admin.

Il backend effettua questa verifica attraverso il metodo `verifyConfiguration()`, che interroga il database recuperando tutti gli utenti non eliminati e calcolando quanti di essi possiedono il flag `is_admin`. Il risultato viene restituito al frontend sotto forma di valore booleano, che indica se il sistema è già stato inizializzato oppure no. Il codice seguente mostra la logica utilizzata.

```

1 $users = (new User($this->db))->getAll();
2 $adminCount = count(array_filter($users, fn($u) => $u->
   is_admin));
3 ResponseHelper::success(["configured" => $adminCount >=
   1], 200, $this->logManager);

```

Codice 4.14: Verifica configurazione iniziale

Il presenter interpreta questa informazione nella fase di avvio dell'applicazione. Se il backend segnala che non esiste alcun amministratore, il sistema non permette l'accesso al normale flusso di login e indirizza l'utente verso la vista di registrazione dell'amministratore. In caso contrario, viene mostrata direttamente la schermata di autenticazione. La logica è sintetizzata nel seguente frammento.

```

1 if (response.Content.configured === false)
2     this.__showAdminRegistrationView();
3 else
4     this.__showLoginView();

```

Codice 4.15: Logica di inizializzazione nel presenter

La procedura di creazione dell'amministratore utilizza il metodo `registerAdmin()` del layer di storage frontend, che invia al backend i dati raccolti dalla vista dedicata. Il gateway elabora la richiesta distinguendo automaticamente il caso di prima configurazione da quello di creazione di utenti successivi. Quando il sistema è al primo avvio, l'utente creato viene marcato come amministratore senza possibilità di modificare tale comportamento, poiché l'assenza di un amministratore renderebbe il sistema non governabile.

Durante questa fase, il backend applica controlli rigorosi sui dati forniti. La password deve essere presente, deve coincidere con la conferma e deve rispettare i criteri di sicurezza definiti da `PasswordUtility::verifySecurity()`. Lo username deve essere univoco e non deve esistere alcun tentativo di manipolare i permessi, poiché l'assegnazione del ruolo amministrativo è gestita esclusivamente dal sistema. Solo dopo aver superato tutte le verifiche, l'utente viene inserito nel database tramite il metodo `User::insert()`, che genera un nuovo identificativo, registra il timestamp di creazione e salva il documento nella collezione `users`.

L'intero flusso può essere rappresentato tramite un diagramma UML che evidenzia la sequenza di interazioni tra `AppPresenter`, `User2Storage`, `UserGateway`, `User` e `AdminRegistrationView`. Tale rappresentazione permette di visualizzare con chiarezza la transizione dallo stato "non configurato" allo stato operativo del sistema, mostrando come la presenza dell'amministratore costituisca la condizione necessaria per l'avvio del normale ciclo di vita dell'applicazione.

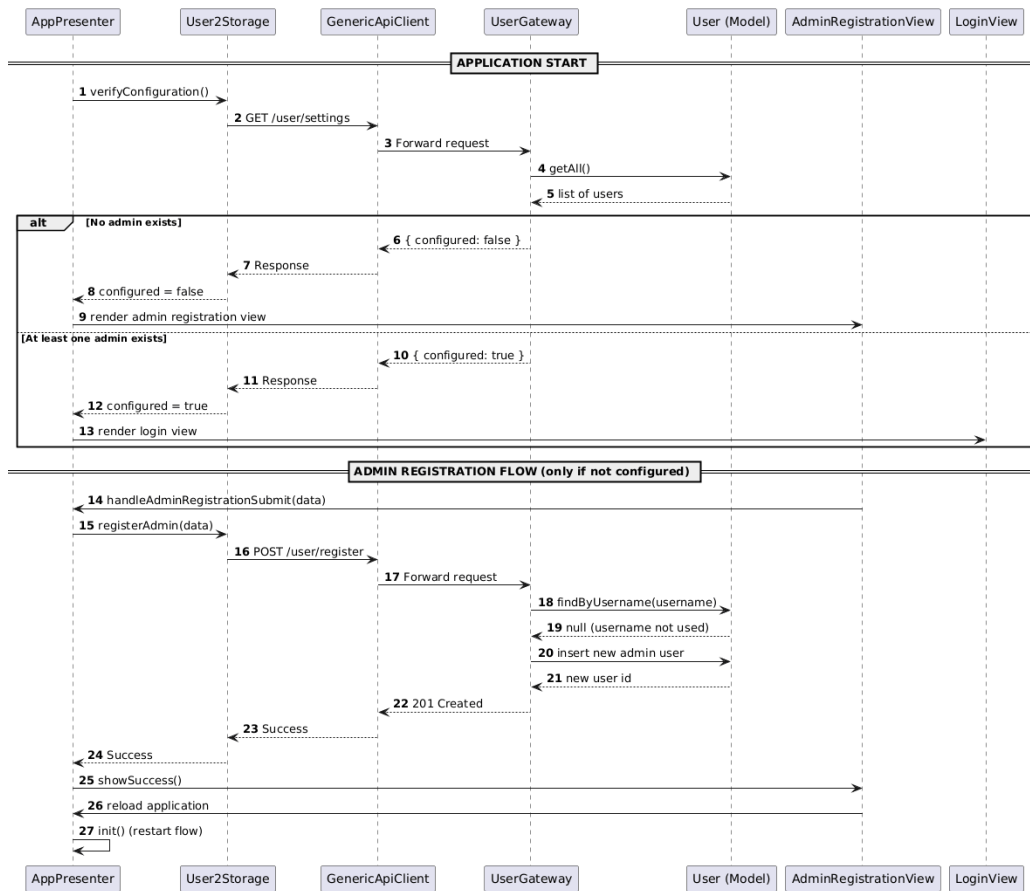


Figura 4.13: Diagramma di sequenza della configurazione utente admin.

4.3.3 Creazione Utenti

La creazione di utenti successivi all'amministratore rappresenta il passaggio con cui il sistema bioCMS estende la propria base di utilizzatori, mantenendo al tempo stesso un controllo rigoroso sui permessi e sulla coerenza dei dati. L'intero flusso è orchestrato dal presenter, che coordina la vista `UserRegistrationView` e il layer di comunicazione `User2Storage`, mentre il backend gestisce la validazione e la persistenza attraverso il metodo `registerUser()` del `UserGateway`.

← Back



New User

Create a new account

FIRST NAME LAST NAME

John Doe

USERNAME

 username

PASSWORD



CONFIRM PASSWORD



Create account

Figura 4.14: View creazione utente.

Il processo inizia con la compilazione del modulo di registrazione da parte dell'utente autorizzato. La vista raccoglie i dati necessari — nome, cognome, username, password e conferma della password — e li restituisce al presenter, che esegue una prima validazione lato client. Questa validazione ha lo scopo di intercettare errori immediati, come campi mancanti o password non coincidenti, evitando chiamate superflue al backend. Una volta verificata la correttezza dei dati, il presenter invia la richiesta al server tramite il metodo `registerUser()` del layer di storage:

```
1 const response = await UserStorage.registerUser({  
2   ...data,  
3   display_name: data.username  
4 });
```

Codice 4.16: Invio dati per la creazione utente

Il backend riceve la richiesta e applica una serie di controlli che garantiscono la sicurezza e la coerenza del sistema. La prima verifica riguarda i permessi dell'utente che sta tentando di creare un nuovo account. Se il sistema è già configurato e l'utente autenticato non possiede il permesso globale `can_create_users`, la creazione viene immediatamente bloccata. Questa logica impedisce che utenti privi di autorizzazione possano ampliare gli account disponibili, preservando l'integrità del modello di sicurezza. Il controllo è implementato nel gateway come segue.


```

1 if ($adminCount > 0 && !$this->permManager->
    hasGlobalPermission($currentUser, 'can_create_users')
    )
2     ResponseHelper::error("Operation not Allowed.", 401,
        $this->logManager);

```

Codice 4.17: Controllo permessi nella creazione utenti

Superata la verifica dei permessi, il backend procede con la validazione dei dati. Viene controllata la presenza di tutti i campi obbligatori, la corrispondenza tra password e conferma, la robustezza della password tramite `PasswordUtility::verifySecurity()`, e l'unicità dello username attraverso una ricerca nel database. Ogni violazione genera una risposta di errore immediata, evitando la creazione di account inconsistenti o potenzialmente vulnerabili. Di seguito l'algoritmo di verifica di sicurezza della password.

Algoritmo 4 Implementazione Verifica Sicurezza Password.

```

1 public static function verifySecurity(string $password):
    array
2 {
3     $errors = [];
4     // Minimum length
5     if (strlen($password) < 12)
6         $errors[] = "Password must be at least 12
            characters long.";
7
8     // Uppercase
9     if (!preg_match('/[A-Z]/', $password))
10         $errors[] = "Password must contain at least one
            uppercase letter.";
11
12     // Lowercase
13     if (!preg_match('/[a-z]/', $password))
14         $errors[] = "Password must contain at least one
            lowercase letter.";
15
16     // Digits
17     if (!preg_match('/[0-9]/', $password))
18         $errors[] = "Password must contain at least one
            digit.";
19

```

```
20     // Symbols
21     if (!preg_match('/[\W_]/', $password))
22         $errors[] = "Password must contain at least one
special character.";
23
24     // Blacklist of weak passwords
25     $common = [
26         'password', '123456', 'qwerty', 'admin', '
welcome',
27         'abc123'
28     ];
29     if (in_array(strtolower($password), $common, true))
30         $errors[] = "Password is too common and easily
guessable.";
31     return [
32         'valid' => empty($errors),
33         'errors' => $errors
34     ];
35 }
```

Una volta completate le verifiche, il backend costruisce un nuovo oggetto User, assegna i valori ricevuti e imposta i permessi iniziali. Nel caso di utenti non amministratori, i permessi vengono assegnati in modo esplicito e non possono essere elevati arbitrariamente tramite la richiesta, poiché il gateway ignora eventuali tentativi di manipolazione dei campi relativi ai privilegi. La persistenza dell'utente avviene tramite il metodo User::insert(), che genera un nuovo identificativo ObjectId, registra il timestamp di creazione e inserisce il documento nella collezione users. Questo passaggio conclude la procedura di registrazione, restituendo al frontend una risposta di successo che consente alla vista di mostrare la conferma all'utente.

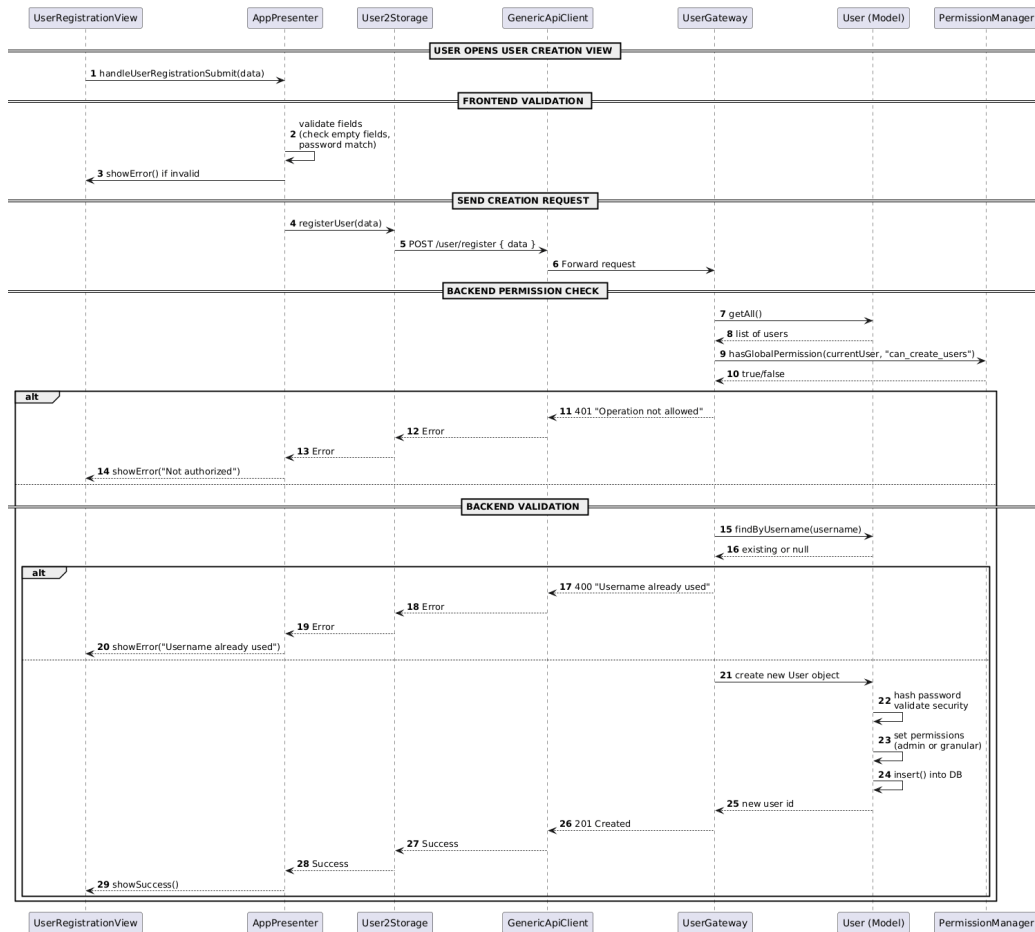


Figura 4.15: Diagramma di sequenza della creazione utenti.

4.3.4 Gestione Permessi Utente

La gestione dei permessi utente costituisce uno degli elementi centrali del modello di sicurezza di bioCMS. L'architettura prevede un insieme di privilegi granulari, rappresentati da campi booleani associati a ciascun utente, che determinano la possibilità di creare nuovi account, modificarli, eliminarli o accedere ai log di sistema. Questa struttura consente di modellare ruoli differenziati senza ricorrere a un sistema di ruoli complesso, mantenendo al tempo stesso un controllo preciso sulle operazioni sensibili.

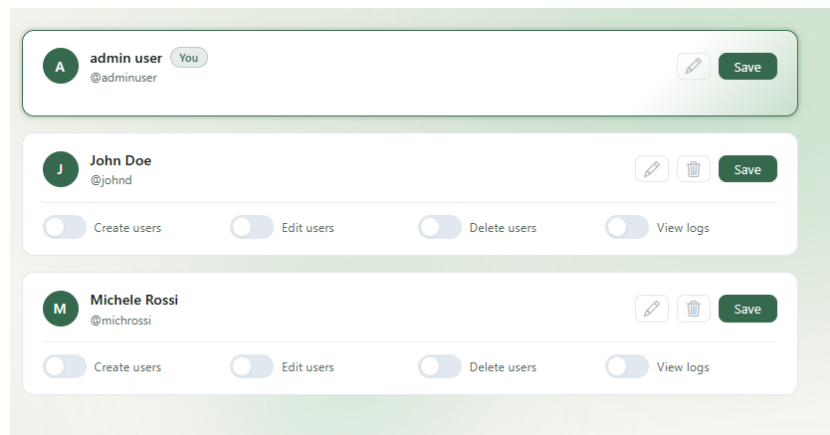


Figura 4.16: View modifica permessi utente.

Il backend svolge un ruolo determinante nel garantire che tali permessi non possano essere manipolati in modo improprio. Ogni richiesta di modifica dei permessi viene infatti sottoposta a un controllo rigoroso, che impedisce a utenti privi di privilegi amministrativi di alterare il proprio livello di autorizzazione o quello di altri utenti. La logica è implementata nel metodo `updateUser()` del `UserGateway`, dove il gateway verifica se la richiesta contiene campi relativi ai permessi e, in tal caso, controlla che l'utente autenticato sia effettivamente un amministratore. In assenza di tale requisito, la modifica viene immediatamente bloccata, come mostrato nel seguente frammento.

```

1 if ((isset($data["can_create_users"]) || isset($data["
2   can_edit_users"]) ||
3     isset($data["can_delete_users"]) || isset($data[
4   "can_view_logs"]))
5   && !$currentUser->is_admin)
6 {
7     ResponseHelper::error("Operation not Allowed.", 401,
8   $this->logManager);
9 }

```

Codice 4.18: Controllo permessi nella modifica.

Questa logica impedisce che un utente possa elevare i propri privilegi tramite una semplice richiesta HTTP, garantendo che l'assegnazione dei permessi rimanga un'operazione strettamente riservata agli amministratori. Inoltre, il gateway applica ulteriori controlli quando l'utente oggetto della modifica è esso stesso un amministratore, impedendo che un utente non amministratore possa alterarne il profilo. In questo modo, il sistema preserva la gerarchia dei

privilegi e impedisce che un amministratore venga degradato o modificato da utenti con privilegi inferiori.

Sul lato frontend, il presenter e le viste riflettono dinamicamente le capacità dell'utente autenticato, evitando di mostrare pulsanti o azioni che non sarebbero comunque autorizzate dal backend. Questa coerenza tra interfaccia e logica server riduce il rischio di errori e migliora l'esperienza d'uso, poiché l'utente visualizza esclusivamente le operazioni che è effettivamente autorizzato a compiere. Il presenter calcola tali capacità combinando il ruolo amministrativo con i permessi granulari, come illustrato nel seguente frammento.

```
1 const canDelete = this.currentUser?.is_admin || this.  
  currentUser?.can_delete_users;  
2 const canEditInfo = this.currentUser?.is_admin || this.  
  currentUser?.can_edit_users;
```

Codice 4.19: Calcolo permessi lato client.

Questa logica consente alla `UserManagementView` di adattare la propria interfaccia in modo coerente con il profilo dell'utente, mostrando ad esempio i pulsanti di eliminazione o modifica solo quando l'utente possiede i permessi necessari. Il frontend non sostituisce i controlli del backend, ma li anticipa, offrendo un'interazione più chiara e riducendo la possibilità di tentativi di operazioni non autorizzate.

Nel complesso, la gestione dei permessi in `bioCMS` si basa su una collaborazione stretta tra frontend e backend: il primo si occupa di presentare un'interfaccia coerente con le capacità dell'utente, mentre il secondo garantisce che nessuna operazione possa violare le regole di sicurezza stabilite dal sistema. Questa duplice protezione assicura che i permessi rimangano affidabili e che la struttura gerarchica degli utenti non possa essere compromessa da modifiche arbitrarie o non autorizzate. Inoltre il centro del controllo dei permessi viene effettuato dal metodo `hasGlobalPermission` che verifica i permessi posseduti da un utente.

```
1 if ($user->is_admin) return true;  
2 return match ($permission) {  
3     'can_edit_users' => $user->can_edit_users ,  
4     'can_delete_users' => $user->can_delete_users ,  
5     'can_create_users' => $user->can_create_users ,  
6     'can_view_logs' => $user->can_view_logs ,  
7     default => false ,  
8 };
```

Codice 4.20: Controllo permessi globali utente.

CAPITOLO 4. METODOLOGIA

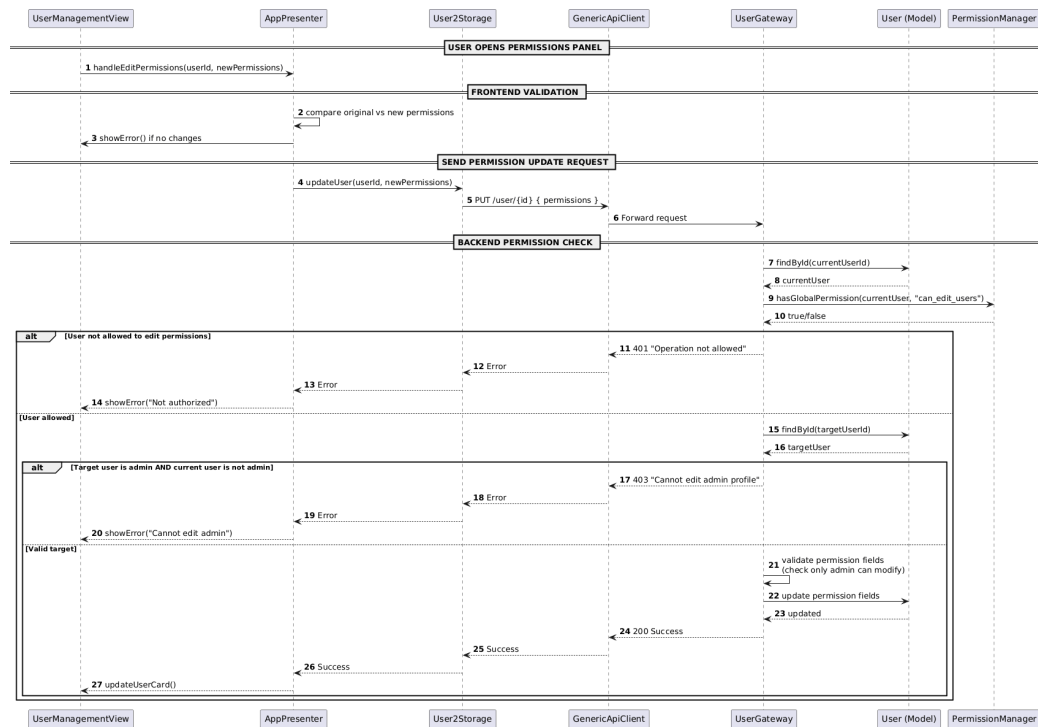


Figura 4.17: Diagramma di sequenza della gestione permessi utente.

4.3.5 Modifica Utenti

La modifica dei profili utente rappresenta una delle operazioni critiche, poiché coinvolge sia la coerenza dei dati anagrafici sia la sicurezza del sistema. L'architettura di bioCMS affronta questo processo attraverso una collaborazione strutturata tra frontend e backend, in cui ciascun livello svolge un ruolo specifico nel garantire che le modifiche siano applicate in modo corretto, sicuro e privo di effetti collaterali indesiderati.

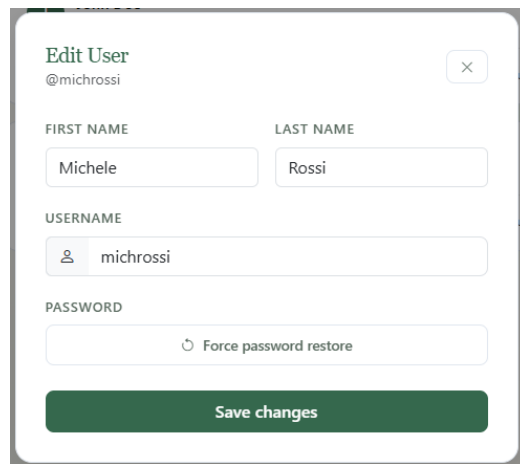


Figura 4.18: View modifica utente.

Il frontend, coordinato dal AppPresenter, adotta una strategia ottimizzata che consiste nell’inviare al backend esclusivamente i campi effettivamente modificati dall’utente. Questa scelta riduce il traffico di rete, evita aggiornamenti superflui e permette al backend di distinguere con precisione quali attributi devono essere aggiornati. Il presenter confronta i valori inseriti nel modulo con quelli presenti nel profilo originale e costruisce un payload minimale, come mostrato nel seguente frammento.

```
1 if (data.name !== original.name) payload.name = data.name;  
2 if (data.surname !== original.surname) payload.surname = data.surname;  
3 if (data.username !== original.username) payload.username = data.username;
```

Codice 4.21: Costruzione payload di modifica.

Una volta costruito il payload, il presenter invia la richiesta al backend tramite il metodo `updateUser()` del layer di storage. Il backend riceve i dati e applica una serie di controlli che garantiscono la validità e la coerenza delle modifiche richieste. Tra questi, uno dei più importanti riguarda l’unicità dello username. Poiché lo username costituisce un identificatore logico dell’utente, il sistema impedisce che due utenti possano dividerlo. Per questo motivo, ogni volta che viene richiesto un aggiornamento dello username, il gateway esegue una ricerca nel database per verificare che il nuovo valore non sia già utilizzato da un altro utente. La logica è implementata come segue.

```
1 $tmpuser = (new User($this->db))->findByUsername($data["  
    username"]);  
2 if ($tmpuser !== null)  
3     ResponseHelper::error("Username already used.", 400,  
        $this->logManager);
```

Codice 4.22: Verifica unicità username

Questo controllo impedisce collisioni tra account e preserva l'integrità del sistema. Oltre alla verifica dell'unicità, il backend applica ulteriori controlli relativi ai permessi dell'utente autenticato, impedendo che un utente privo di privilegi possa modificare il profilo di altri utenti o intervenire su account amministrativi. La logica di protezione è particolarmente rigorosa quando il profilo oggetto della modifica appartiene a un amministratore, poiché il sistema impedisce che un utente con privilegi inferiori possa alterarne i dati.

Una volta superate tutte le verifiche, il backend aggiorna il documento corrispondente nella collezione users tramite il metodo `User::update()`, che applica le modifiche in modo atomico e sicuro. Il frontend, dopo aver ricevuto la conferma dell'avvenuto aggiornamento, aggiorna la rappresentazione grafica del profilo tramite la `UserManagementView`, garantendo una sincronizzazione immediata tra stato applicativo e stato persistente.

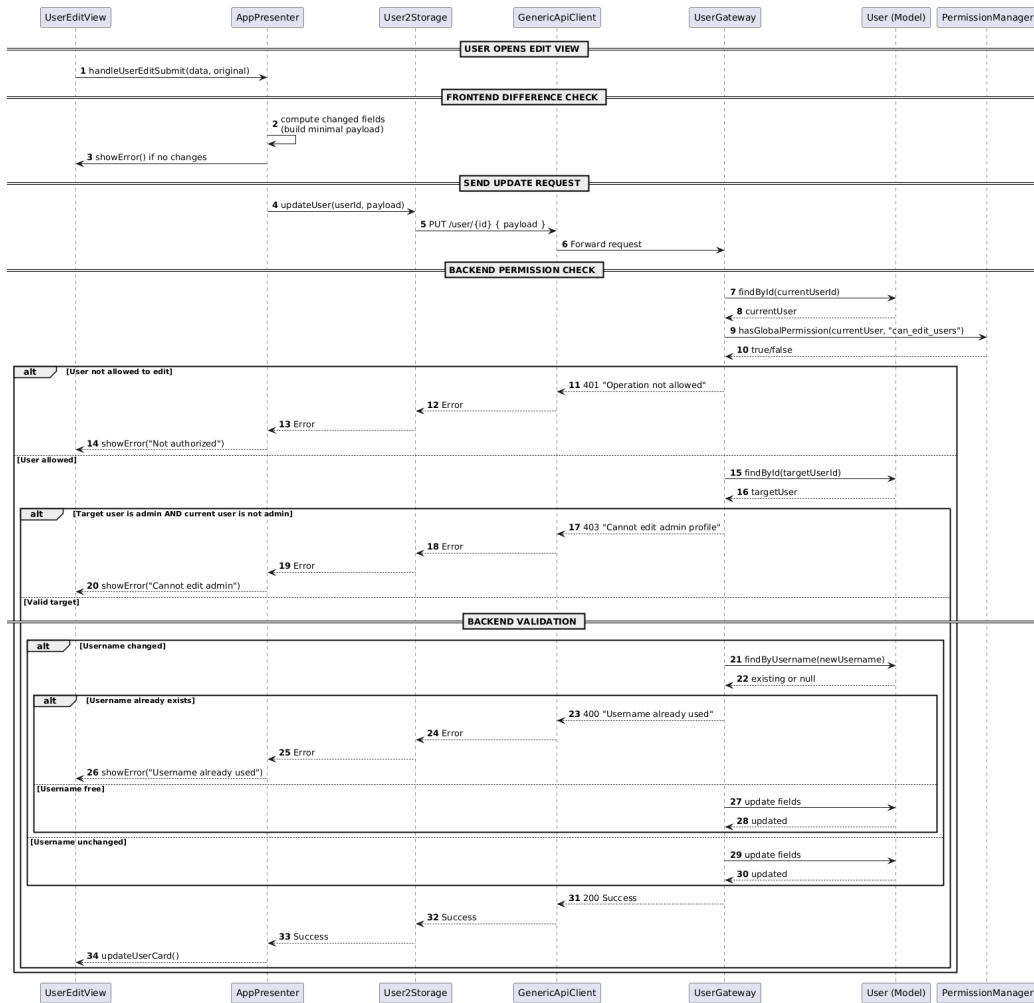
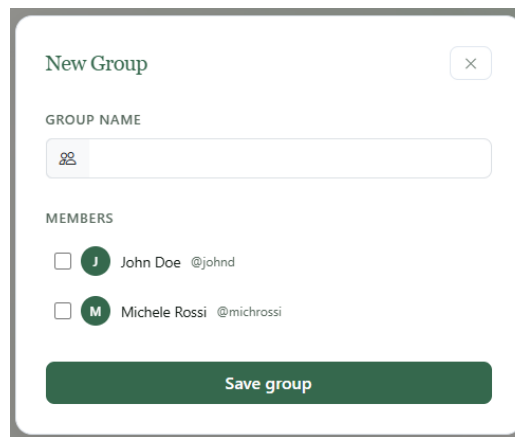


Figura 4.19: Diagramma di sequenza della modifica utenti.

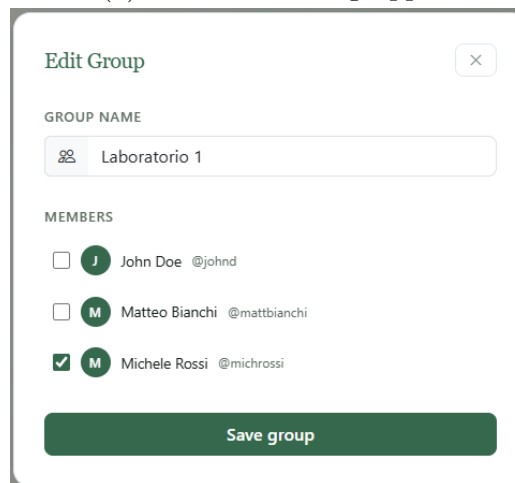
4.3.6 Gestione Gruppi Utenti

La gestione dei gruppi utenti costituisce un'estensione naturale del modello di autorizzazione di bioCMS. Mentre i permessi globali definiscono ciò che un singolo utente può fare all'interno del sistema, i gruppi permettono di organizzare gli utenti in insiemi logici, facilitando la gestione di risorse condivise o la definizione di politiche di accesso più articolate. L'architettura prevede una chiara separazione tra la logica di presentazione, la comunicazione con il backend e la persistenza dei dati, garantendo un flusso coerente e facilmente estendibile.



The 'New Group' form is a modal window with a title bar containing 'New Group' and a close button (X). It features a 'GROUP NAME' section with a text input field containing a group icon and a placeholder. Below this is a 'MEMBERS' section with two entries: 'John Doe @johnd' and 'Michele Rossi @michrossi', each with an unchecked checkbox. At the bottom is a green 'Save group' button.

(a) View creazione gruppi.



The 'Edit Group' form is a modal window with a title bar containing 'Edit Group' and a close button (X). It features a 'GROUP NAME' section with a text input field containing a group icon and the text 'Laboratorio 1'. Below this is a 'MEMBERS' section with three entries: 'John Doe @johnd' (unchecked), 'Matteo Bianchi @mattbianchi' (unchecked), and 'Michele Rossi @michrossi' (checked). At the bottom is a green 'Save group' button.

(b) View modifica gruppi.

Figura 4.20: View gestione gruppi.

Il modello di dominio è rappresentato dalla classe `Group.php`, che incapsula le informazioni fondamentali di un gruppo: un identificativo univoco, un nome descrittivo e l'elenco degli utenti che ne fanno parte. La classe fornisce metodi per la creazione, l'aggiornamento, la cancellazione e il caricamento dei gruppi, delegando al layer di persistenza l'interazione con il database. La struttura è volutamente semplice, poiché la complessità della gestione risiede principalmente nel coordinamento tra frontend e backend.

Il presenter svolge un ruolo centrale nel coordinamento delle operazioni. Quando l'utente accede alla sezione dedicata ai gruppi, il presenter avvia il caricamento simultaneo dei gruppi e degli utenti, utilizzando le classi `Group2Storage` e `User2Storage`. Questa scelta consente di ottenere in un'unica fase tutte le informazioni necessarie per popolare l'interfaccia, evitando

chiamate sequenziali e migliorando la reattività del sistema. Il caricamento avviene tramite una richiesta asincrona combinata, come mostrato nel seguente frammento.

```

1 const [groupRes, userRes] = await Promise.all([
2   GroupStorage.getGroups(),
3   UserStorage.getUsers()
4 ]);

```

Codice 4.23: Caricamento gruppi e utenti

Una volta ottenuti i dati, il presenter li passa alla GroupManagement-View, che si occupa di renderizzare l'elenco dei gruppi e di mostrare l'overlay di modifica o creazione quando richiesto. L'utente può quindi intervenire sul nome del gruppo o sulla composizione dei membri, selezionando gli utenti da includere o rimuovere. Il presenter raccoglie i dati modificati e li invia al backend tramite i metodi `createGroup()` o `updateGroup()`, a seconda che si tratti di un nuovo gruppo o di un aggiornamento. La chiamata di aggiornamento è esemplificata dal seguente frammento.

```

1 await GroupStorage.updateGroup(data.id, data.group_name,
   data.member_ids);

```

Codice 4.24: Aggiornamento gruppo

Il backend riceve la richiesta e la elabora attraverso il gateway dedicato ai gruppi. La persistenza è gestita dal modello Group, che aggiorna il documento corrispondente nella collezione groups. L'operazione avviene in modo atomico, sostituendo il nome del gruppo e l'elenco dei membri con i nuovi valori forniti dal frontend. Il codice seguente mostra la logica di aggiornamento.

```

1 $this->db->update("groups", [
2   "filter" => ["_id" => new ObjectId($this->id)],
3   "update" => ['$set' => [
4     "group_name" => $this->group_name,
5     "member_ids" => $this->member_ids
6   ]
7 ]);

```

Codice 4.25: Persistenza gruppo

Il flusso di gestione dei gruppi si conclude con l'aggiornamento dell'interfaccia, che riflette immediatamente le modifiche apportate. La vista aggiorna la card del gruppo modificato o aggiunge un nuovo elemento all'elenco, garantendo una sincronizzazione costante tra stato applicativo e stato persistente. Questo approccio, basato su una chiara separazione delle responsabilità,

consente di mantenere un codice leggibile e facilmente estendibile, oltre a garantire che ogni modifica sia applicata in modo sicuro e coerente.

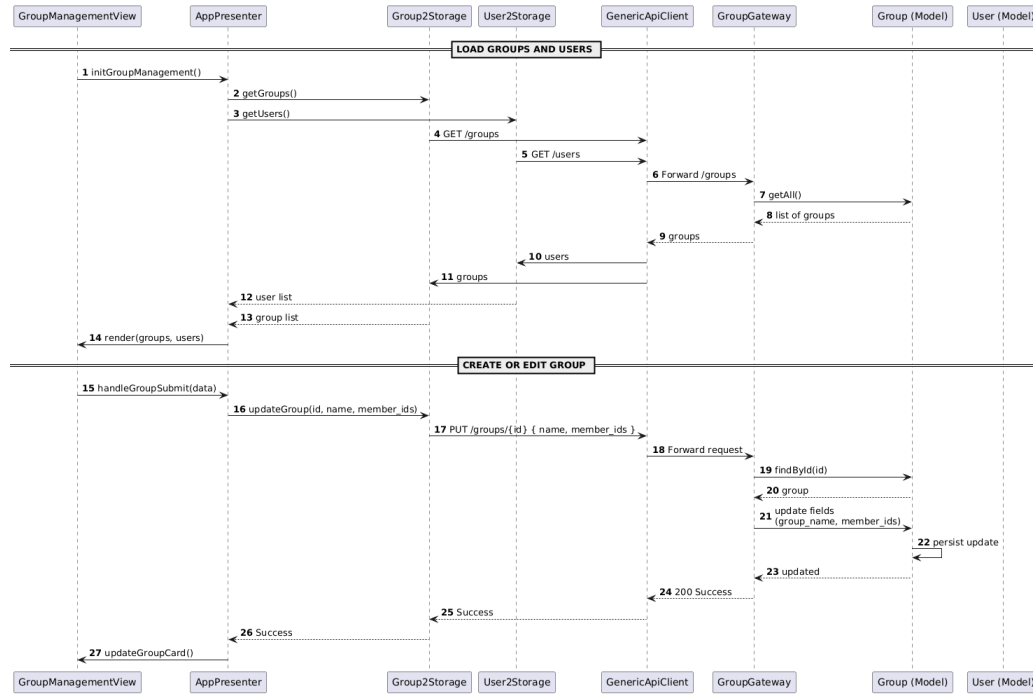


Figura 4.21: Diagramma di sequenza della gestione gruppi.

4.4 Gestione Categorie

La gestione delle categorie costituisce un elemento fondamentale dell'architettura applicativa, poiché consente di classificare e strutturare le transazioni in modo coerente e semanticamente significativo. L'intero flusso operativo coinvolge componenti backend e frontend che cooperano per garantire la creazione, la modifica, l'associazione e la persistenza delle categorie nel database MongoDB. La logica principale risiede nella classe CategoryGateway, che espone gli endpoint REST dedicati, mentre la classe Category modella il documento MongoDB e ne gestisce la persistenza. Il frontend, attraverso viste dedicate, permette all'utente amministratore di interagire con tali funzionalità in modo intuitivo e immediato.

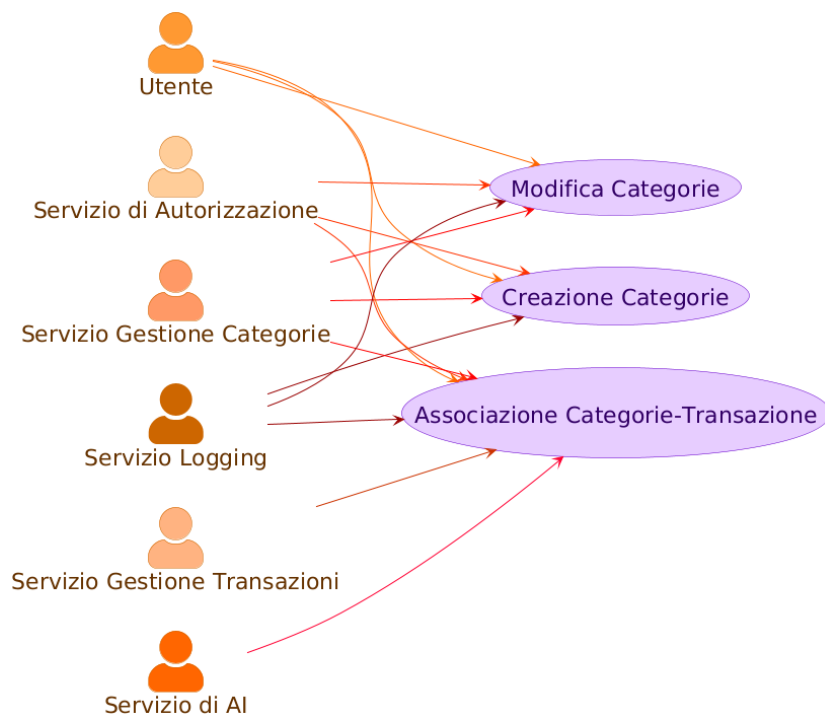


Figura 4.22: Diagramma dei casi d'uso relativi alla gestione categorie.

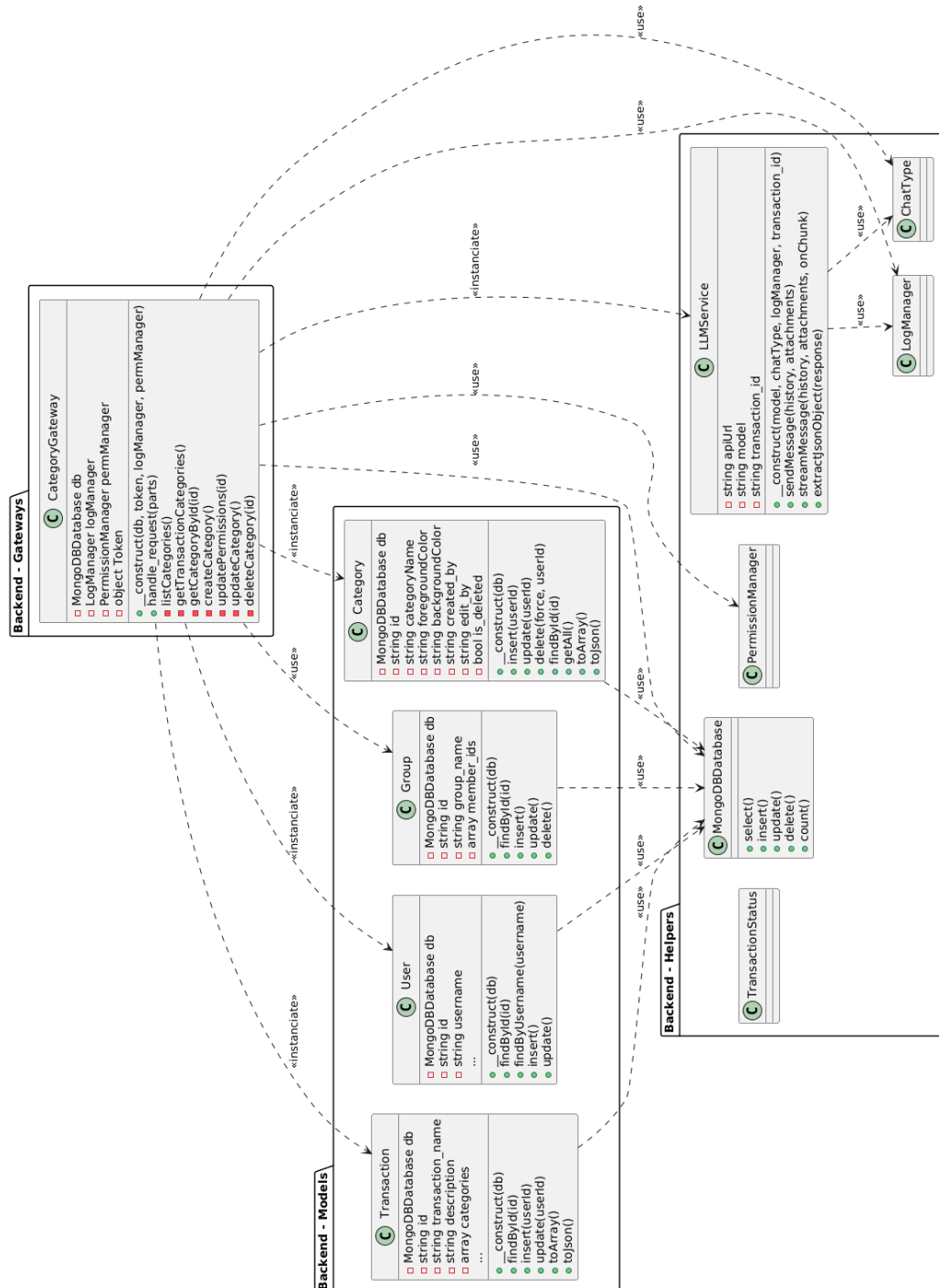


Figura 4.23: Diagramma delle classi backend gestione categorie.

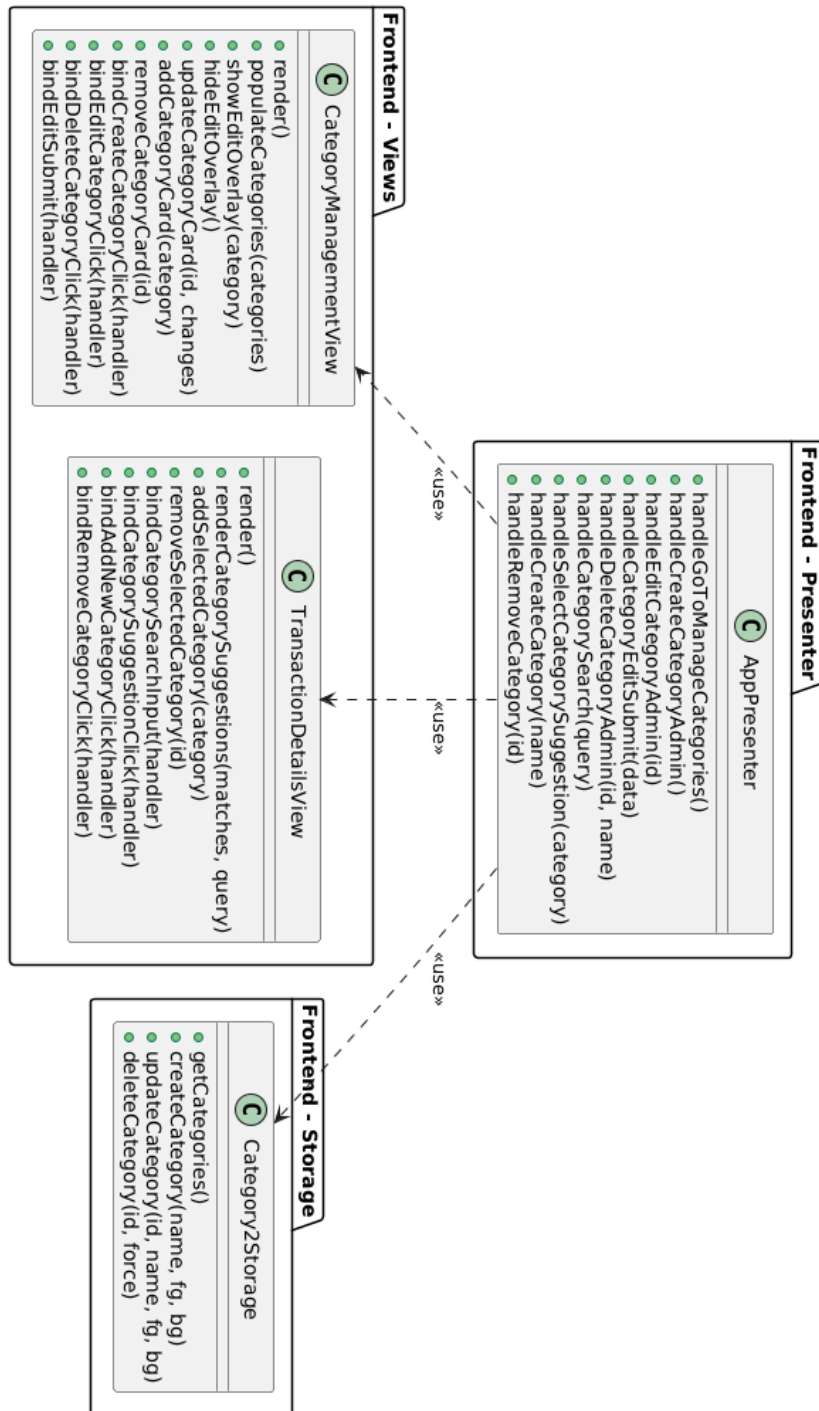


Figura 4.24: Diagramma delle classi frontend gestione categorie.

4.4.1 Creazione Categorie

La creazione di una nuova categoria avviene mediante una richiesta POST all'endpoint `/category`. Il gateway interpreta il payload JSON, verifica la presenza del nome della categoria e costruisce un nuovo oggetto `Category`. L'operazione di persistenza è affidata al metodo `insert()`, che genera un nuovo identificativo `ObjectId`, assegna i metadati relativi all'utente e inserisce il documento nella collezione `categories`. Il seguente estratto mostra la logica essenziale.

```
1 $category = new Category($this->db);  
2 $category->categoryName = $data["categoryName"];  
3 $category->foregroundColor = $data["foregroundColor"] ??  
   "#ffffff";  
4 $category->backgroundColor = $data["backgroundColor"] ??  
   "#000000";  
5 $category->insert($this->Token->uid);
```

Codice 4.26: Controllo permessi globali utente

Sul lato frontend, la vista `CategoryManagementView` presenta un'interfaccia che consente all'amministratore di inserire i dati della nuova categoria e inviarli al backend tramite la classe `Category2Storage`. Una volta ricevuta la conferma, la vista aggiorna dinamicamente la lista delle categorie, garantendo una rappresentazione immediata dello stato aggiornato.

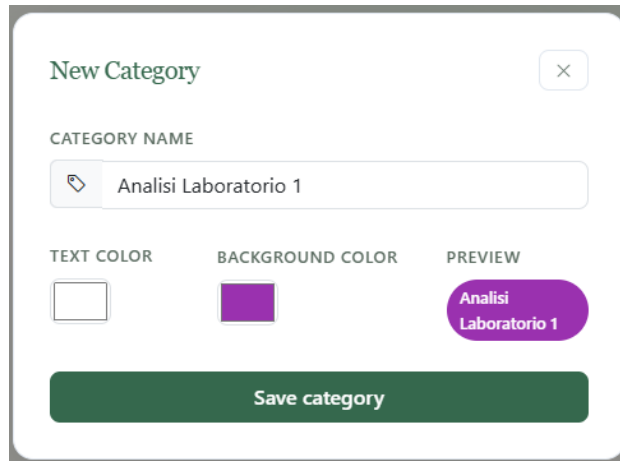


Figura 4.25: View creazione categorie.

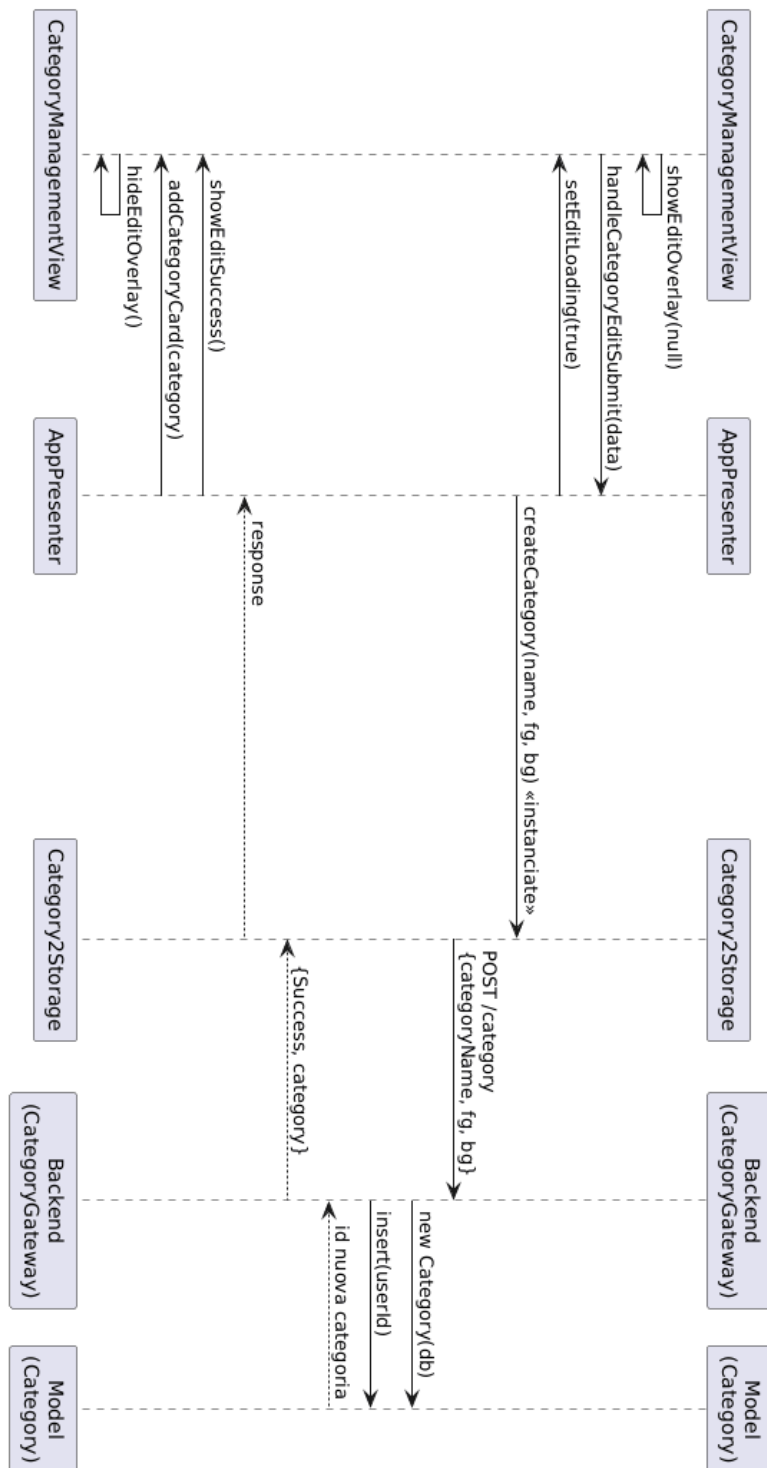


Figura 4.26: Diagramma di sequenza processo di creazione di una categoria.

4.4.2 Associazione Categorie-Transazione

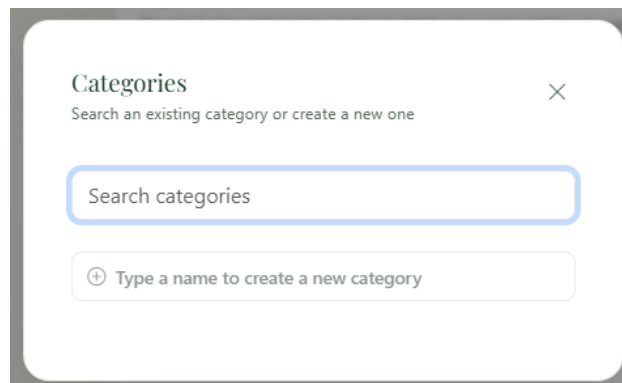
L'associazione tra categorie e transazioni può avvenire sia tramite selezione manuale da parte dell'utente sia tramite generazione automatica basata su modello linguistico. Nel primo caso, la `TransactionDetailsView` consente all'utente di cercare categorie esistenti, filtrandole in tempo reale attraverso richieste asincrone al backend. Le categorie selezionate vengono memorizzate localmente e successivamente incluse nel payload della transazione.

Nel secondo caso, l'endpoint `/category/generate` utilizza gli allegati della transazione per costruire un contesto semantico da fornire al modello linguistico. Il gateway prepara uno storico di messaggi, filtra gli allegati rilevanti e invoca il servizio `LLMService`. Il risultato atteso è un oggetto JSON contenente gli identificativi delle categorie suggerite, che vengono poi recuperate dal database.

```
1 $response = LLMService::extractJsonObject($response);  
2 foreach ($response["associated_categories"] as  
   $idCategory)  
3     $selectedCategories[] = (new Category($this->db))->  
       findById($idCategory);
```

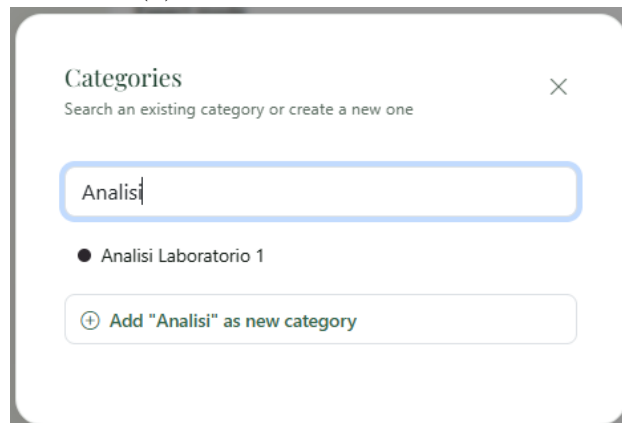
Codice 4.27: Ottenimento associazioni categorie-transazione

Il frontend integra automaticamente tali categorie nella vista di dettaglio della transazione, rendendole immediatamente disponibili all'utente.



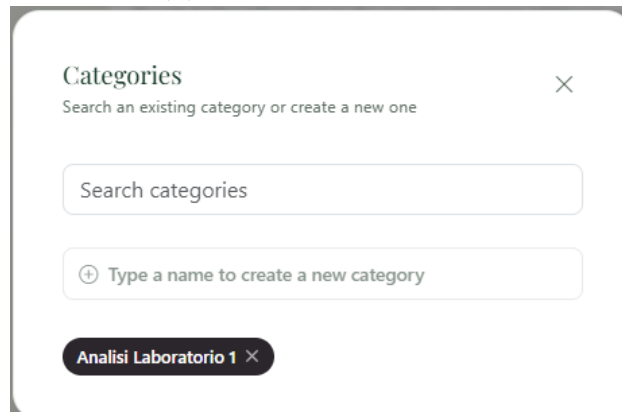
A screenshot of a mobile application interface titled "Categories" with a close button (X) in the top right corner. Below the title is the instruction "Search an existing category or create a new one". There are two input fields: the first is labeled "Search categories" and is empty; the second is labeled "Type a name to create a new category" and has a plus icon on the left.

(a) View creazione categorie.



A screenshot of the same "Categories" interface. The "Search categories" field now contains the text "Analisi". Below this field, a single search result is displayed: a radio button followed by the text "Analisi Laboratorio 1". The "Type a name to create a new category" field now has a plus icon and the text "Add 'Analisi' as new category".

(b) View risultati ricerca.



A screenshot of the "Categories" interface. The "Search categories" field is empty. The "Type a name to create a new category" field has a plus icon and the text "Type a name to create a new category". At the bottom, there is a dark button with the text "Analisi Laboratorio 1" and a close button (X) on the right.

(c) View selezione risultato ricerca.

Figura 4.27: View selezione categorie.

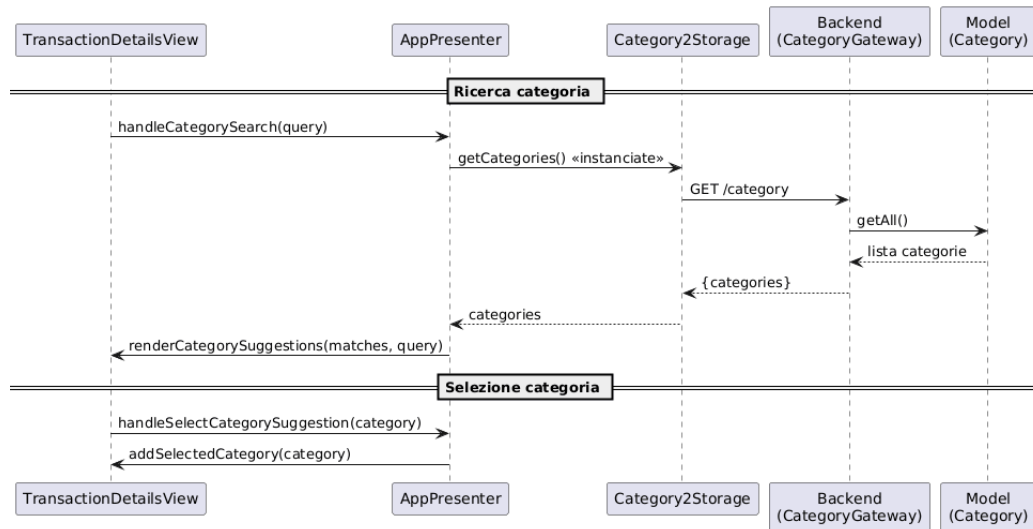


Figura 4.28: Diagramma di sequenza associazione categorie-transazione.

4.4.3 Modifica Categorie

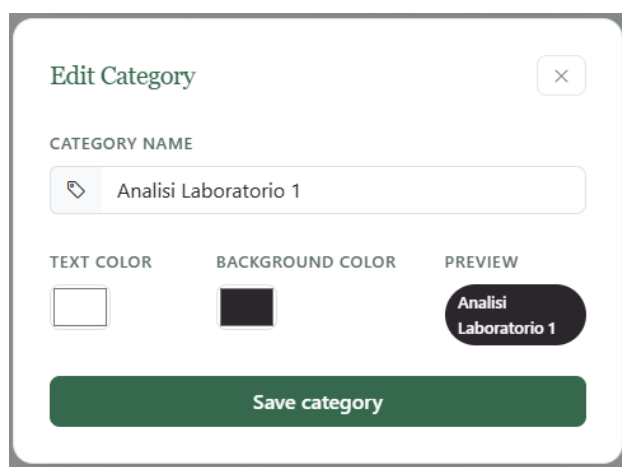
La modifica delle categorie è gestita tramite una richiesta PUT all'endpoint /category. Il gateway verifica la presenza dell'identificativo della categoria, recupera il documento corrispondente e applica le modifiche ai campi forniti nel payload. Prima di procedere, viene effettuato un controllo sui permessi tramite la classe PermissionManager, che determina se l'utente corrente è autorizzato a eseguire l'operazione.

```

1 if (!$this->permManager->canPerformOperation(
2     $currentUser, $category, Permission::
3     EDIT_CATEGORY)) {
4     ResponseHelper::error("Operation not Allowed.", 401,
5     $this->logManager);
6 }
  
```

Codice 4.28: Verifica permessi aggiornamento categorie

Una volta validate le modifiche, il metodo update() della classe Category aggiorna il documento nella collezione MongoDB. Sul lato frontend, la CategoryManagementView aggiorna dinamicamente la card corrispondente, garantendo una rappresentazione coerente dello stato aggiornato e una continuità visiva per l'utente amministratore.



The image shows a mobile application interface for editing a category. At the top, the title "Edit Category" is displayed in a green font, with a close button (X) to its right. Below the title, the label "CATEGORY NAME" is followed by a text input field containing "Analisi Laboratorio 1". Underneath, there are three sections: "TEXT COLOR" with a white color swatch, "BACKGROUND COLOR" with a dark blue color swatch, and "PREVIEW" showing a dark blue pill-shaped button with the text "Analisi Laboratorio 1". At the bottom, there is a large green button labeled "Save category".

Figura 4.29: View modifica categorie.

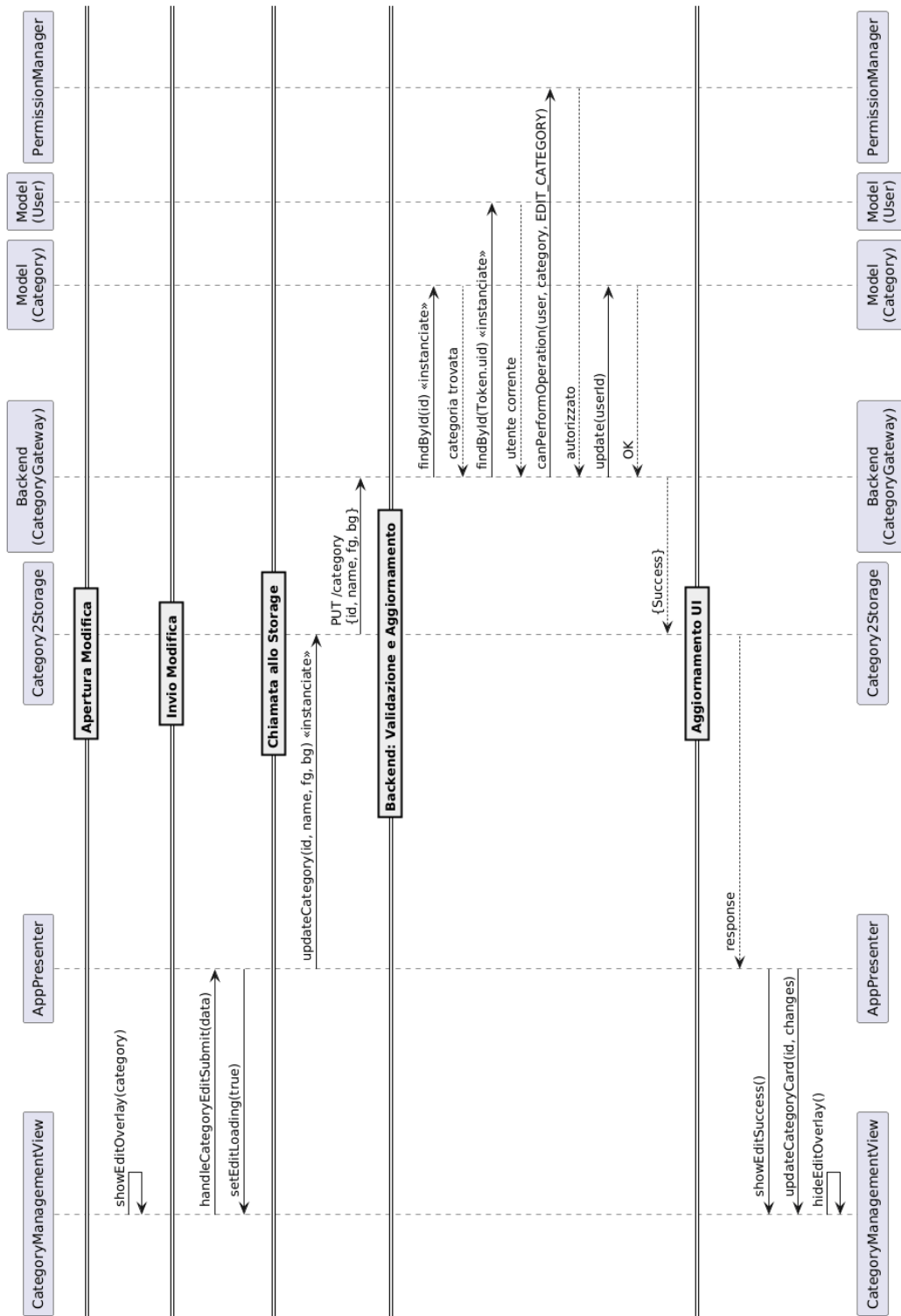


Figura 4.30: Diagramma di sequenza processo di modifica di una categoria.

4.5 Gestione Transazioni

La gestione delle transazioni costituisce la funzionalità principale dell'architettura applicativa, articolandosi in un insieme coordinato di componenti frontend e backend che cooperano per garantire la creazione, la modifica, la persistenza delle informazioni associate a ciascuna transazione.

La struttura complessiva può essere rappresentata tramite un diagramma UML delle classi, che evidenzia le relazioni tra Transaction, Attachment, Category, AttachmentGateway, PermissionManager, User e le componenti frontend responsabili dell'interazione.

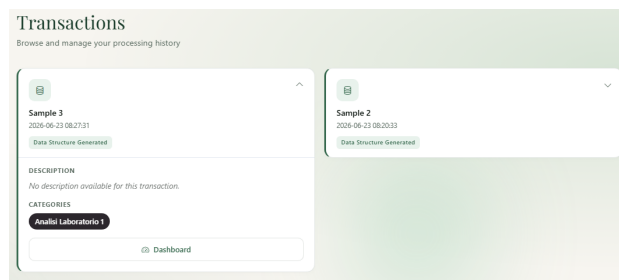


Figura 4.31: View lista transazioni.

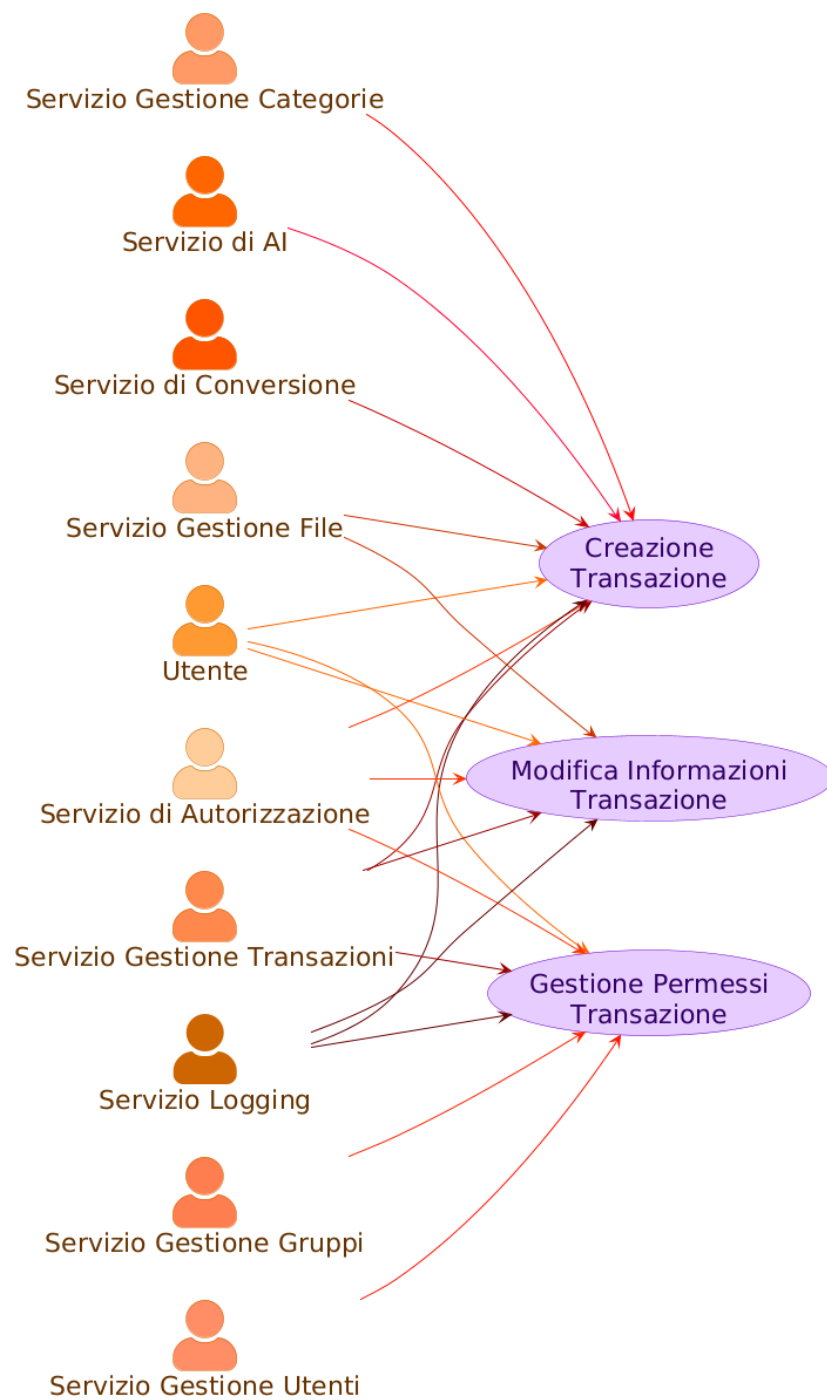


Figura 4.32: Diagramma dei casi d'uso relativi alla gestione transazioni.

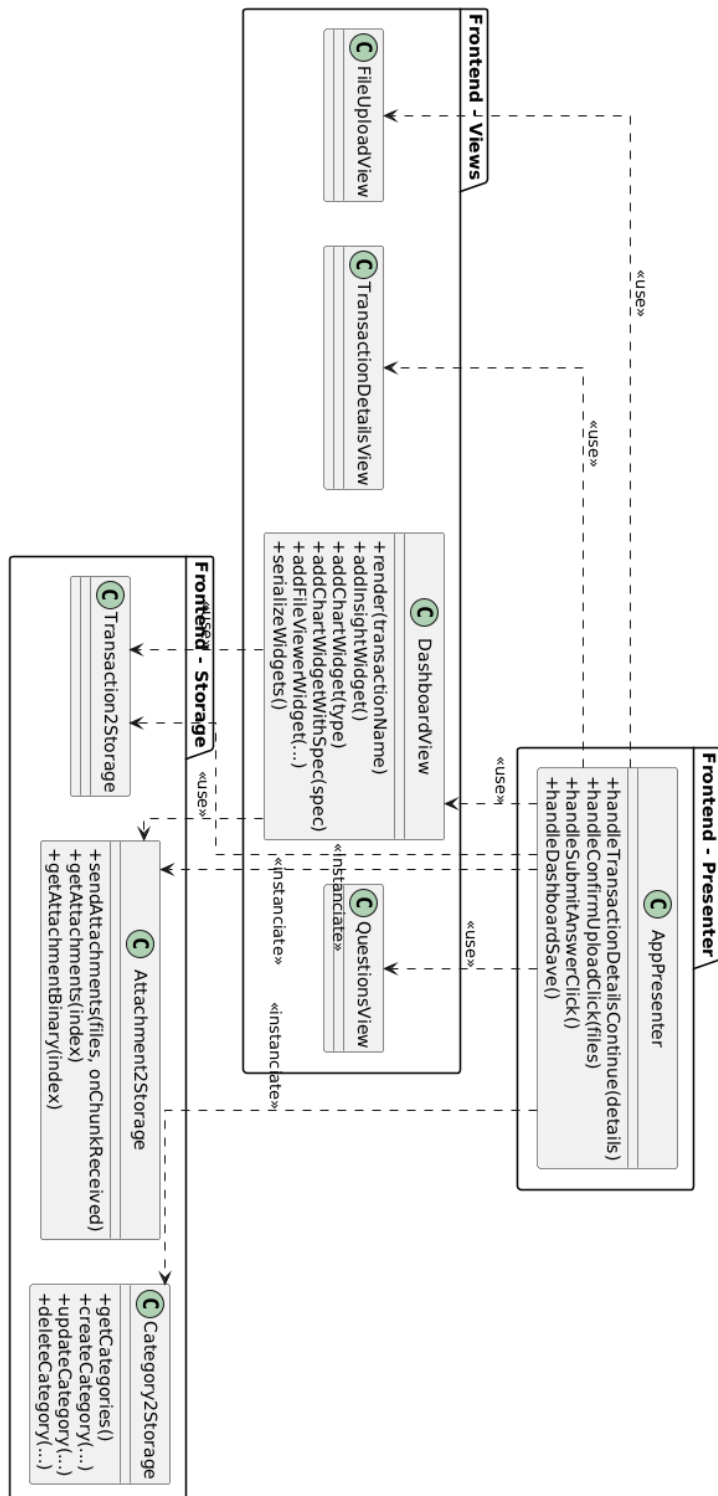


Figura 4.33: Diagramma delle classi frontend gestione transazioni.

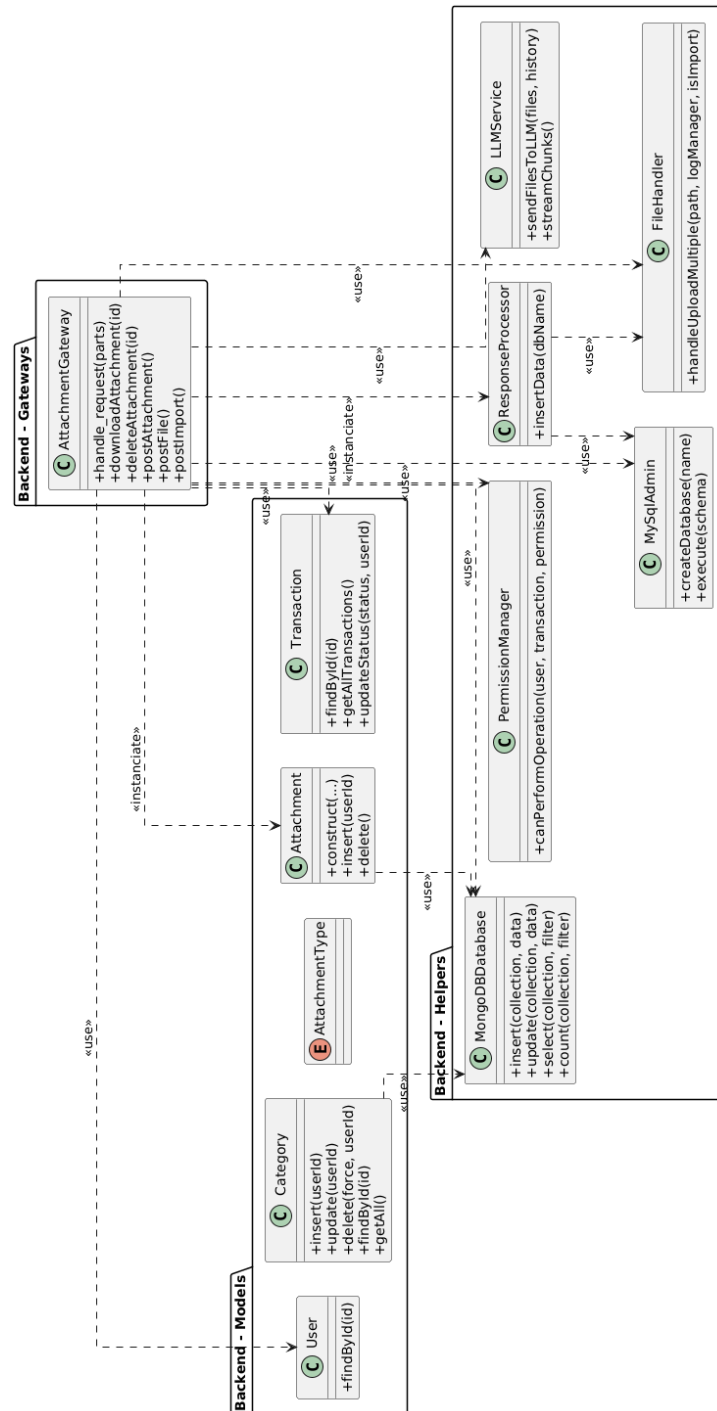


Figura 4.34: Diagramma delle classi backend gestione transazioni.

4.5.1 Creazione Transazioni

La creazione di una transazione rappresenta l'operazione mediante la quale viene generato il contenitore logico destinato a raccogliere allegati, categorie, dashboard e metadati generati durante l'elaborazione. Il processo di creazione è realizzato tramite la chiamata `POST /transaction`, gestita dal metodo `createTransaction` del `TransactionGateway`. Tale endpoint inizializza un nuovo documento di transazione nel database, assegnandogli un identificativo univoco e predisponendo le strutture necessarie per le fasi successive.

Una volta creata la transazione, il sistema consente immediatamente l'associazione di allegati, che costituiscono uno dei primi arricchimenti informativi. Gli allegati sono rappresentati dalla classe `Attachment`, progettata per essere incorporata direttamente nel documento della transazione. Questa scelta architetturale consente di mantenere coerenza tra metadati e contenuto, evitando riferimenti esterni e semplificando le operazioni di lettura e aggiornamento.

La costruzione dell'oggetto allegato avviene tramite il metodo `construct`, che inizializza i campi fondamentali quali nome originale, percorso relativo, tipo di allegato e flag di caricamento. Una volta popolato, l'oggetto viene inserito nella transazione tramite il metodo `insert`, che genera un nuovo identificatore univoco e registra le informazioni di tracciamento. Il codice mostra tale processo.

```

1 $this->id = (string) new ObjectId();
2 $this->upload_timestamp = date("Y-m-d H:i:s");
3 $this->db->update("transactions",
4 ['$push' => ["attachments" => [...]]]);

```

Codice 4.29: Salvataggio allegati transazione

Questa operazione realizza l'inserimento fisico dell'allegato nel documento di transazione, garantendo persistenza e integrità dei dati.

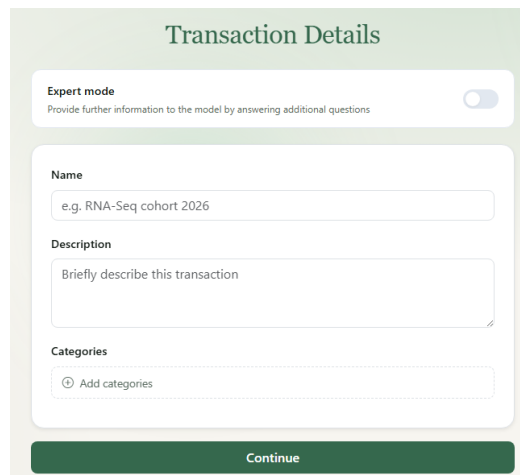
La gestione del caricamento degli allegati è orchestrata dal metodo `postAttachment()` della classe `AttachmentGateway`. Tale metodo recupera la transazione corrente tramite `findById`, verifica i permessi dell'utente e aggiorna lo stato della transazione a `OBTAINING_ATTACHMENTS`. Successivamente, i file vengono processati tramite `FileHandler::handleUploadMultiple`, che restituisce una struttura contenente nome, percorso e tipo di ciascun file. Per ogni file, il gateway crea un oggetto `Attachment`, imposta l'identificativo della transazione e invoca il metodo `insert`, completando così la fase di creazione degli allegati.

Parallelamente, il frontend svolge un ruolo essenziale nell'invio dei file al backend. La classe `Attachment2Storage` incapsula la logica di comunicazione

e utilizza un oggetto `FormData` per trasmettere i file. Di seguito un'estratto del metodo `sendAttachments`.

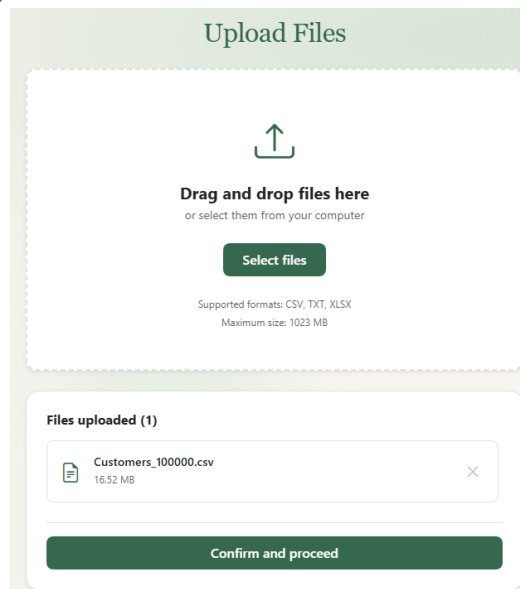
```
1 filesToSend.append("files[]", files[i]);  
2 client.MakeCall(App.HTTPMethod.POST, filesToSend, ...,  
    onChunkReceived);
```

Codice 4.30: Fase invio file



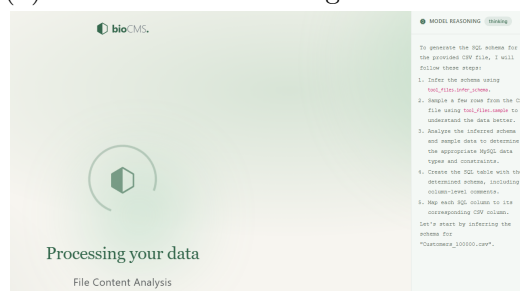
The 'Transaction Details' form is a light green card with a white background. At the top, it has a title 'Transaction Details' in a dark green font. Below the title is a toggle switch for 'Expert mode' with the text 'Provide further information to the model by answering additional questions'. The form contains three input fields: 'Name' with a placeholder 'e.g. RNA-Seq cohort 2026', 'Description' with a placeholder 'Briefly describe this transaction', and 'Categories' with a placeholder 'Add categories'. At the bottom of the card is a dark green button labeled 'Continue'.

(a) View inserimento informazioni transazione.



The 'Upload Files' section is a light green card with a white background. It features a large dashed border containing a green upload icon and the text 'Drag and drop files here or select them from your computer'. Below this is a dark green button labeled 'Select files'. Underneath the button, it lists 'Supported formats: CSV, TXT, XLSX' and 'Maximum size: 1023 MB'. Below the dashed border is a section titled 'Files uploaded (1)' showing a file named 'Customers_100000.csv' with a size of '16.52 MB'. At the bottom of the card is a dark green button labeled 'Confirm and proceed'.

(b) View inserimento allegati transazione.



The 'Processing your data' section is a light green card with a white background. It features a green bioCMS logo and a circular icon with a cube inside. The text 'Processing your data' and 'File Content Analysis' is displayed. On the right side, there is a 'MODEL REASONING' section with a 'Show' button and a list of steps: 1. Infer the schema using the provided CSV file, 2. Assign a role from the CSV file using the provided schema to understand the data better, 3. Assign the inferred schema and sample data to generate the appropriate R2G data type and constraints, 4. Create the R2G table with the generated schema, including column-level comments, 5. Map each R2G table to its corresponding CSV column. Let's start by inferring the schema for 'Customers_100000.csv'.

(c) View elaborazione dati da parte del modello linguistico.

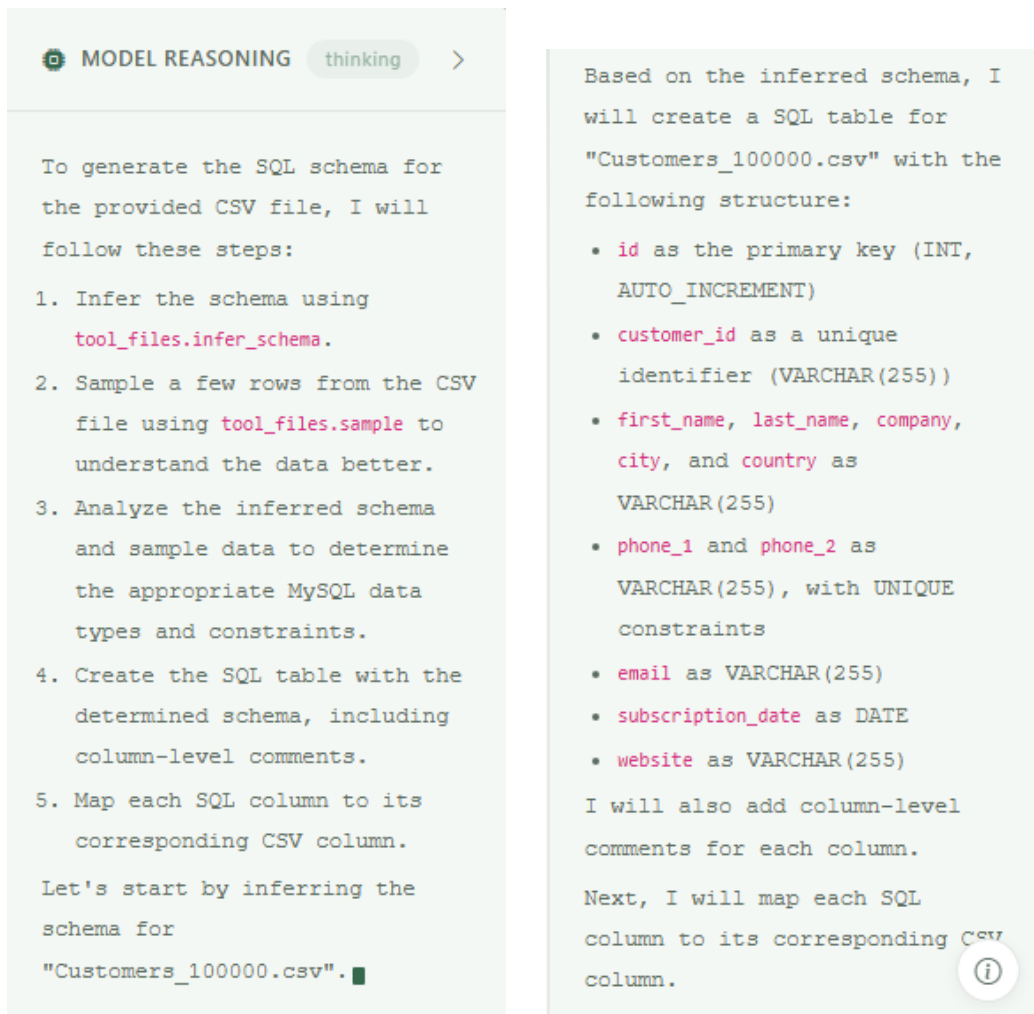
Figura 4.35: View creazione transazioni.

Una volta che i file sono stati inviati al backend e incorporati nella transazione, essi vengono inoltrati al servizio di elaborazione linguistica tramite la classe `LLMService`, responsabile della comunicazione con il backend LLM realizzato in Python. Tale servizio costruisce la richiesta, trasmette i contenuti dei file e gestisce il flusso di risposta generato dal modello. L'interazione non avviene in modalità sincrona, ma attraverso uno streaming progressivo di frammenti di output (chunk), che consente al sistema di elaborare e mostrare in tempo reale il reasoning prodotto dal modello.

Il backend PHP riceve questi chunk e li analizza man mano che vengono generati. Durante questa fase, il sistema distingue tra la parte di reasoning interno del modello e la risposta finale strutturata. Il reasoning viene immediatamente inoltrato al frontend, che lo visualizza tramite il callback `onChunkReceived`. Il codice frontend che gestisce tale comportamento è esemplificato dal seguente estratto.

```
1 return await client.MakeCall(App.HTTPMethod.POST,  
    filesToSend, App.HTTPType.JSON, App.HTTPContentType.  
    FORM_DATA, false, onChunkReceived);
```

Codice 4.31: Callback `onChunkReceived`



(a) Primo estratto del reasoning ottenuto dal modello linguistico.

(b) Secondo estratto del reasoning ottenuto dal modello linguistico.

Figura 4.36: Reasoning prodotto dal modello linguistico.

Questo meccanismo consente all'interfaccia utente di aggiornarsi dinamicamente, mostrando l'evoluzione del processo inferenziale del modello senza attendere la risposta finale.

La risposta definitiva prodotta dal modello segue una struttura rigorosa, definita dal sistema e imposta tramite lo schema JSON fornito al modello stesso. Essa assume la forma seguente.

```
{ "sql_schema": "",
  "column_mapping": [] }
```

Il campo `sql_schema` contiene la definizione completa delle tabelle da creare, comprensiva di colonne, tipi, chiavi primarie e vincoli. Il campo `column_mapping` rappresenta invece la mappatura tra le colonne del database e le colonne dei file caricati, secondo la sintassi

`nomefile.colonna->nomefile.estensione.colonna.`

Una volta ottenuta la risposta finale, il backend avvia la fase di costruzione del database tramite la classe `ResponseProcessor`. Tale classe riceve in ingresso lo schema SQL e la mappatura delle colonne, come mostrato nel codice seguente.

```
1 $processor = new ResponseProcessor((object)[ "sql_schema"  
    " => $schema, "column_mapping" => $normalizedMapping  
    ], $importFiles, $this->mysqlAdmin, $this->logManager  
    );
```

Codice 4.32: Istanziatura `ResponseProcessor`

Il processore provvede innanzitutto alla creazione del database associato alla transazione, utilizzando lo schema SQL generato dal modello per definire tabelle, colonne e vincoli. Prima di procedere alla generazione fisica delle tabelle, esso analizza le dipendenze intrarelazionali espresse tramite chiavi esterne e costruisce il grafo delle relazioni tra le entità. Su tale grafo viene applicato un algoritmo di ordinamento topologico, che consente di ottenere una sequenza lineare di creazione delle tabelle compatibile con i vincoli di dipendenza. In questo modo, ogni tabella viene creata solo dopo che tutte le tabelle da cui dipende sono già state generate, garantendo la correttezza strutturale del database risultante. Successivamente, procede all'inserimento dei dati provenienti dai file caricati, applicando la mappatura definita nel campo `column_mapping`. Ogni colonna del file viene associata alla colonna corrispondente della tabella SQL, garantendo coerenza semantica e integrità referenziale.

Il processo di importazione prevede la lettura dei file CSV o Excel, la normalizzazione dei nomi delle colonne e la trasformazione dei valori secondo le specifiche del modello. Per garantire la scalabilità del sistema e supportare dataset di grandi dimensioni, la lettura dei file non avviene mediante il caricamento integrale del contenuto in memoria, ma attraverso l'impiego di iteratori che consentono di processare i dati riga per riga. Tale approccio evita l'allocazione di strutture dati estese in RAM e permette al sistema di gestire file di dimensioni potenzialmente molto elevate senza degradazioni prestazionali o rischi di esaurimento della memoria.

Il metodo `insertData` del `ResponseProcessor` realizza l'inserimento dei dati nel database della transazione sfruttando proprio questa modalità iterativa.

Ogni riga del file viene letta progressivamente, trasformata secondo le regole definite dal modello e mappata verso la colonna SQL corrispondente tramite il `column_mapping`. L'uso degli iteratori consente quindi di mantenere costante l'occupazione di memoria, indipendentemente dalla dimensione del dataset, e di effettuare l'inserimento in streaming, garantendo efficienza e robustezza anche in presenza di file di input molto voluminosi.

```
1 $processor->insertData($transaction->db_name);
```

Codice 4.33: Inserimento dati all'interno del database creato

Una volta completata la fase di popolamento, il database risulta pienamente operativo e pronto per essere interrogato, visualizzato o analizzato tramite la dashboard della transazione.

L'intero flusso, dalla trasmissione dei file al modello fino alla creazione del database, può essere rappresentato tramite un diagramma di sequenza che evidenzia l'interazione tra frontend, gateway PHP, servizio LLM e processore di risposta:

di chiarificazione guidata. Tale modalità è progettata per gestire situazioni in cui i file forniti non contengono informazioni sufficienti per consentire al modello di generare autonomamente uno schema SQL completo e una mappatura delle colonne coerente. In questo scenario, il modello non procede immediatamente alla produzione della risposta finale, ma esegue una fase preliminare di analisi dei dati in ingresso, valutando la presenza di ambiguità, lacune informative o scelte progettuali che richiedono un intervento esplicito dell'utente.

Quando il modello rileva una di queste condizioni, esso produce un messaggio strutturato di tipo *question*, che viene intercettato dal backend e inoltrato al frontend. Tale messaggio contiene una domanda formulata dal modello e rappresenta un punto di decisione necessario per completare la modellazione dei dati. Le domande possono riguardare, ad esempio, la natura semantica di una colonna, la relazione tra due entità, la scelta del tipo di chiave primaria, la necessità di normalizzare un attributo o la gestione di valori mancanti. Ogni domanda è formulata in modo da richiedere una risposta univoca e direttamente utilizzabile dal modello.

Il frontend presenta la domanda all'utente attraverso l'interfaccia dedicata, permettendogli di fornire una risposta testuale o selezionare un'opzione tra quelle proposte. La risposta dell'utente viene quindi reinserita nel contesto conversazionale della transazione, arricchendo la storia che viene nuovamente inviata al modello. Questo meccanismo consente al modello di aggiornare la propria comprensione del dominio dei dati e di integrare le informazioni mancanti direttamente dalle indicazioni dell'utente.

Il processo è di natura iterativa: dopo ogni risposta, il modello valuta se le informazioni acquisite siano sufficienti per procedere alla generazione dello schema SQL e della mappatura delle colonne, oppure se siano necessarie ulteriori chiarificazioni. Qualora persistano ambiguità, il modello produce una nuova domanda, dando luogo a un ciclo domanda-risposta che prosegue fino alla completa risoluzione delle incertezze. Solo quando il modello dispone di tutte le informazioni necessarie, esso produce la risposta finale nel formato previsto, contenente i campi `sql_schema` e `column_mapping`.

Questo approccio consente di ottenere una rappresentazione strutturale del database più accurata e aderente alle intenzioni dell'utente, evitando assunzioni implicite e garantendo un controllo più fine sul processo di modellazione dei dati. La modalità avanzata si configura quindi come un dialogo guidato tra utente e modello, in cui il sistema utilizza la capacità del modello linguistico di individuare incertezze e formulare domande mirate per costruire una struttura dati completa, coerente e semanticamente corretta.

In sintesi, la creazione delle transazioni si configura come un processo multilivello in cui il backend garantisce persistenza, coerenza e controllo dei

permessi, mentre il frontend gestisce l'interazione utente e la trasmissione dei file. L'integrazione tra questi componenti produce un flusso robusto, tracciabile e predisposto per le successive fasi di analisi, visualizzazione e arricchimento semantico e permette al sistema di trasformare file eterogenei in una struttura dati coerente, interrogabile e persistente, garantendo al contempo trasparenza all'utente grazie alla visualizzazione progressiva del reasoning prodotto dal modello..

4.5.2 Gestione Permessi

La gestione dei permessi relativi alle transazioni costituisce un elemento centrale del sistema, poiché determina quali utenti siano autorizzati ad accedere, modificare o elaborare una specifica transazione. Ogni transazione è infatti associata a un proprietario, identificato dal campo `created_by`, e può essere consultata o manipolata solo da utenti che dispongono delle autorizzazioni necessarie. Il modello di sicurezza adottato è di tipo discrezionale: il proprietario della transazione e gli amministratori del sistema possiedono privilegi completi, mentre gli altri utenti possono interagire con la transazione solo se esplicitamente autorizzati.

Il controllo dei permessi viene effettuato dal componente `PermissionManager`, il quale incapsula la logica di verifica e viene invocato dai gateway prima dell'esecuzione di qualsiasi operazione sensibile. Tale componente riceve in ingresso l'utente corrente, la transazione richiesta e il tipo di operazione da eseguire, restituendo un valore booleano che indica se l'azione è consentita. Un estratto significativo del codice mostra la struttura del controllo.

```
1 if (!$this->permManager->canPerformOperation(  
    $currentUser, $transaction, "upload_attachment")) {  
2     ResponseHelper::error("Operation not allowed.", 401,  
        $this->logManager);  
3 }
```

Codice 4.34: Verifica permessi su una transazione

Questo frammento evidenzia come il sistema impedisca l'accesso a operazioni critiche, quali il caricamento degli allegati, qualora l'utente non sia il proprietario della transazione o non disponga dei privilegi necessari. La stessa logica viene applicata in tutte le fasi del ciclo di vita della transazione: creazione degli allegati, avvio dell'elaborazione avanzata, visualizzazione dei risultati e modifica delle informazioni.

Il `PermissionManager` implementa una serie di regole che definiscono in modo rigoroso le condizioni di accesso. In particolare, un utente è autorizzato a interagire con una transazione se ricorre almeno una delle seguenti con-

dizioni: è il proprietario della transazione, è un amministratore del sistema oppure appartiene a un gruppo che possiede diritti specifici sulla transazione. La logica di controllo è espressa nel codice che segue.

```

1 public function canPerformOperation($user, $transaction,
   $operation) {
2     if ($user->is_admin) return true;
3     if ($transaction->created_by === $user->id) return
   true;
4     return in_array($operation, $user->
   transaction_permissions[$transaction->id] ?? []);
5 }

```

Codice 4.35: Logica di autorizzazione nel PermissionManager

Questa struttura consente al sistema di supportare scenari complessi, come la condivisione selettiva delle transazioni tra utenti o gruppi, mantenendo al contempo un elevato livello di sicurezza e tracciabilità.

Il frontend riflette dinamicamente tali permessi, mostrando o nascondendo le funzionalità in base alle capacità dell'utente. Ad esempio, la vista di caricamento degli allegati abilita il pulsante di upload solo se l'utente è autorizzato a eseguire l'operazione. Tale comportamento è ottenuto tramite una valutazione locale dei permessi, come illustrato nel seguente estratto.

```

1 const canUpload = transaction.created_by === currentUser
   .id || currentUser.is_admin;
2 if (!canUpload) disableUploadButton();

```

Codice 4.36: Controllo permessi lato frontend

In questo modo, l'interfaccia utente si adatta automaticamente al ruolo dell'utente, prevenendo tentativi di accesso non autorizzati e migliorando l'esperienza d'uso.

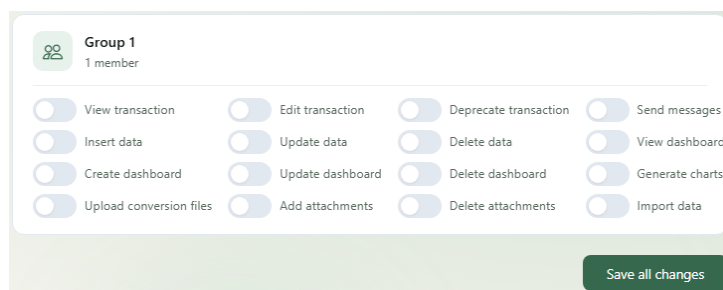


Figura 4.38: View gestione permessi transazioni.

In sintesi, la gestione dei permessi sulle transazioni si basa su un modello di autorizzazione robusto e flessibile, che combina controlli centralizzati nel backend con adattamenti dinamici dell'interfaccia frontend. Tale approccio garantisce che ogni operazione venga eseguita nel rispetto delle regole di sicurezza, preservando l'integrità dei dati e la coerenza del sistema. La seguente tabella riassume tutti i permessi attivabili dal proprietario della transazione, specificandone il significato e l'ambito di applicazione all'interno del sistema.

Variabile	Descrizione
CAN_VIEW_TRANSACTION	Permette all'utente di visualizzare una transazione e i suoi metadati.
CAN_UPLOAD_ATTACHMENTS	Consente il caricamento di nuovi allegati nella transazione.
CAN_DELETE_ATTACHMENTS	Autorizza la rimozione degli allegati già presenti.
CAN_START_ADVANCED_PROCESSING	Permette di avviare la modalità avanzata con interazione LLM.
CAN_ANSWER_QUESTIONS	Consente di rispondere alle domande generate dal modello durante l'elaborazione.
CAN_VIEW_REASONING	Permette di visualizzare i chunk di reasoning prodotti dal modello.
CAN_VIEW_RESULTS	Autorizza l'accesso allo schema SQL e alla mappatura generati dal modello.
CAN_IMPORT_DATA	Consente l'avvio del processo di importazione dei dati nel database generato.
CAN_VIEW_DASHBOARD	Permette di accedere alla dashboard della transazione.
CAN_EDIT_DASHBOARD	Consente la modifica, creazione e rimozione dei widget della dashboard.
CAN_EDIT_TRANSACTION_INFO	Autorizza la modifica delle informazioni descrittive della transazione.
CAN_DELETE_TRANSACTION	Permette la cancellazione dell'intera transazione.
CAN_CREATE_USERS	Consente la creazione di nuovi utenti nel sistema.
CAN_EDIT_USERS	Autorizza la modifica dei dati e dei permessi degli utenti.
CAN_DELETE_USERS	Permette la rimozione degli utenti dal sistema.
CAN_VIEW_LOGS	Consente la visualizzazione dei log di sistema e delle attività.
CAN_MANAGE_GROUPS	Autorizza la creazione, modifica e cancellazione dei gruppi utente.
CAN_MANAGE_CATEGORIES	Permette la gestione delle categorie (creazione, modifica, eliminazione).

Tabella 4.2: Permessi attivabili sulle transazioni.

4.5.3 Modifica Informazioni Transazione

La modifica delle informazioni associate a una transazione comprende anche la gestione del suo stato applicativo, che non influisce in alcun modo sul comportamento del modello e le operazioni che si possono svolgere sui dati della transazione ma ha la funzione di comunicare all'utente il livello di aggiornamento dei dati. Quando una transazione viene dichiarata obsoleta, il sistema non altera i risultati generati né rimuove gli allegati, ma aggiorna esclusivamente tre campi del documento MongoDB che ne descrivono la condizione. Il primo è `is_deprecated`, un flag booleano che indica se la transazione è stata effettivamente deprecata. Il secondo è `substituting_transaction_id`, che contiene l'identificativo della transazione più recente che sostituisce quella corrente, qualora esista. Il terzo è `deprecated_at`, un timestamp che registra il momento esatto in cui la transazione è stata marcata come non più attuale.

Queste informazioni permettono al sistema di segnalare chiaramente all'utente che i dati contenuti nella transazione non rappresentano più la versione più aggiornata e che, se disponibile, esiste una transazione sostitutiva generata a partire da file più recenti o da una nuova elaborazione. Il frontend uti-

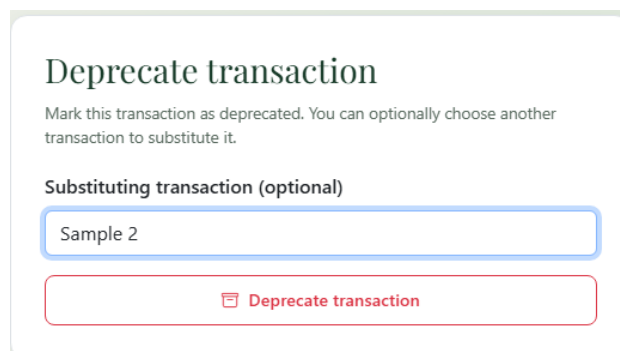
lizza tali campi per mostrare un avviso contestuale, guidando l'utente verso la transazione più affidabile senza impedirgli di consultare quella precedente.

Dal punto di vista implementativo, il deprecamento consiste in un aggiornamento atomico del documento della transazione, come mostrato nel seguente estratto.

```
1 public function deprecate(string $substitutingId, string
   $timestamp): bool {
2     return $this->db->update(
3         "transactions",
4         ["id" => $this->id],
5         [
6             "is_deprecated" => true,
7             "substituting_transaction_id" =>
8             $substitutingId,
9             "deprecated_at" => $timestamp
10        ]
11    );
}
```

Codice 4.37: Aggiornamento dei campi di deprecamento

Una transazione deprecata rimane completamente consultabile: l'utente può continuare a visualizzare lo schema SQL generato, la mappatura delle colonne, gli allegati e la dashboard. Tuttavia, il sistema impedisce l'avvio di nuove elaborazioni o l'importazione di dati su una transazione obsoleta, evitando che informazioni non aggiornate vengano ulteriormente propagate. In questo modo, il deprecamento svolge una funzione informativa e gestionale, preservando la storicità dei risultati e mantenendo la tracciabilità delle versioni, mentre guida l'utente verso l'utilizzo della transazione più recente e affidabile.



Deprecate transaction

Mark this transaction as deprecated. You can optionally choose another transaction to substitute it.

Substituting transaction (optional)

Sample 2

 Deprecate transaction

Figura 4.39: View deprecamento transazioni.

4.5.4 Creazione Vetrine Transazioni

Per rendere disponibili i dati delle transazioni anche a utenti esterni al sistema, è stata introdotta una funzionalità dedicata alla creazione delle cosiddette vetrine. Una vetrina rappresenta una proiezione pubblicabile dei contenuti della transazione, costruita a partire dai dati elaborati dal modello e dagli allegati caricati dall'utente. L'obiettivo è consentire la diffusione controllata delle informazioni, mantenendo al contempo la separazione tra l'ambiente interno dell'applicazione e gli utenti che non dispongono di un account o di permessi diretti sulla transazione originale.

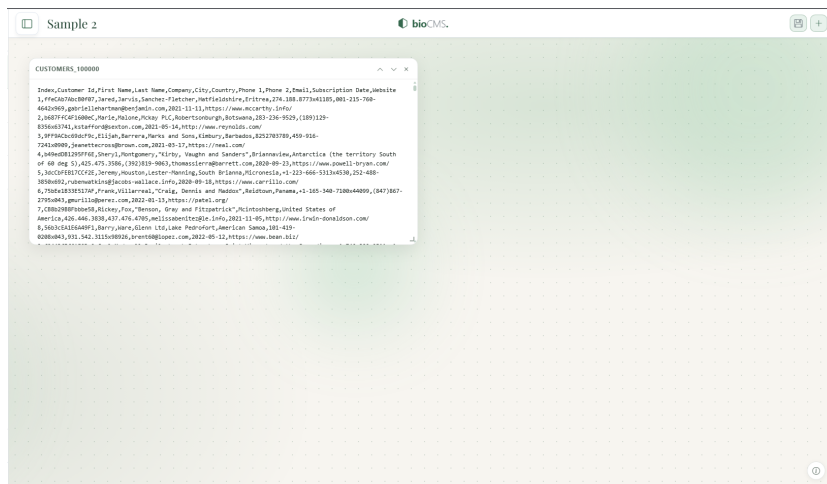
La generazione di una vetrina avviene a partire dai risultati prodotti dal modello, come lo schema SQL, la mappatura delle colonne e gli eventuali dati importati, integrati con gli allegati che l'utente ha deciso di rendere pubblici. Il sistema costruisce una rappresentazione coerente e autonoma della transazione, che può essere consultata senza accedere al backend principale. In questo modo, la vetrina diventa un contenitore autosufficiente, pensato per essere condiviso con terze parti, utilizzato come documentazione o pubblicato in contesti esterni.

Dal punto di vista applicativo, la creazione di una vetrina non modifica la transazione originale, ma ne deriva una versione esportabile che riflette lo stato corrente dei dati. L'utente può decidere quali elementi includere, quali allegati rendere visibili e quali informazioni mantenere riservate, come mostrare queste informazioni, aggregate mediante l'utilizzo di grafici generati utilizzando un modello linguistico oppure tabellari mediante l'utilizzo di tabelle. Il sistema garantisce che la vetrina rimanga sincronizzata con la transazione fino al momento della sua generazione; eventuali modifiche successive non vengono propagate automaticamente, preservando così la stabilità del contenuto pubblicato.

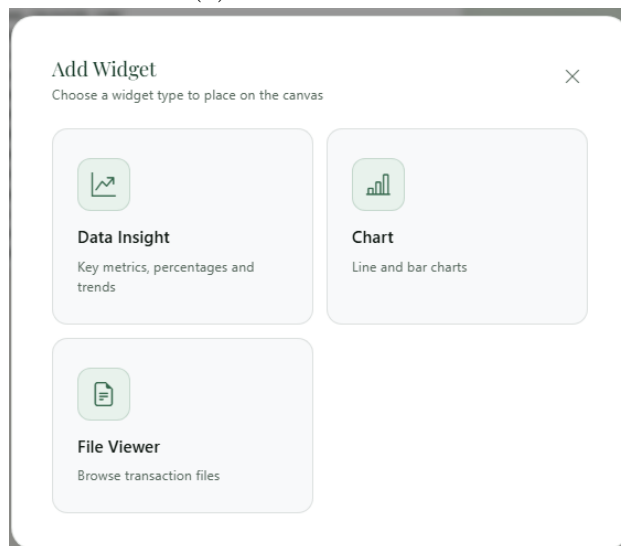
La funzionalità è stata progettata per rispondere a esigenze di condivisione, trasparenza e interoperabilità. Le vetrine permettono infatti di distribuire i risultati delle elaborazioni senza esporre l'intera infrastruttura applicativa, mantenendo un elevato livello di sicurezza e controllo. Allo stesso tempo, offrono un mezzo efficace per comunicare i dati elaborati a clienti, collaboratori o sistemi esterni, senza richiedere loro l'accesso diretto alla piattaforma interna.

In sintesi, la creazione delle vetrine costituisce un'estensione naturale del ciclo di vita della transazione, trasformando i dati elaborati in un artefatto condivisibile e stabile, progettato per essere consultato da utenti esterni in modo sicuro, controllato e indipendente dal sistema principale.

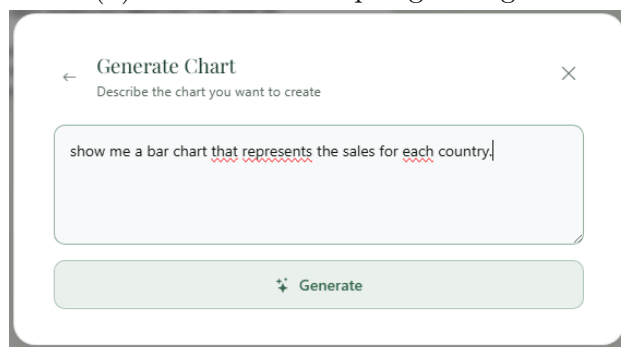
CAPITOLO 4. METODOLOGIA



(a) View dashboard.



(b) View selezione tipologia widget.



(c) View inserimento prompt per generazione grafico.

Figura 4.40: View creazione vetrine.

4.6 Gestione Logging

Il sistema di logging implementato nel codice allegato si configura come un'infrastruttura modulare, estensibile e coerente, progettata per garantire un tracciamento uniforme delle operazioni svolte all'interno della piattaforma. L'architettura si fonda su una classe astratta comune, BaseLogger, da cui derivano logger specializzati, e su un gestore centrale, il LogManager, che coordina la selezione e l'utilizzo dei diversi componenti. Tale struttura consente di mantenere una chiara separazione delle responsabilità e un'elevata coesione interna, favorendo la manutenibilità del sistema.

La classe BaseLogger rappresenta il nucleo dell'intero meccanismo. Essa definisce il metodo astratto formatData(), che ogni sottoclasse deve implementare per normalizzare i dati specifici del proprio dominio applicativo, e fornisce il metodo finale log(), responsabile dell'intero processo di validazione, formattazione e inserimento dei dati nel database. Di seguito un estratto significativo del metodo mostra la costruzione dinamica della query SQL e la sua esecuzione.

```
1 $formatted = $this->formatData($data);
2 $formatted["level"] = $level->value;
3 $sql = "INSERT INTO {$this->table} ($columns) VALUES (
4     $placeholders)";
5 return $this->db->insert($sql, $params, $useTransaction)
6     ;
```

Il metodo si occupa inoltre di inviare messaggi al log nativo di PHP quando il livello di severità è ERROR o CRITICAL, garantendo un duplice livello di tracciamento.

I logger concreti estendono BaseLogger e definiscono la tabella di destinazione e la logica di normalizzazione dei dati. La classe UserActionLogger registra le azioni degli utenti, includendo identificativo, timestamp, componente e dettagli opzionali. La struttura dei dati è definita come segue.

```
1 return [
2     "user_id"    => $data["user_id"],
3     "timestamp"  => date("Y-m-d H:i:s"),
4     "action"     => $data["action"],
5     "component"  => $data["component"],
6     "details"    => $data["details"] ?? null
7 ];
```

Il logger `ApiRESTfulLogger` è dedicato alle chiamate RESTful e registra metodo HTTP, endpoint, codice di stato, tempo di generazione della risposta e indirizzo IP. La normalizzazione dei dati avviene tramite.

```

1 return [
2     "method"      => $data["method"],
3     "endpoint"    => $data["endpoint"],
4     "status_code" => $data["status_code"],
5     "generation_ms" => $data["generation_ms"],
6     "ip"          => $data["ip"],
7     "error"       => $data["error"] ?? null,
8     "timestamp"   => date("Y-m-d H:i:s")
9 ];

```

La classe `LlmLogger` registra informazioni relative alle operazioni LLM, come identificativo della transazione, tipo di prompt, ordine dei file elaborati e stato dell'operazione. La struttura dei dati è definita da.

```

1 return [
2     "id_transaction" => $data["id_transaction"],
3     "prompt_type"   => $data["prompt_type"],
4     "file_number"   => $data["file_number"],
5     "ip"            => $data["ip"],
6     "status"        => $data["status"],
7     "timestamp"     => date("Y-m-d H:i:s")
8 ];

```

I logger `MongoDBLogger` e `MySQLLogger` condividono una struttura simile, registrando tipo di query, contenuto della query, durata, stato dell'operazione e messaggi di errore. Essi permettono di monitorare in modo uniforme le operazioni sui database, indipendentemente dalla tecnologia utilizzata.

Il ruolo del `LogManager` è centrale nell'architettura. Esso riceve una richiesta di logging, valida il tipo di log tramite l'enum `LogType` e istanzia dinamicamente il logger appropriato attraverso un'espressione `match`. Il frammento seguente mostra la selezione del logger.

```

1 $logger = match ($type) {
2     LogType::USER      => new UserActionsLogger($this->db)
3     ,
4     LogType::API       => new ApiRESTfulLogger($this->db),
5     LogType::MONGODB   => new MongoDBLogger($this->db),
6     LogType::LLM       => new LlmLogger($this->db),

```

CAPITOLO 4. METODOLOGIA

```
6     LogType::MYSQL    => new MySqlLogger($this->db)
7 };
```

Questa soluzione consente ai componenti esterni di interagire con un'unica interfaccia coerente, senza conoscere i dettagli implementativi dei singoli logger. Il LogManager offre inoltre un metodo `getLog()` che permette di recuperare i dati registrati da ciascun logger, mantenendo un accesso uniforme alle diverse tabelle.

La gestione dei livelli di log è affidata all'enum `LogLevel`, che definisce i livelli `INFO`, `WARNING`, `ERROR` e `CRITICAL`. Il metodo statico `isValid()` garantisce la validazione dei livelli, mentre la classe `BaseLogger` utilizza tali informazioni per determinare se inviare messaggi al log nativo di PHP.

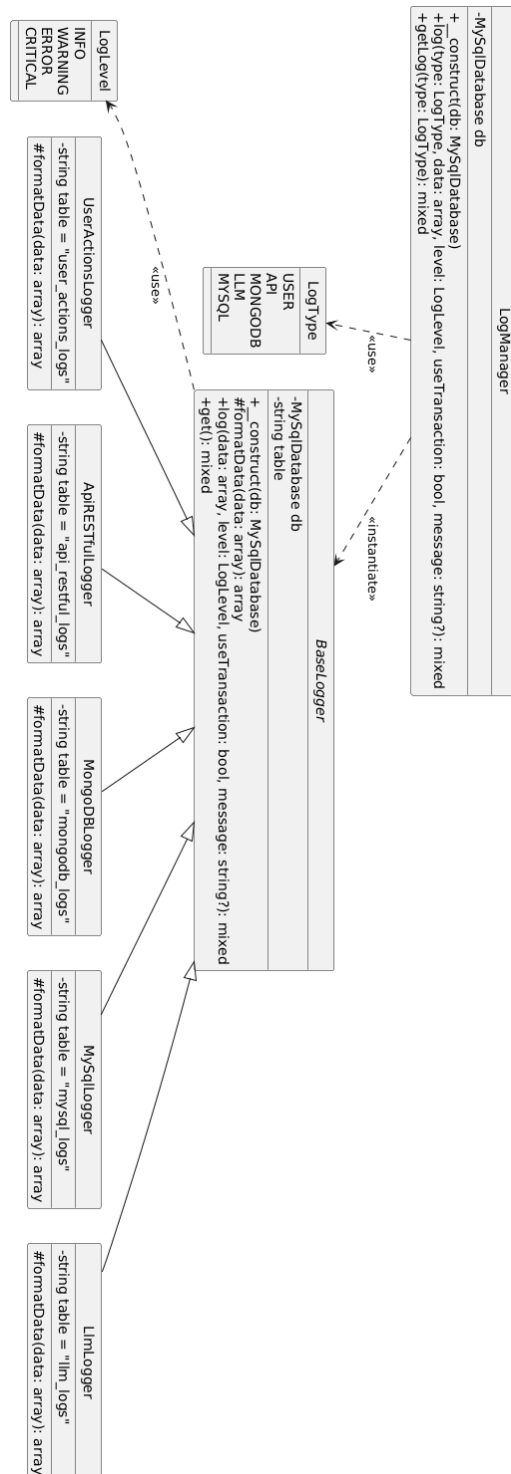


Figura 4.41: Diagramma delle classi logging.

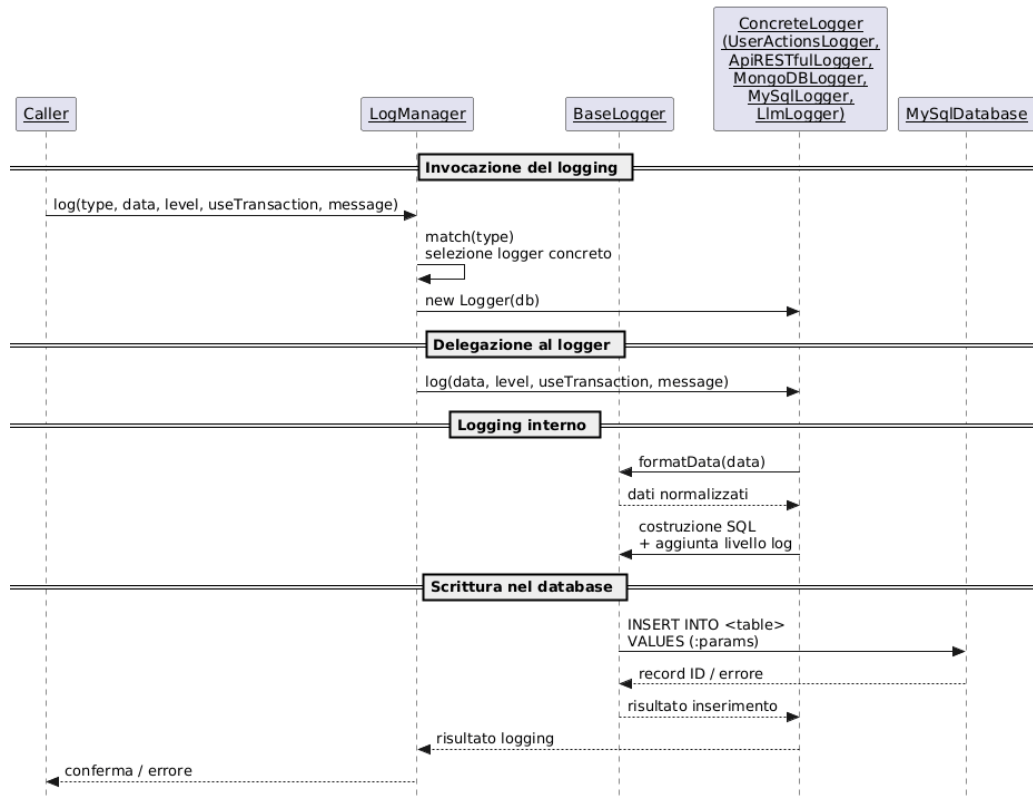


Figura 4.42: Diagramma di sequenza logging.

Nel complesso, il sistema di logging si distingue per chiarezza strutturale, modularità e robustezza. La presenza di una classe astratta che centralizza la logica comune, unita a logger specializzati che implementano esclusivamente la normalizzazione dei dati, consente di mantenere un elevato livello di estensibilità. Il LogManager garantisce un'interfaccia uniforme verso l'esterno, mentre l'uso di enum per la gestione dei tipi e dei livelli di log assicura coerenza e prevenzione degli errori.

Capitolo 5

Risultati

L'insieme delle ottimizzazioni introdotte nel sistema ha prodotto un miglioramento significativo delle prestazioni complessive. In particolare, l'integrazione di un meccanismo di cache per la gestione delle risposte parziali del modello e la ristrutturazione dell'analisi dell'output hanno ridotto drasticamente i tempi medi di elaborazione. Le misurazioni effettuate mostrano che il tempo necessario per completare un'intera pipeline di analisi è passato dai precedenti tre minuti di media a circa un minuto, con una riduzione superiore al 60% rispetto alla versione iniziale del sistema.

Tali risultati sono stati ottenuti eseguendo i test su una macchina dotata di processore Intel i5-10600K, scheda grafica AMD Radeon RX 5700 XT e 32 GB di memoria RAM. Questa configurazione hardware, pur non appartenendo alla fascia più alta del mercato, ha garantito un ambiente stabile e rappresentativo per valutare l'efficacia delle ottimizzazioni software introdotte.

Per valutare l'impatto delle ottimizzazioni anche su dataset di dimensioni crescenti, sono stati eseguiti test su quattro file contenenti rispettivamente 1 000, 10 000, 100 000 e 1 000 000 record. I tempi di elaborazione registrati sono riportati nel grafico seguente, che evidenzia l'andamento quasi lineare dell'incremento del tempo al crescere della dimensione del dataset. Tale comportamento conferma la buona scalabilità del sistema, che mantiene tempi di risposta contenuti anche in presenza di volumi di dati elevati.

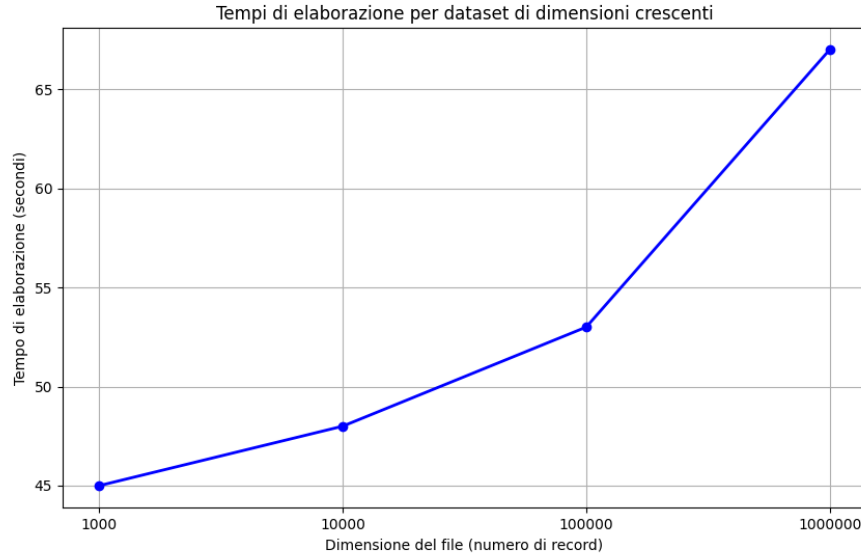


Figura 5.1: Tempi di elaborazione per dataset di dimensioni crescenti.

L'analisi dei risultati mostra come il sistema, grazie alle ottimizzazioni introdotte, sia ora in grado di gestire in modo efficiente dataset anche molto estesi, mantenendo tempi di elaborazione competitivi e garantendo un'esperienza d'uso notevolmente migliorata. La combinazione tra caching, ottimizzazione dell'analisi dell'output e gestione incrementale dei dati ha permesso di ottenere un sistema più reattivo e scalabile.

Oltre ai file di dimensioni crescenti impiegati per misurare l'andamento dei tempi di elaborazione, sono stati condotti test approfonditi su insiemi di dati eterogenei, caratterizzati da strutture differenti, distribuzioni non uniformi e livelli variabili di qualità informativa. Questa scelta metodologica ha permesso di verificare la robustezza del sistema in scenari realistici, nei quali la natura dei dati può influire in modo significativo sulle prestazioni.

Parallelamente, è stato fatto ricorso a dataset mock appositamente generati per simulare condizioni operative specifiche, come la presenza di valori anomali, campi mancanti o schemi non perfettamente regolari. L'impiego di tali dati sintetici ha consentito di isolare e analizzare il comportamento del sistema in situazioni controllate, garantendo una valutazione più precisa dell'efficacia delle ottimizzazioni introdotte. I risultati ottenuti confermano che il sistema mantiene prestazioni stabili e tempi di risposta prevedibili anche in presenza di dataset complessi o non perfettamente strutturati, evidenziando un'elevata capacità di generalizzazione.

Capitolo 6

Conclusioni

Il lavoro svolto ha portato alla progettazione e allo sviluppo di un sistema avanzato per l'analisi automatizzata di dataset eterogenei, capace di inferire strutture dati, generare schemi SQL coerenti, gestire transazioni complesse e ottimizzare i tempi di elaborazione attraverso tecniche mirate di caching e analisi incrementale dell'output. Le ottimizzazioni introdotte hanno consentito di ridurre sensibilmente i tempi medi di esecuzione, passando da circa tre minuti a un minuto, con un miglioramento significativo dell'efficienza complessiva del processo. La sperimentazione condotta su dataset reali, mock e con dimensioni variabili ha confermato la robustezza, la scalabilità e l'affidabilità del sistema anche in scenari caratterizzati da elevata eterogeneità dei dati.

L'architettura modulare implementata, basata su una chiara separazione tra backend, modello linguistico e interfaccia utente, ha dimostrato di essere flessibile e facilmente estendibile. La pipeline di elaborazione, dalla ricezione dei file alla generazione dello schema SQL e alla visualizzazione dei risultati, si è rivelata stabile e adatta a un utilizzo reale, garantendo un'esperienza d'uso fluida e un'elevata qualità delle informazioni prodotte.

Nonostante i risultati ottenuti siano già rilevanti, il sistema presenta numerose possibilità di evoluzione. Un primo ambito di sviluppo riguarda il completamento della dashboard, con l'integrazione di ulteriori tipologie di file viewer e l'aggiunta di nuovi widget interattivi, così da ampliare le capacità di analisi e visualizzazione dei dati. L'estensione del supporto a formati più complessi, come JSON annidati, XML articolati o dataset provenienti da API esterne, rappresenta un ulteriore passo verso una piattaforma più completa e versatile.

Un'altra prospettiva di particolare interesse riguarda l'introduzione di una funzionalità che consenta agli utenti di applicare algoritmi personalizzati direttamente sui dati elaborati dal sistema. Tali algoritmi, sviluppati

autonomamente o recuperati da fonti esterne, potrebbero essere eseguiti in modo controllato e sicuro, permettendo di effettuare trasformazioni, analisi statistiche, modelli predittivi o procedure di data cleaning specifiche per il contesto applicativo. L'output generato da questi algoritmi potrebbe essere salvato all'interno della transazione stessa, diventando parte integrante del ciclo di vita dei dati e costituendo la base per ulteriori elaborazioni o visualizzazioni.

Ulteriori sviluppi potrebbero includere l'introduzione di meccanismi di validazione automatica dello schema generato, l'implementazione di un sistema di monitoraggio continuo delle prestazioni e l'abilitazione di funzionalità collaborative, come la condivisione delle analisi o la gestione di versioni multiple dello schema. Infine, l'esplorazione di tecniche di elaborazione distribuita potrebbe consentire di gestire dataset ancora più estesi, migliorando ulteriormente la scalabilità del sistema.

Nel complesso, il progetto rappresenta una base solida per la realizzazione di una piattaforma completa e intelligente per l'analisi automatizzata dei dati. Le prospettive di sviluppo individuate aprono la strada a un'evoluzione naturale del sistema, orientata verso una maggiore flessibilità, potenza analitica e adattabilità alle esigenze reali degli utenti.

Bibliografia

- [Bishop, 2015] Bishop, M. (2015). Hypertext transfer protocol version 2 (http/2). Technical Report RFC 7540, IETF. Contesto evolutivo successivo a HTTP/1.1.
- [Bishop, 2022] Bishop, M. (2022). Http/3. Technical Report RFC 9114, IETF. Evoluzione moderna del protocollo basata su QUIC.
- [Boettiger, 2015] Boettiger, C. (2015). An introduction to docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79.
- [Bootstrap Team, 2025] Bootstrap Team (2025). About bootstrap · bootstrap v5.3. <https://getbootstrap.com/docs/5.3/about/overview/>. Documentazione ufficiale su storia, sviluppo e architettura del framework.
- [Brown et al., 2020] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.
- [Chaudhary and Kumar, 2017] Chaudhary, M. and Kumar, A. (2017). Evolution of jquery. In *Practical jQuery*, pages 1–18. Springer.
- [Colvin and Team, 2025] Colvin, S. and Team, P. (2025). *Pydantic Documentation*. Accessed: 2026-06-16.
- [CommonMark Working Group, 2024] CommonMark Working Group (2024). Commonmark specification. <https://spec.commonmark.org/>. Specifiche formali del linguaggio Markdown, base teorica per parser conformi.
- [Cormen et al., 2022] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press. Accessed: 2026-06-16.

- [DeepWiki Contributors, 2024] DeepWiki Contributors (2024). Qwen2.5 model family overview. Accessed: 2026-06-16.
- [Elmasri and Navathe, 2016] Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson, 7th edition.
- [Fielding et al., 1997] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., and Berners-Lee, T. (1997). Hypertext transfer protocol – http/1.1. Technical Report RFC 2068, Internet Engineering Task Force (IETF). Prima introduzione del chunked transfer encoding.
- [Fielding et al., 1999] Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., and Berners-Lee, T. (1999). Hypertext transfer protocol – http/1.1. Technical Report RFC 2616, Internet Engineering Task Force (IETF). Standardizzazione definitiva del chunked transfer encoding.
- [Fielding, 2000] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine. Opera fondamentale per comprendere REST e l’evoluzione di HTTP.
- [Foundation, 2024] Foundation, P. S. (2024). Python virtual environments. <https://docs.python.org/3/library/venv.html>.
- [Fungo, 2023] Fungo, L. (2023). Sheet2sql: An excel to sql script generator. <https://github.com/lucafungo/Sheet2SQL>. Accesso: 2026-06-17.
- [Garcia, 2023a] Garcia, R. F. (2023a). Components in MVP. In *iOS Architecture Patterns*, pages 55–78. Springer Nature.
- [Garcia, 2023b] Garcia, R. F. (2023b). *iOS Architecture Patterns: MVP — Model–View–Presenter*. Springer Nature.
- [GMOD, 2024] GMOD (2024). Gmod: Generic model organism database project. <https://gmod.org>. Accesso: 2026-06-17.
- [Goyal et al., 2023] Goyal, A. et al. (2023). Tool use in machine learning. *arXiv*.
- [Halfond et al., 2006] Halfond, W. G., Viegas, J., and Orso, A. (2006). A classification of sql injection attacks and countermeasures. *Proceedings of the IEEE International Symposium on Secure Software Engineering*.
- [Highlight.js Contributors, 2024] Highlight.js Contributors (2024). Highlight.js github repository. <https://github.com/highlightjs/highlight.js>. Repository ufficiale del progetto, con codice sorgente e documentazione tecnica.

- [Highlight.js Project, 2024] Highlight.js Project (2024). Highlight.js official website. <https://highlightjs.org/>. Sito ufficiale del progetto Highlight.js.
- [Highlight.js Team, 2024] Highlight.js Team (2024). Highlight.js documentation. <https://highlightjs.readthedocs.io/>. Documentazione ufficiale della libreria Highlight.js.
- [Jones and Hardt, 2012] Jones, M. and Hardt, D. (2012). The oauth 2.0 authorization framework: Bearer token usage. RFC 6750.
- [jQuery Foundation, 2023] jQuery Foundation (2023). History of jquery. <https://jquery.com/history/>. Cronologia ufficiale del progetto jQuery.
- [Kahn, 1962] Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562. Accessed: 2026-06-16.
- [Kaplan et al., 2020] Kaplan, J. et al. (2020). Scaling laws for neural language models. *arXiv*.
- [Lin, 2024] Lin, J. (2024). Excel-conversion-tool: A powerful, user-friendly excel conversion tool. <https://github.com/lin-jiaxue/Excel-Conversion-Tool>. Accesso: 2026-06-17.
- [Liu et al., 2023] Liu, P. et al. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods. *ACM Computing Surveys*.
- [markdown-it Contributors, 2024] markdown-it Contributors (2024). markdown-it: Markdown parser done right. <https://github.com/markdown-it/markdown-it>. Repository ufficiale con codice sorgente, benchmark e documentazione tecnica.
- [markdown-it Team, 2024] markdown-it Team (2024). markdown-it documentation. <https://markdown-it.github.io/>. Documentazione ufficiale della libreria markdown-it.
- [Merkel, 2014] Merkel, D. (2014). Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239).
- [MITRE Corporation, 2024] MITRE Corporation (2024). Cwe-89: Improper neutralization of special elements used in an sql command ('sql injection'). Accessed: 2026-06-15.
- [MongoDB Inc., 2024] MongoDB Inc. (2024). *MongoDB Manual*. Documentazione ufficiale di MongoDB.

- [Naqvi, 2024] Naqvi, S. (2024). Xcelsql: Sql-powered excel transformation engine. <https://github.com/naqvis/XcelSQL>. Accesso: 2026-06-17.
- [Ollama Project, 2024a] Ollama Project (2024a). Ollama documentation. Accessed: 2026-06-16.
- [Ollama Project, 2024b] Ollama Project (2024b). Ollama github repository. Accessed: 2026-06-16.
- [Ouyang et al., 2022] Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. *arXiv*.
- [OWASP Foundation, 2023] OWASP Foundation (2023). Sql injection. Accessed: 2026-06-15.
- [Packagist Contributors, 2024] Packagist Contributors (2024). *SimpleXLSX on Packagist*. Distribuzione ufficiale tramite Composer, con informazioni su versione e dipendenze.
- [Pandas Development Team, 2025] Pandas Development Team (2025). *Pandas Documentation*. Accessed: 2026-06-16.
- [Passerini, 2025] Passerini, A. (2025). Scientific programming lecture a07 - pandas. Materiale didattico basato sulla documentazione ufficiale.
- [Qwen Team, 2024] Qwen Team (2024). Qwen2.5-coder-7b-instruct. Accessed: 2026-06-16.
- [Ramírez, 2025] Ramírez, S. (2025). *FastAPI Documentation*. Accessed: 2026-06-16.
- [Resig, 2011] Resig, J. (2011). Building jquery. *IEEE Internet Computing*. Discussione sulla nascita e l’impatto di jQuery.
- [Russell and Norvig, 2021] Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson.
- [Salatti, 2024] Salatti, D. (2024). Agentic design patterns. Accessed: 2026-06-16.
- [Schick et al., 2023] Schick, T. et al. (2023). Toolformer: Language models can teach themselves to use tools. *arXiv*.
- [SciNote, 2024] SciNote (2024). Scinote: Open source electronic lab notebook. <https://github.com/scinote-elnl/scinote-web>. Accesso: 2026-06-17.

- [Sedgewick and Wayne, 2011] Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley. Accessed: 2026-06-16.
- [SENAITE, 2024] SENAITE (2024). Senaite: Open source lims for serious laboratories. <https://www.senaite.com>. Accesso: 2026-06-17.
- [Shapiro et al., 2024] Shapiro, S., Pearson, D., and Gonzalez Venegas, E. (2024). Pandas for reproducible data analysis: From spreadsheets to research-grade python workflows. *arXiv*.
- [SheetJS Contributors, 2024] SheetJS Contributors (2024). Sheetjs spreadsheet data toolkit. <https://git.sheetjs.com/SheetJS/sheetjs>. Repository ufficiale con codice sorgente, documentazione e specifiche tecniche.
- [SheetJS Team, 2026] SheetJS Team (2026). Api reference – sheetjs community edition. <https://docs.sheetjs.com/docs/api/>. Riferimento tecnico alle API di parsing, scrittura e utilità.
- [Shuchkin, 2024a] Shuchkin, S. (2024a). Simplexlsx api reference. Technical report, Deep Documentation Archive. Documentazione tecnica avanzata su metodi, struttura interna e gestione degli errori.
- [Shuchkin, 2024b] Shuchkin, S. (2024b). *SimpleXLSX: Lightweight PHP Excel 2007+ Reader*. Repository ufficiale e documentazione primaria del progetto SimpleXLSX.
- [Simon, 1955] Simon, H. A. (1955). A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118.
- [White et al., 2023] White, J. et al. (2023). Prompt engineering for large language models: Beyond the basics. *arXiv*.
- [Wikipedia contributors, 2022] Wikipedia contributors (2022). jquery. <https://en.wikipedia.org/wiki/JQuery>. Statistiche di utilizzo e informazioni storiche.
- [Wikipedia contributors, 2025] Wikipedia contributors (2025). Bootstrap (framework). [https://en.wikipedia.org/wiki/Bootstrap_\(front-end_framework\)](https://en.wikipedia.org/wiki/Bootstrap_(front-end_framework)). Storia, versioni e caratteristiche del framework.
- [Wooldridge, 2021] Wooldridge, M. (2021). *An Introduction to MultiAgent Systems*. Wiley.

- [Yao et al., 2022] Yao, S. et al. (2022). React: Synergizing reasoning and acting in language models. In *Proceedings of the NeurIPS 2022 Workshop*.
- [Zareie-Vaux and Abernethy, 2024] Zareie-Vaux, F. and Abernethy, G. (2024). Aurora: a free, open-source laboratory information management system (lims) using r shiny. <https://github.com/fvaux/aurora>. Accesso: 2026-06-17.
- [Zhang and Luo, 2010] Zhang, Y. and Luo, Y. (2010). An architecture and implement model for model-view-presenter pattern. *2010 International Conference on Computer Application and System Modeling*, pages V1–615–V1–619.
- [Zhou et al., 2023] Zhou, Y. et al. (2023). A comprehensive survey on prompt engineering. *arXiv*.

Ringraziamenti

Desidero esprimere la mia sincera gratitudine al prof. Bonnici, mio relatore, per la disponibilità, la professionalità e il costante supporto dimostrati durante tutto il percorso, nonché per l'aiuto prezioso nella stesura di questa tesi.

Ringrazio profondamente i miei genitori, per essere sempre stati presenti e per avermi sostenuto in ogni momento.

Un ringraziamento speciale va a mio fratello, per il supporto che non mi ha mai fatto mancare.

Ringrazio con affetto i miei nonni, per la loro presenza e vicinanza.

Un grazie di cuore a Francy e Rimo, che da ormai dodici anni condividono con me un'amicizia autentica, fatta di presenza costante e di tanta pazienza nel sopportarmi ogni giorno.

Ringrazio Marco, con cui ho sviluppato questo applicativo, per la collaborazione, la disponibilità dimostrata in ogni momento in cui sorgesse un problema durante lo sviluppo e per essere sempre stato presente in questi tre anni.

Un sentito ringraziamento a Diego, per aver discusso con me le idee di sviluppo e per aver dedicato tempo ed energie alla revisione completa della tesi.

Infine, ringrazio Clim, Ben, Omar, Hodor e Silviu: compagni di questi tre anni intensi, con cui ho condiviso studio, crescita e la preparazione agli esami.